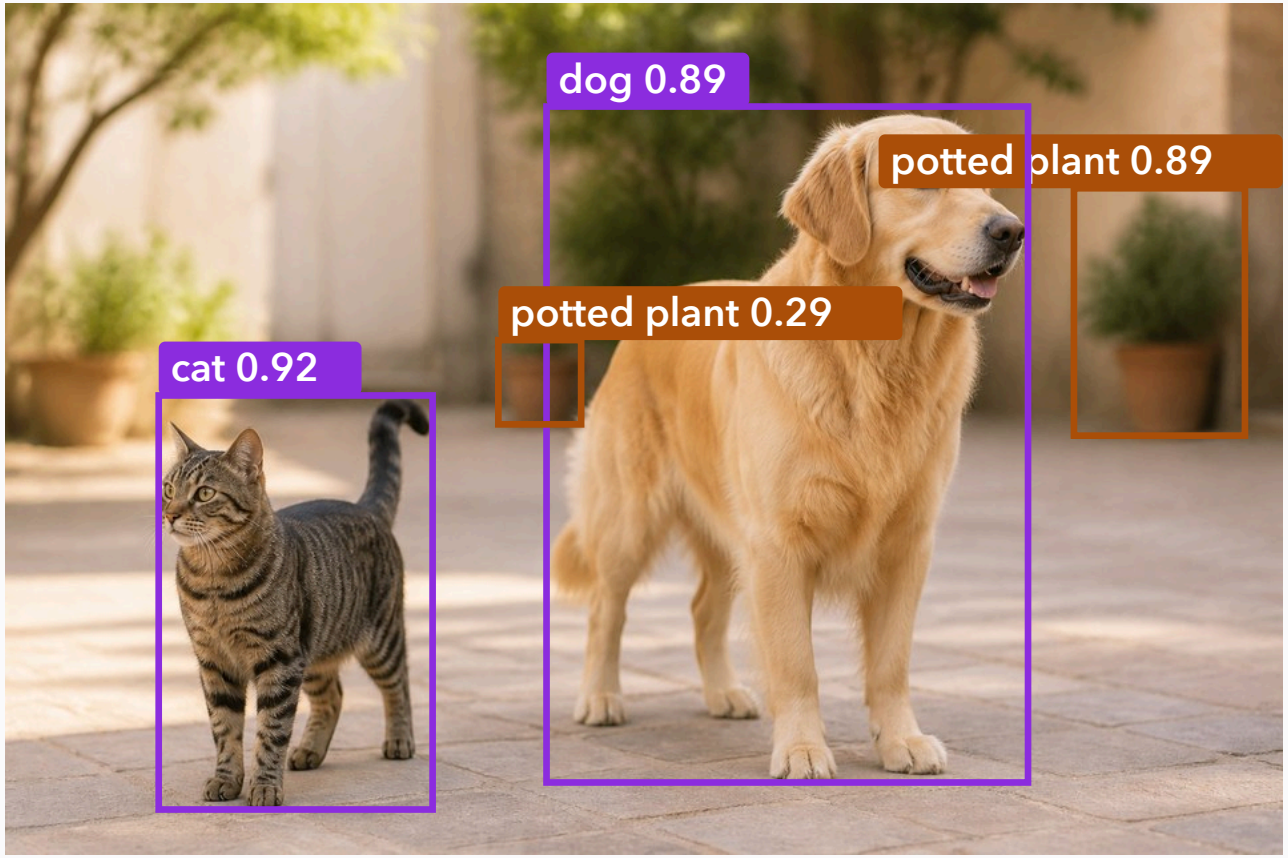
Object detection: from one label to a set of objects

A detector looks at a photo and returns a list: one entry per object, each a class and a box. How can a network with a fixed-size output return a list whose length changes from photo to photo?

ES 667 Deep Learning · Lecture 22
Nipun Batra · IIT Gandhinagar



> The real output of a small detector, YOLOv8n, on today's photo: four objects, each with a class, a score and a box.

YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json.

▶ Notebook, part 1

Part 1 · Why detect objects?

A camera becomes useful when it can say what is where. Start with six jobs that need exactly that.

1. For each of six photos, say what you would want to know, then see the answer.
2. See that every answer has the same shape, and what it is used for.
3. Three things that make the job hard.

> Six jobs that need the same kind of answer, and three reasons it is hard to give.

▶ Notebook, part 1

On the road: who is where?



find people, cyclists, vehicles, animals

use brake or steer; follow each one from frame to frame

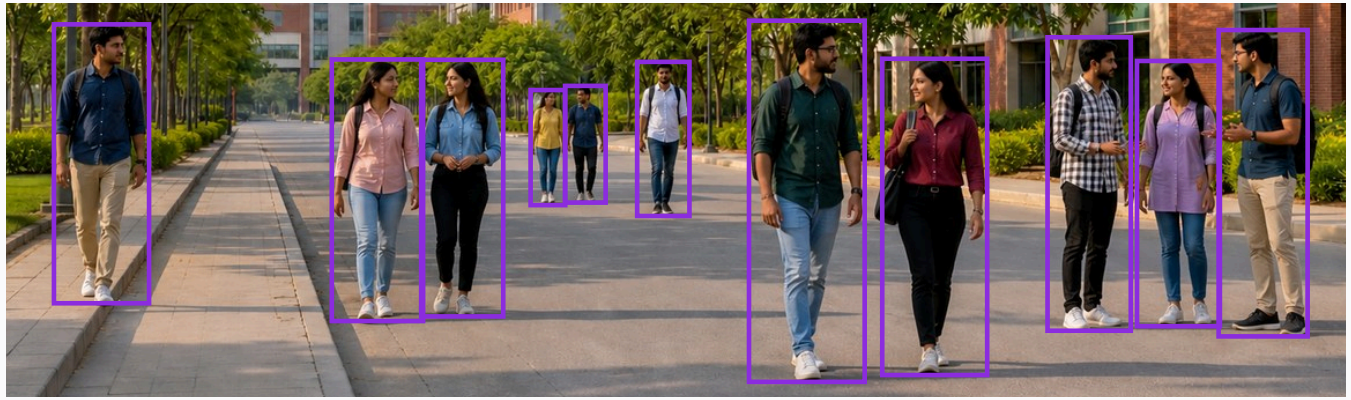
US cars that brake automatically for pedestrians had 27% fewer pedestrian crashes, but no fewer on dark, unlit roads (IIHS, 2022).

YOLOv8n: 4 people, a car, a bicycle and a dog, each boxed and scored. The car acts on this list.

YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json.

▶ Notebook, part 1

In a crowd: how many people?



find people

use count them; warn when a place gets too crowded

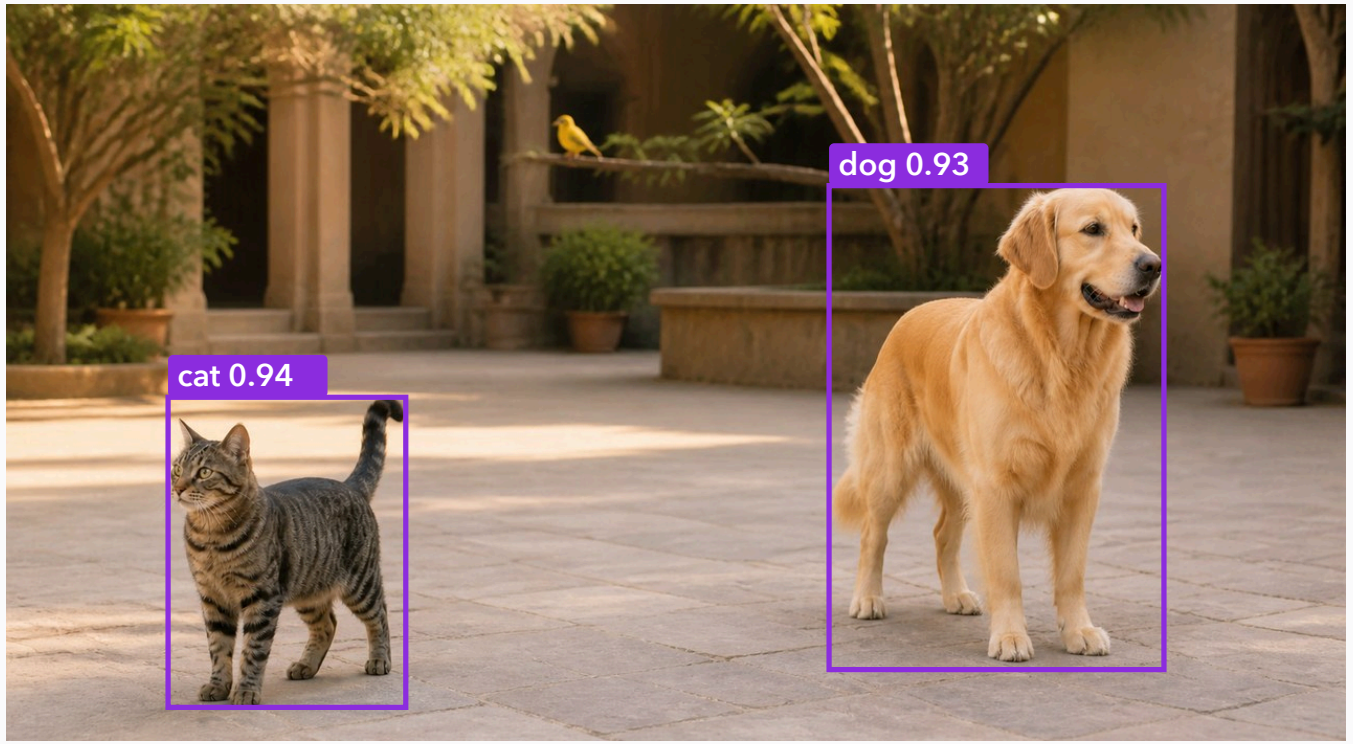
Stations, stadiums and large gatherings count people camera by camera to spot dangerous crowding early.

YOLOv8n boxes 11 people. The count is the list's length, and each entry also says where.

YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json.

▶ Notebook, part 1

At a camera trap: which animals walked past?



find animals by species, people, vehicles

use count each species; skip the empty photos

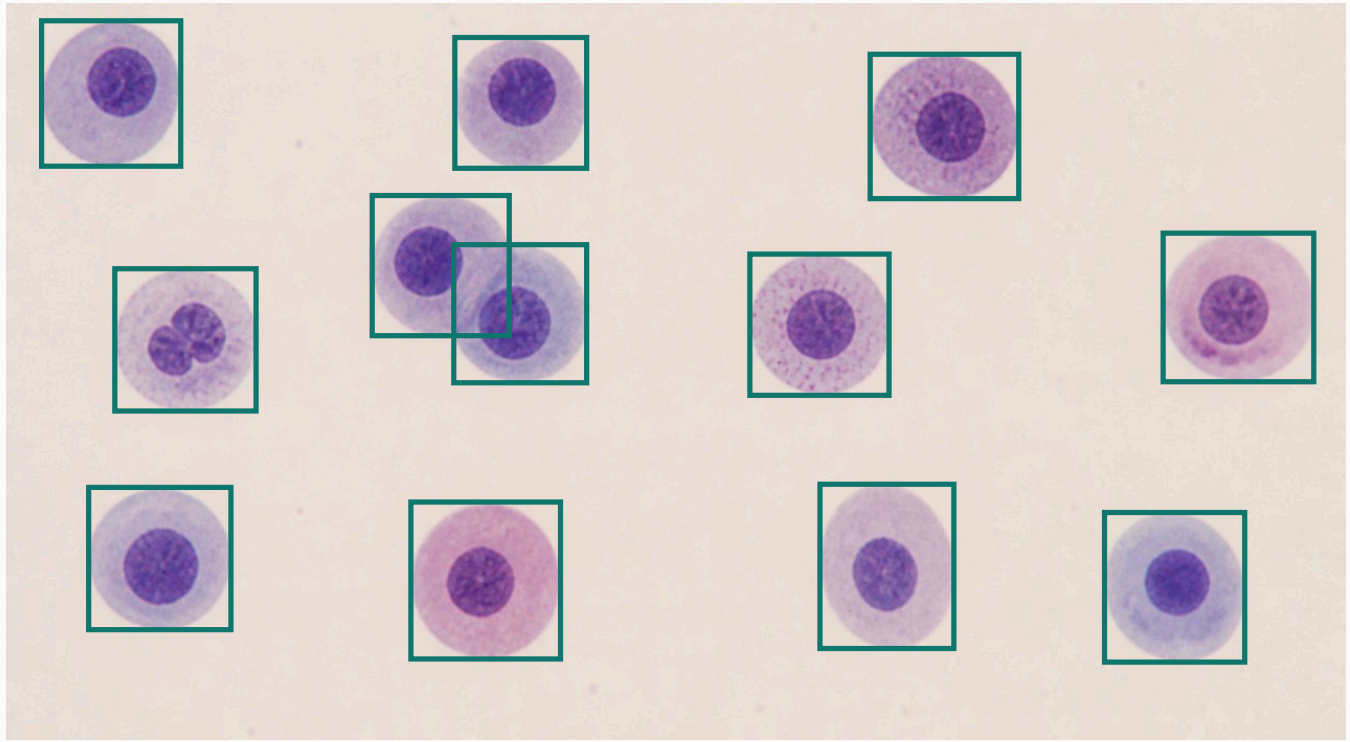
In an Arctic study of 548,627 camera-trap photos, a workflow built on MegaDetector left only 4 to 9% for people to check (Celis et al., 2024).

> YOLOv8n: a cat (0.94) and a dog (0.93). Did it find every animal? We come back to this.

YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json.

▶ Notebook, part 1

Under a microscope: count the cells



find cells, by type

use count them; crop each one and classify its type

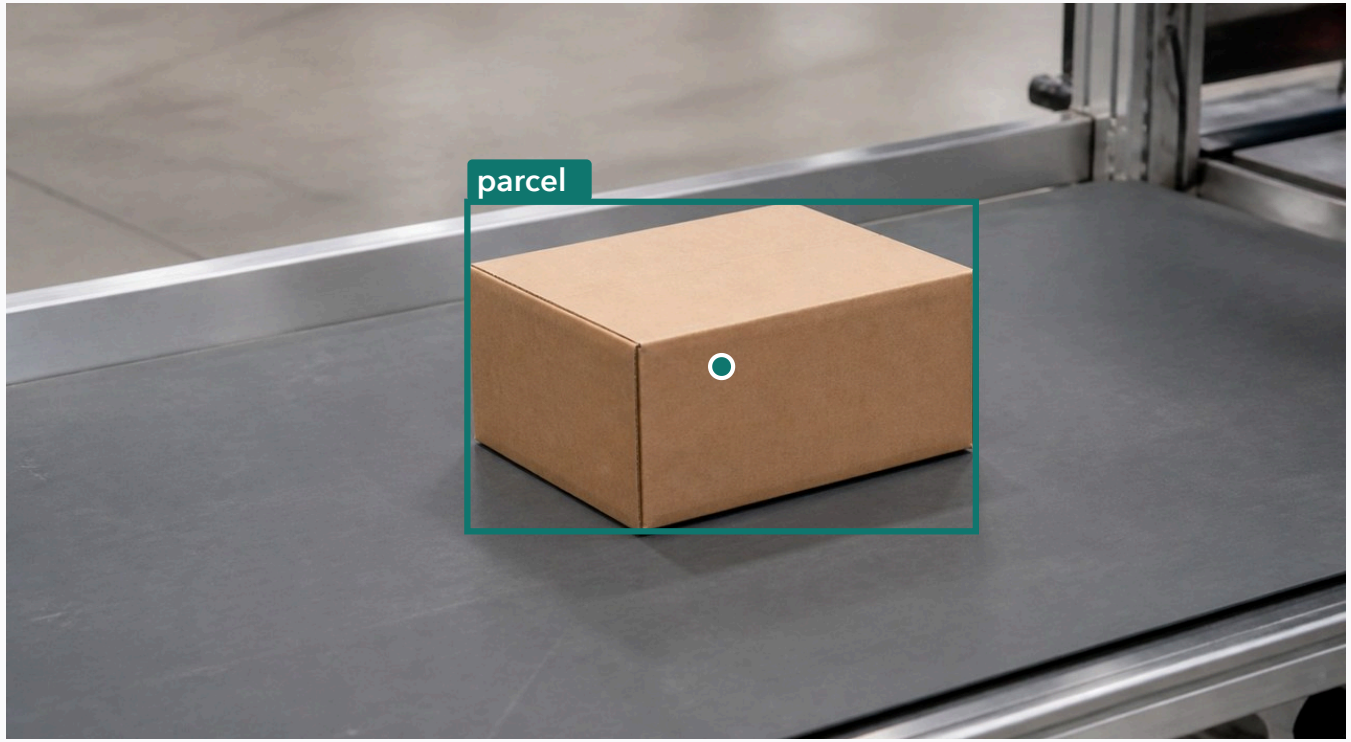
Cell detectors learn from lists like this: the 2018 Data Science Bowl's 841 images carry 37,333 nuclei, each marked by hand (Nature Methods, 2019).

> 12 cells. These teal boxes are ours: COCO has no class "cell", so no detector in this lecture knows one.

Boxes drawn by us (data/applications.json): COCO has no such class, so no detector here can find it. The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 1

In a warehouse: where is the parcel?



find parcels, sacks, envelopes

use aim the gripper at the box; count parcels per hour

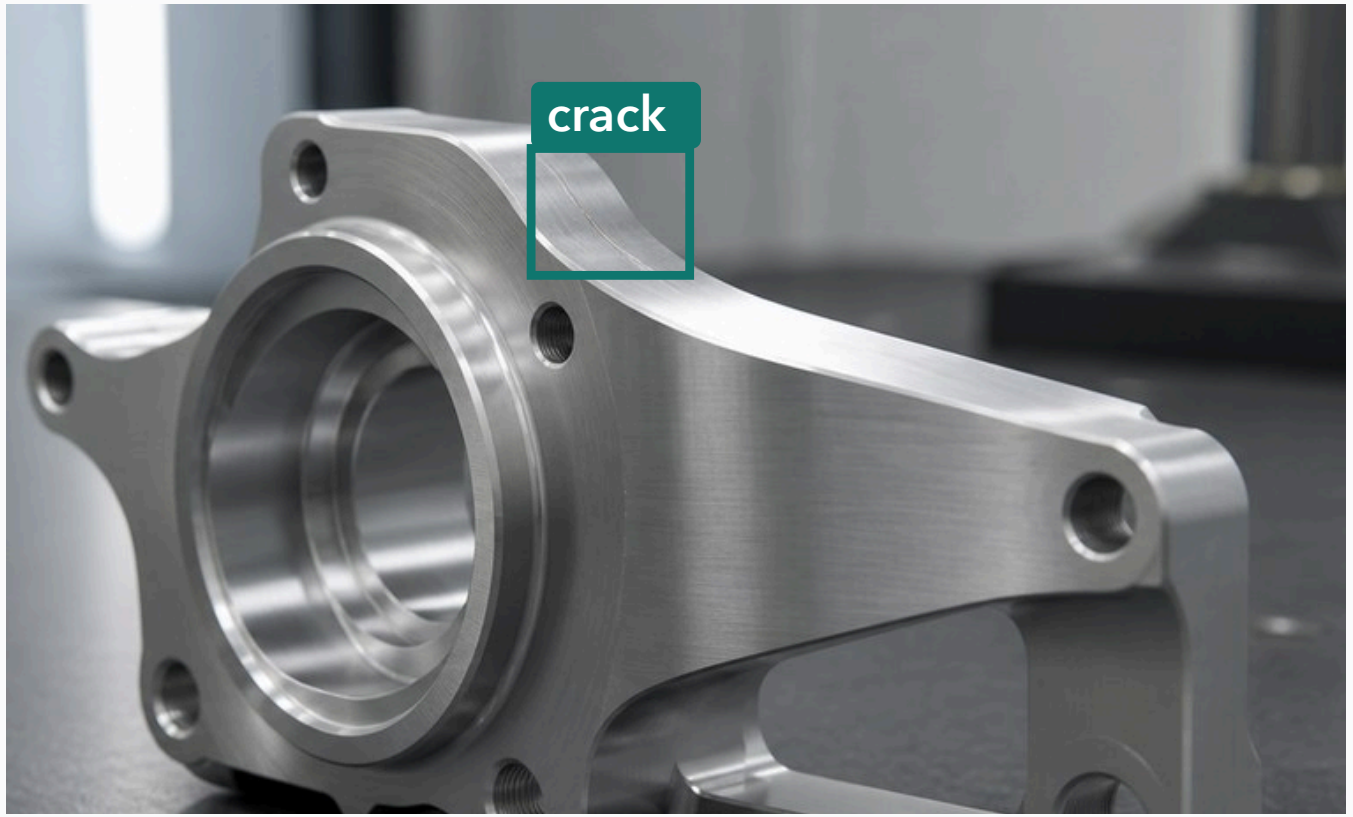
A harder relative: the SKU-110K photos of shop shelves hold about 147 boxed products each, packed edge to edge (Goldman et al., 2019).

One entry: a parcel, 486 × 315 pixels. The arm aims at the box's centre, the dot.

Boxes drawn by us (data/applications.json): COCO has no such class, so no detector here can find it. The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 1

On a production line: is anything wrong?



find cracks, scratches, dents, missing holes

use reject the part; show the inspector where to look

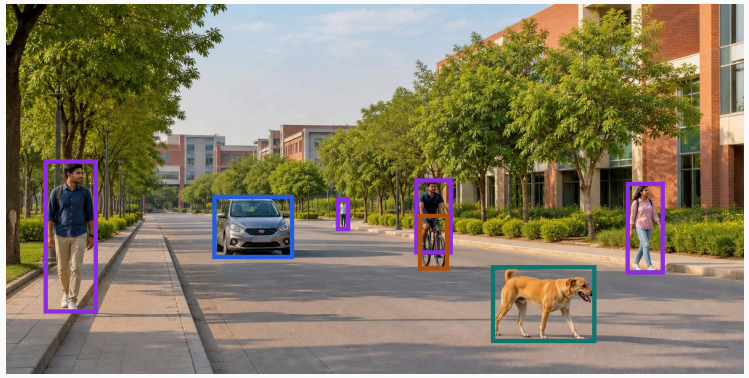
Inspection data sets look like this list: NEU-DET labels six kinds of steel-surface defect, 300 photos of each, every defect boxed.

> A hairline crack on the arm, in a box 90 × 72 pixels: the inspector knows exactly where to look.

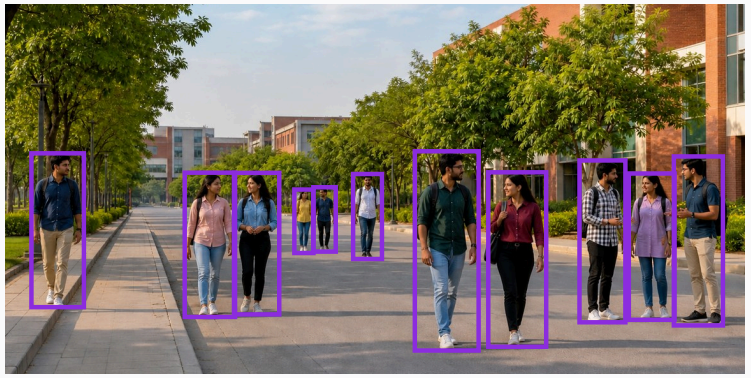
Boxes drawn by us (data/applications.json): COCO has no such class, so no detector here can find it. The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 1

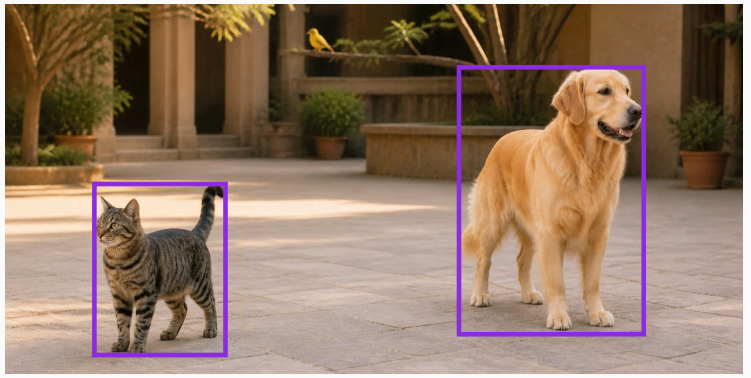
Six jobs, one kind of answer



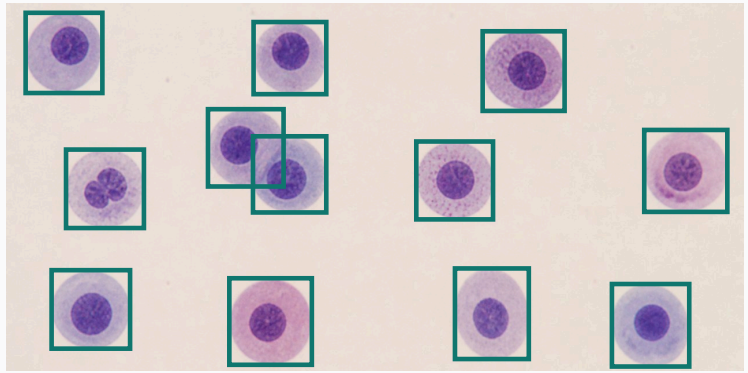
road · **7** objects



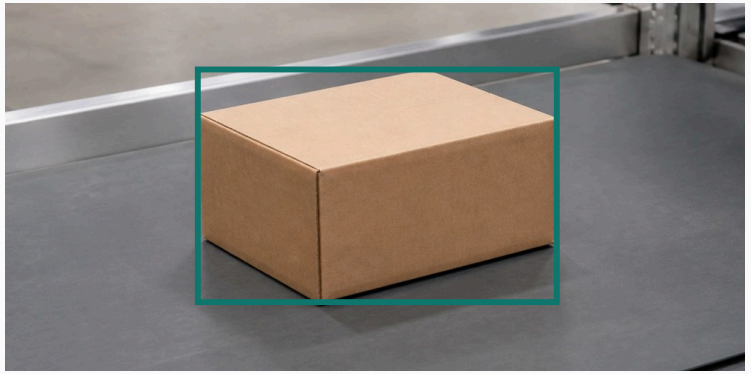
crowd · **11** people



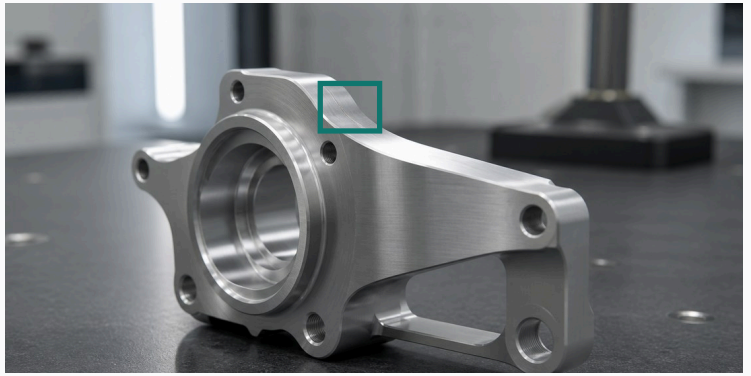
camera trap · **2** animals



microscope · **12** cells



warehouse · **1** parcel



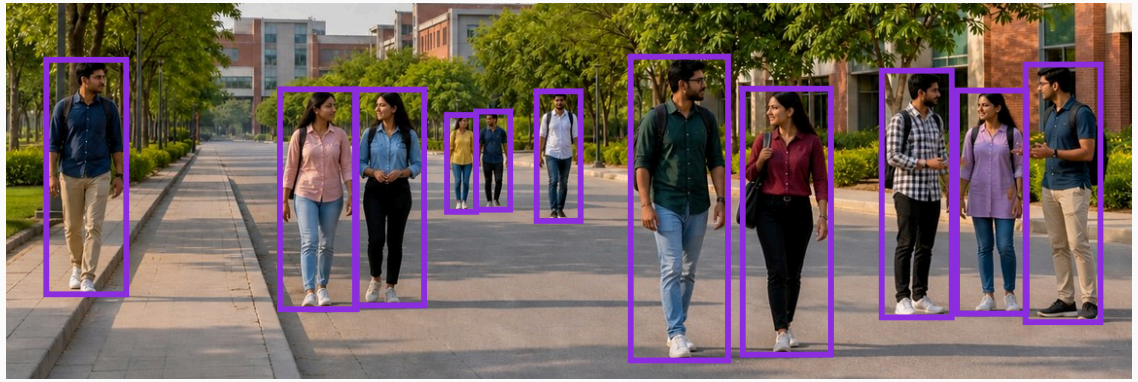
factory · **1** crack

> Three more: 12 cells, 1 parcel, 1 crack (teal: our annotation). Every answer has the same shape: a list, one entry per object, each a class and a box.

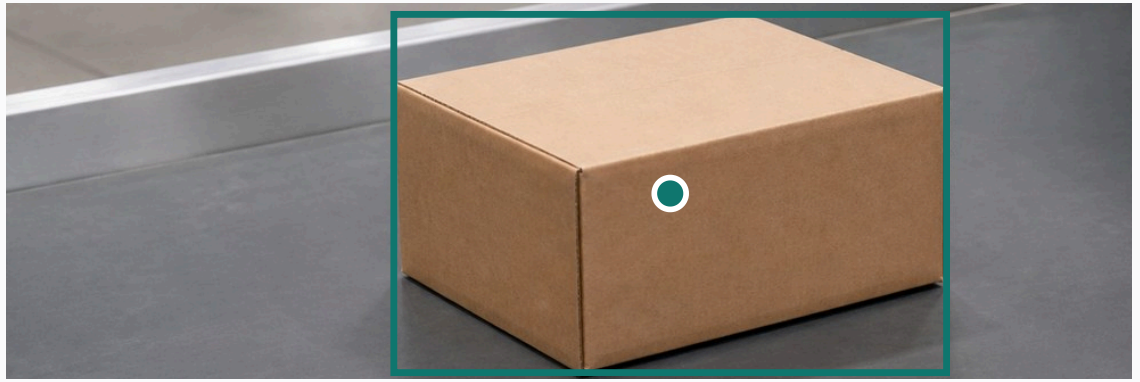
YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json. Cells, parcel and crack: our annotation.

▶ Notebook, part 1

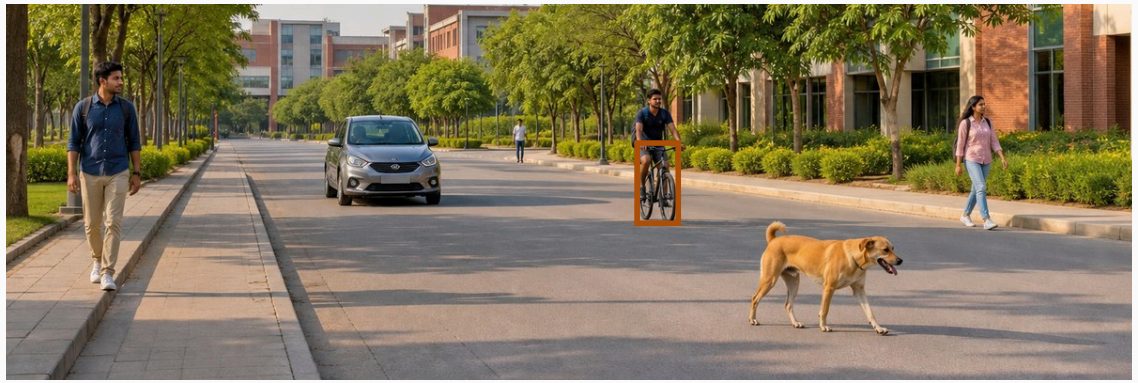
What the list is used for



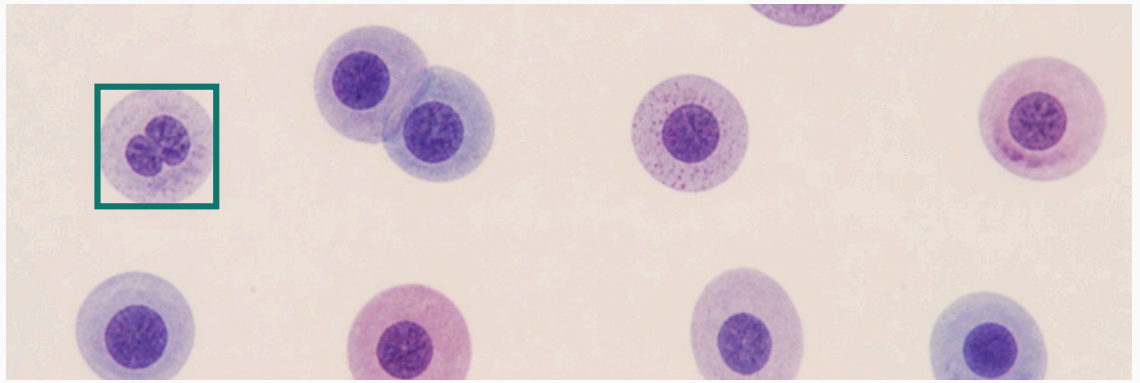
count · 11 people



locate · grip at the box's centre



track · link each box to the next frame's



crop · then name the cell's type

> Crop: cut out each box and pass it to a second model: read a number plate, identify a face, name a cell's type.

YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json. Cells and parcel: our annotation.

▶ Notebook, part 1

Hard, 1: a bus and a distant person in one photo



The bus is 502 pixels wide.

The far pedestrian: 13 pixels, 39 times narrower.

> And a pedestrian far down the same road. A classifier looks for one main thing; a detector must box both, at very different sizes.

Boxes: our annotation (data/annotations.json). The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 1

Hard, 2: small objects are easy to miss



The bird is 50 × 43 pixels. One cell did draw a box on it, IoU **0.913** with ours, but scored it **0.098**.

Below the usual cut-off of 0.25, so the box is thrown away.

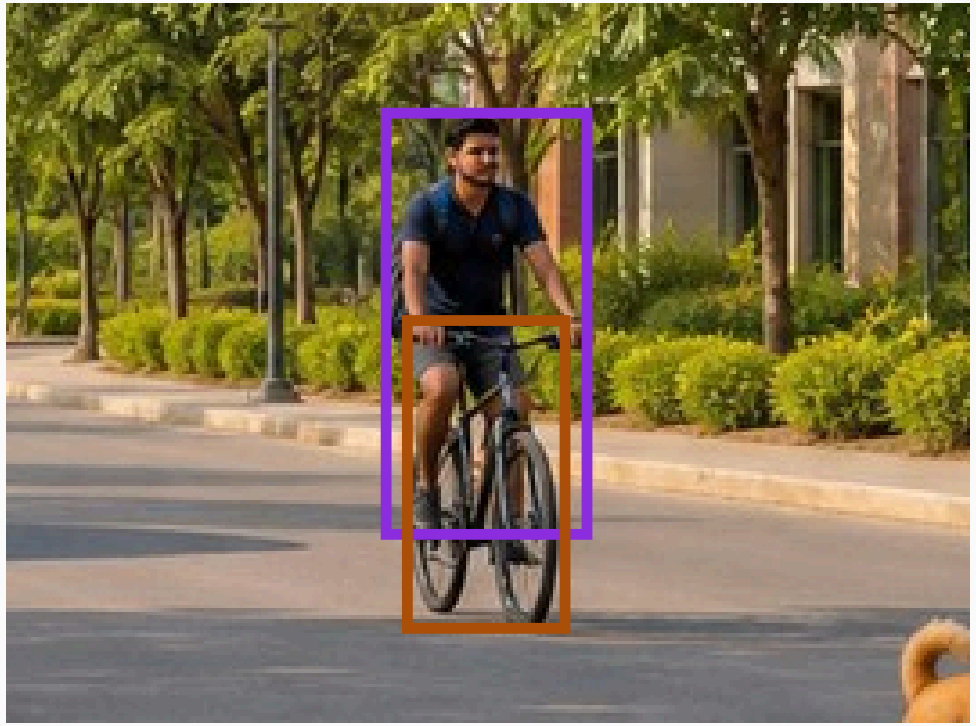
Its next guess for that cell: banana, 0.059, also small and yellow.

> With so few pixels the network has little detail to go on, so small objects get low, unsure scores.

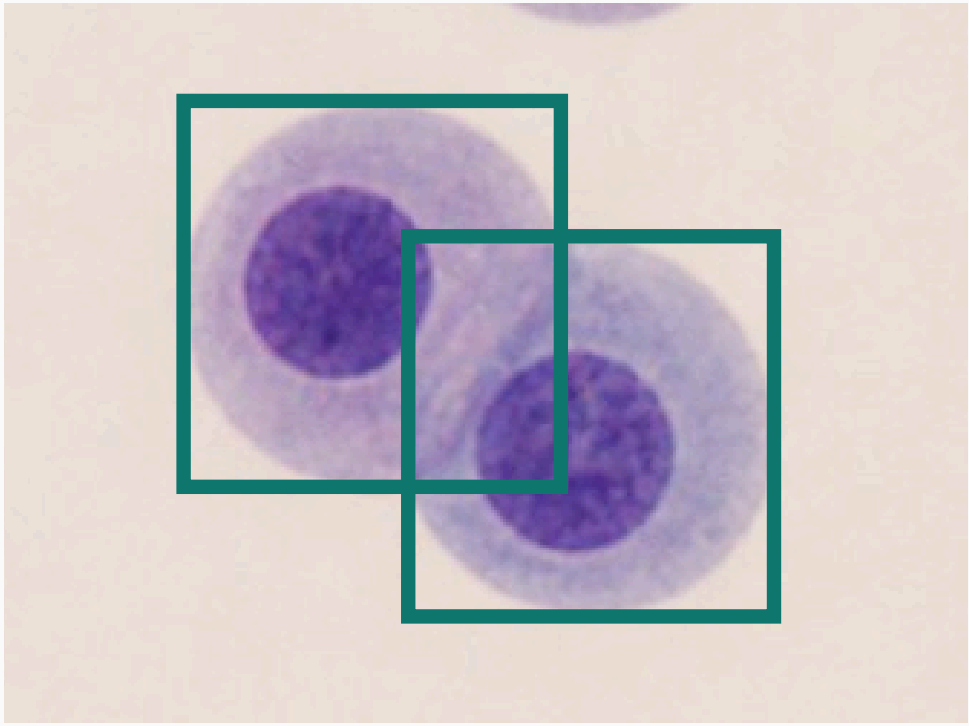
YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json.

▶ Notebook, part 1

Hard, 3: objects overlap



a cyclist: a person and a bicycle



two touching cells

> Two touching cells: their boxes overlap too. A detector must keep both entries, and yet must not report one object twice.

YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json. Cells: our annotation.

Part 1 in one slide

1. Detection answers what is where: one (class, box, score) per object.
2. The same kind of answer works for six very different jobs.
3. The list is used to count, locate, track and crop.
4. Hard: sizes vary 40×; small objects get unsure scores; objects overlap.

Next: why a classifier cannot give this list, and what a network would have to output.

▶ Notebook, part 2

Part 2 · From one label to a list

A classifier answers with one word. We need a list. Why is a list hard for a network?

1. What a classifier says about one photo, and what we want instead.
2. Two awkward facts about the list: its length changes, and its order means nothing.
3. The route through the rest of the lecture.

> A classifier gives one label for the whole photo; detection needs one entry per object.

▶ Notebook, part 2

What is in this photo?



ImageNet class	probability
golden retriever	0.192
flat-coated retriever	0.015
tiger cat	0.013
Labrador retriever	0.012
tabby	0.010

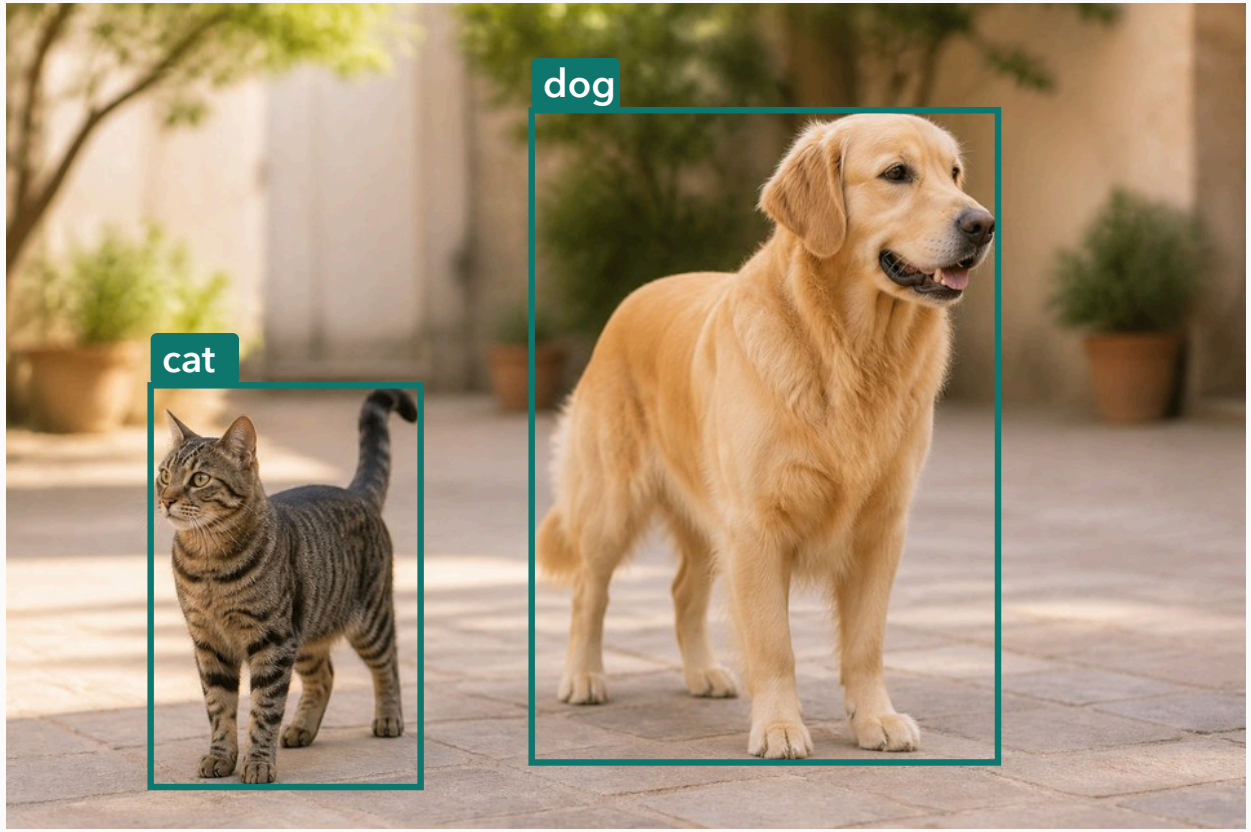
Its 118 dog classes together hold 0.39; the five cat classes, 0.03.

> The cat appears only as a weak runner-up, tiger cat at 0.013. The answer cannot say “a dog and a cat”, how many, or where.

ResNet-50 with torchvision’s IMAGENET1K_V2 weights, run on this photo (scripts/run_models.py).

▶ Notebook, part 2

What we want: a list, one entry per object



class	box: left, top, right, bottom (pixels)
dog	(407, 83, 768, 588)
cat	(112, 296, 321, 607)

The boxes are our annotation of this photo, in pixels from its top-left corner.

> Second entry: cat. Each object gets one entry, so a photo with five objects gets five.

The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 2

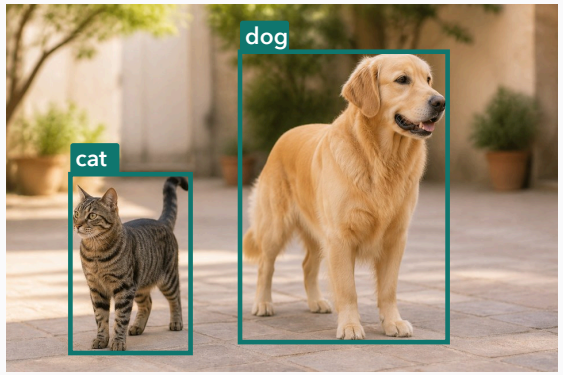
Four tasks, four kinds of answer



Classification: one label



Localisation: one label, one box



Detection: a class and a box per object



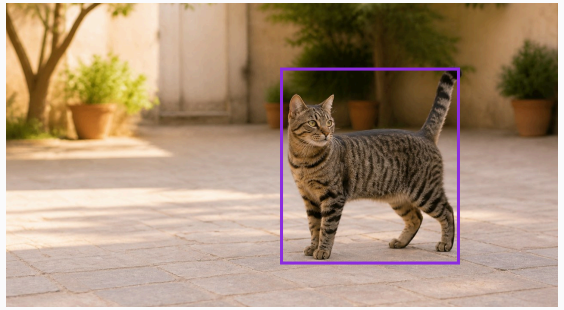
Segmentation: which pixels (Lecture 23)

> Segmentation: which pixels belong to each object. The masks here are from a real YOLOv8n-seg run; this is next lecture.

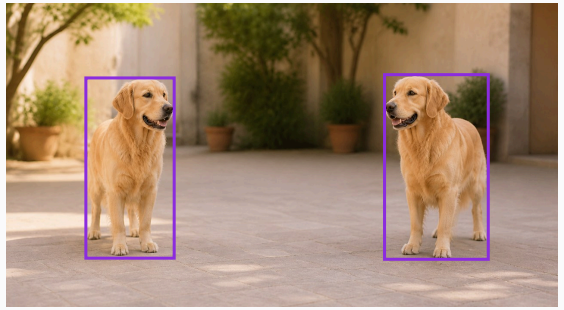
Masks: YOLOv8n-seg run on the photo (scripts/run_models.py). The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 2

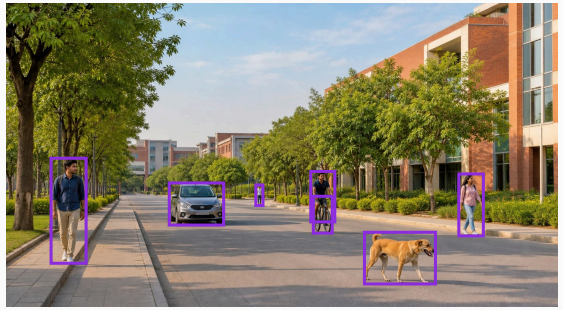
The number of objects changes from photo to photo



1 · one cat



2 · two dogs



7 · a street



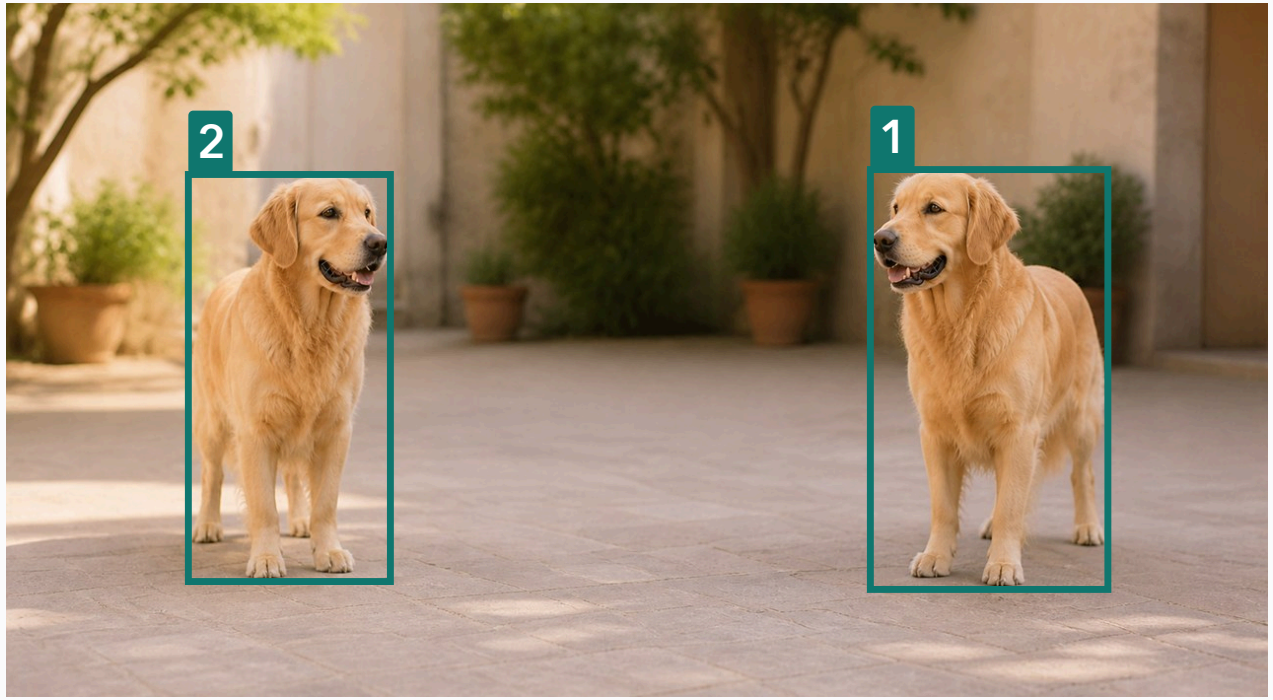
11 · a path of people

A busy path: eleven people. The length of the answer depends on the photo.

YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json.

▶ Notebook, part 2

A list with no order: a set



$$Y = \{(c_1, b_1), \ldots, (c_N, b_N)\}$$

Number the dogs either way and the answer is the same. What must not change is which class goes with which box.

> Or right one first. Both lists describe the same scene: the answer is a set of N (class, box) pairs, N varies, and order means nothing.

The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

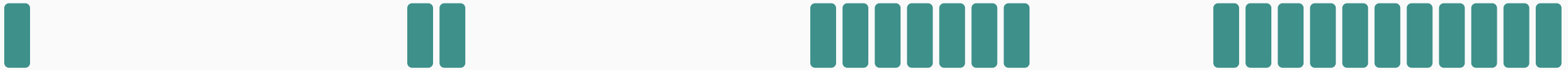
▶ Notebook, part 2

A network's output has a fixed shape

a classifier: always 1,000 numbers (20 drawn), for every photo



the answer we need: N entries, N = 1, 2, 7, 11, … in any order



N = 1 N = 2 N = 7 N = 11

Today's question: **how can a fixed-size output describe a set of any size, in no order?**

> That is the question of the lecture.

▶ Notebook, part 2

Today's route: build a detector step by step

3. **Boxes and IoU**: four numbers for a box; IoU to compare two
4. **One box, then many**: one box works; several break the loss
5. **A candidate at every place**: every grid cell writes a box and class scores
6. **NMS**: keep one box per object
7. **Three grids**: small and large objects
8. **Measuring**: precision, recall and AP

Appendix A: DETR, slots matched to objects, no NMS.

> Then YOLO's answer: a candidate at every place, cleaned up by NMS, at three sizes; and how to measure it.

▶ Notebook, part 3

Part 3 · Describe a box, compare two boxes

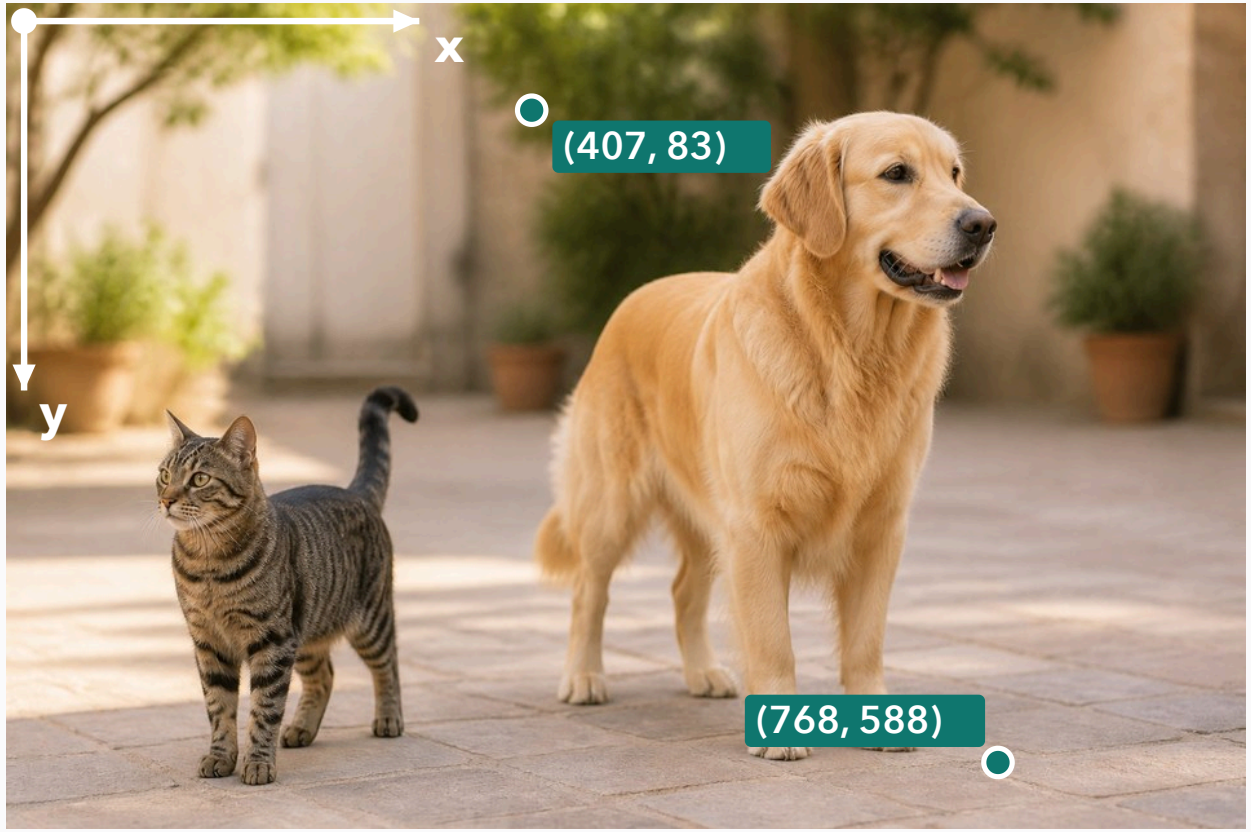
Every later step needs two small tools.

1. Write a box as four numbers, in two common formats, and see how a labelled photo is stored.
2. Measure how well two boxes agree: intersection over union, IoU, computed by hand.
3. Try IoU live, then write it in six lines of PyTorch.

> Two tools: four numbers for a box, and IoU to compare two boxes.

▶ Notebook, part 3

Positions in a photo: pixels from the top-left corner



The photo is 960 pixels wide and 640 tall. A point is (x, y): x counts right from the left edge, y counts **down** from the top.

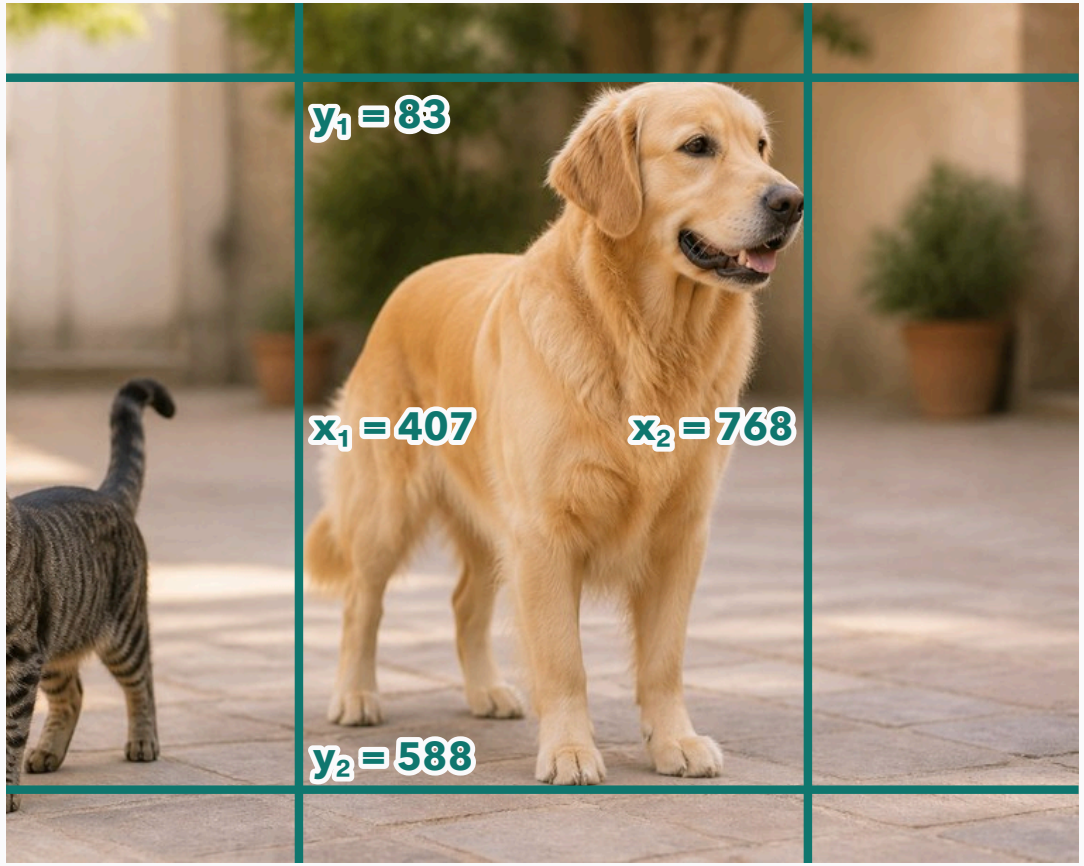
Two corners of the dog: top-left (407, 83), bottom-right (768, 588).

> Two corners of the dog's box. The bottom-right one has the larger y, because it is lower in the photo.

The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 3

A box is four numbers: its left, top, right and bottom edges



Corner format:

$$(x_1,\ y_1,\ x_2,\ y_2)$$

$$= (407,\ 83,\ 768,\ 588)$$

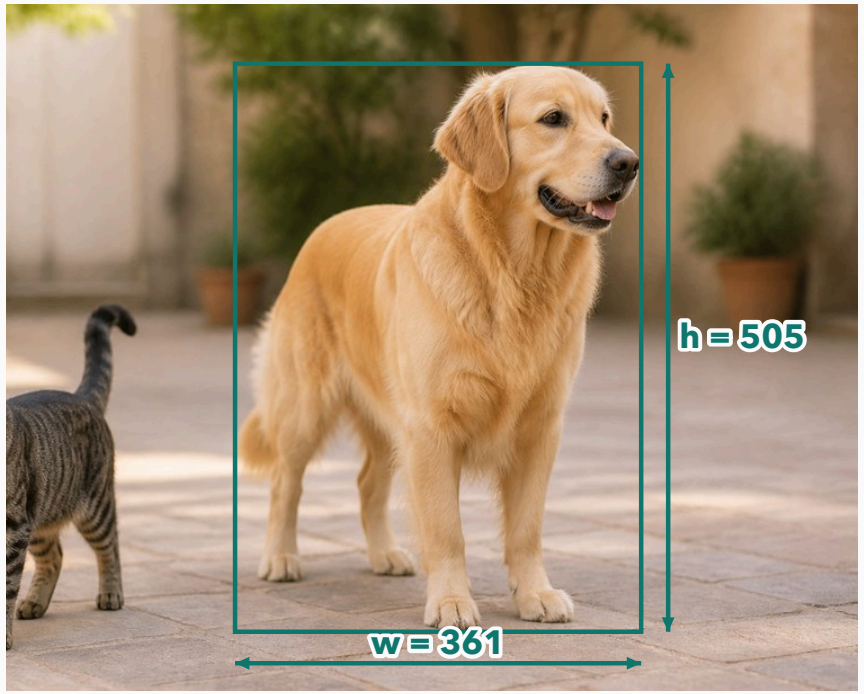
(x_1, y_1) is the top-left corner, (x_2, y_2) the bottom-right. torchvision uses this format.

> Bottom edge: y at most 588, the paws. Four numbers: (407, 83, 768, 588).

The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 3

The box's size: width and height



$$w = x_2 - x_1 = 768 - 407 = 361$$

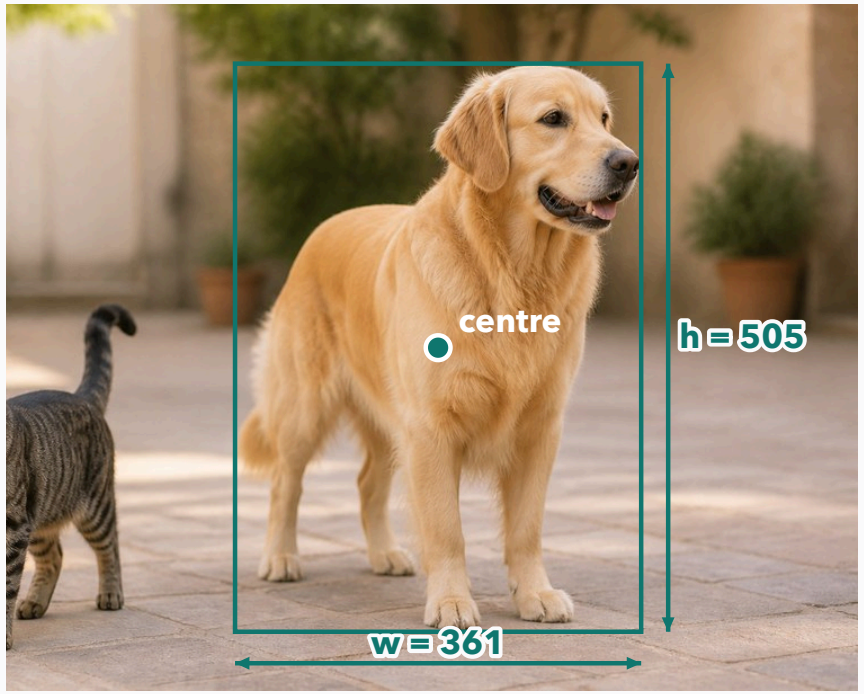
$$h = y_2 - y_1 = 588 - 83 = 505$$

> Height: the bottom minus the top, 588 − 83 = 505. Bottom minus top, because y grows downwards.

The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 3

The same box as a centre and a size



$$c_x = x_1 + \frac{w}{2} = 407 + \frac{361}{2} = 587.5$$

$$c_y = y_1 + \frac{h}{2} = 83 + \frac{505}{2} = 335.5$$

$$(c_x, c_y, w, h) = (587.5,\ 335.5,\ 361,\ 505)$$

> Same box, second format: centre (587.5, 335.5), size 361 × 505. Either format can be computed from the other.

The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 3

Divide by the photo's size: numbers between 0 and 1

$$\left(\frac{c_x}{W}, \frac{c_y}{H}, \frac{w}{W}, \frac{h}{H}\right) = \left(\frac{587.5}{960}, \frac{335.5}{640}, \frac{361}{960}, \frac{505}{640}\right)$$

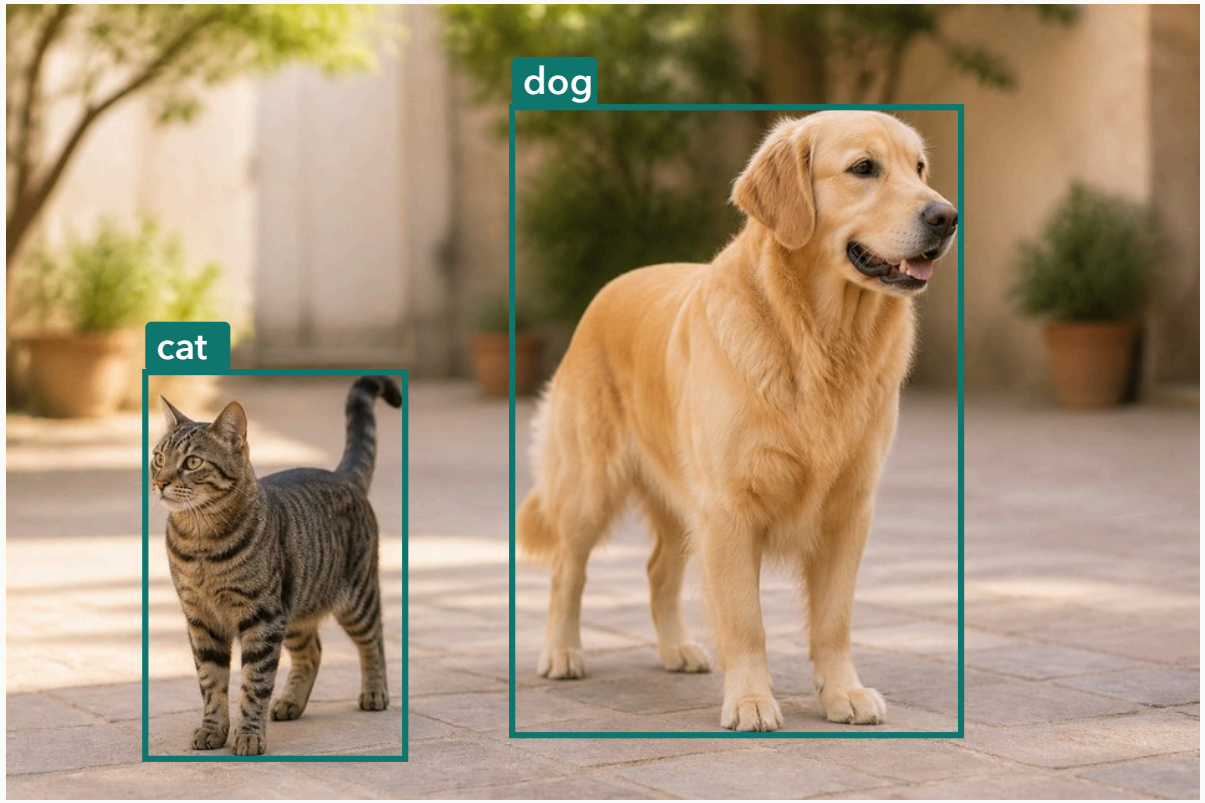
$$= (0.612, 0.524, 0.376, 0.789)$$

Halve the photo to 480 × 320: every pixel number halves, the normalised numbers do not change. The dog's centre is 61% of the way across and 52% of the way down, at any resolution.

> These numbers survive resizing, which is why label files and many networks use them.

▶ Notebook, part 3

A labelled photo on disk: one line per object



`cat-dog.txt`, next to `cat-dog.jpg`:

```
16 0.612 0.524 0.376 0.789
15 0.226 0.705 0.218 0.486
```

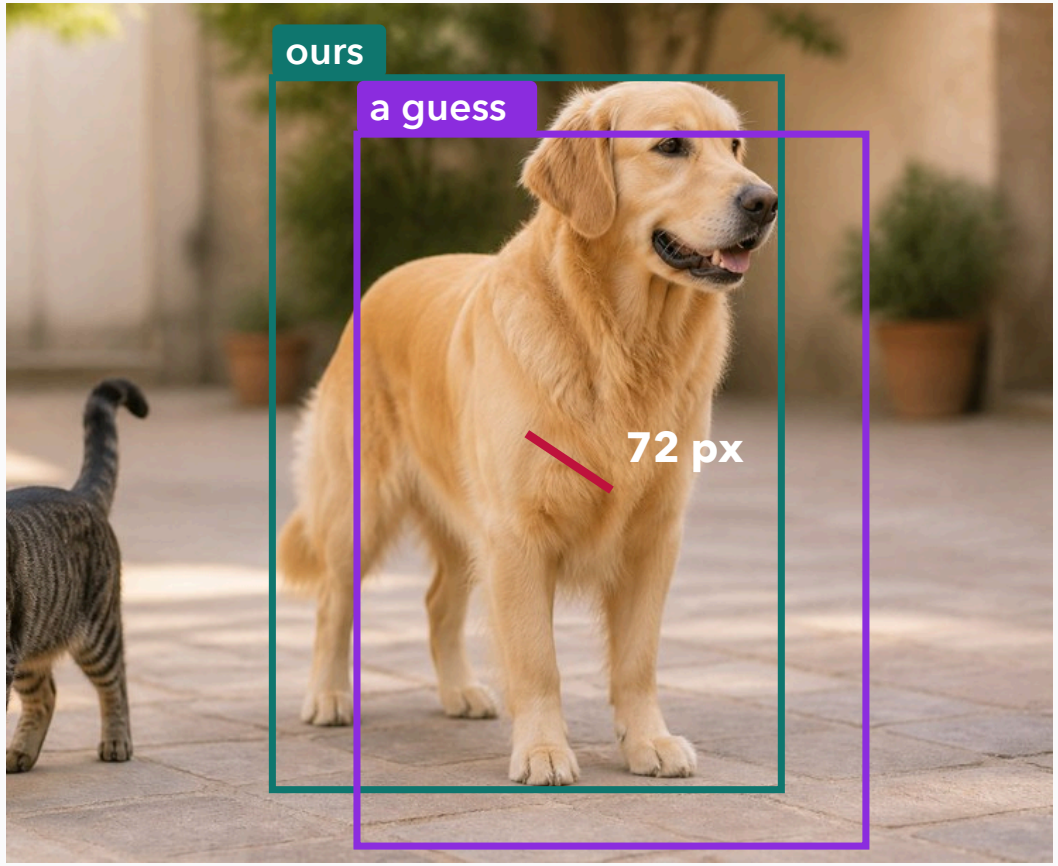
	class	c_x	c_y	w	h
line 1	16 = dog	0.612	0.524	0.376	0.789
line 2	15 = cat	0.226	0.705	0.218	0.486

A class number (COCO counts from 0: 15 is cat, 16 is dog), then the normalised centre and size. A detector learns to write these lines.

Our annotation of the photo (data/annotations.json), written in the YOLO label format.

▶ Notebook, part 3

Two boxes for one dog: how well do they agree?



Teal: our box. Purple: a guess, moved 60 px right and 40 px down.

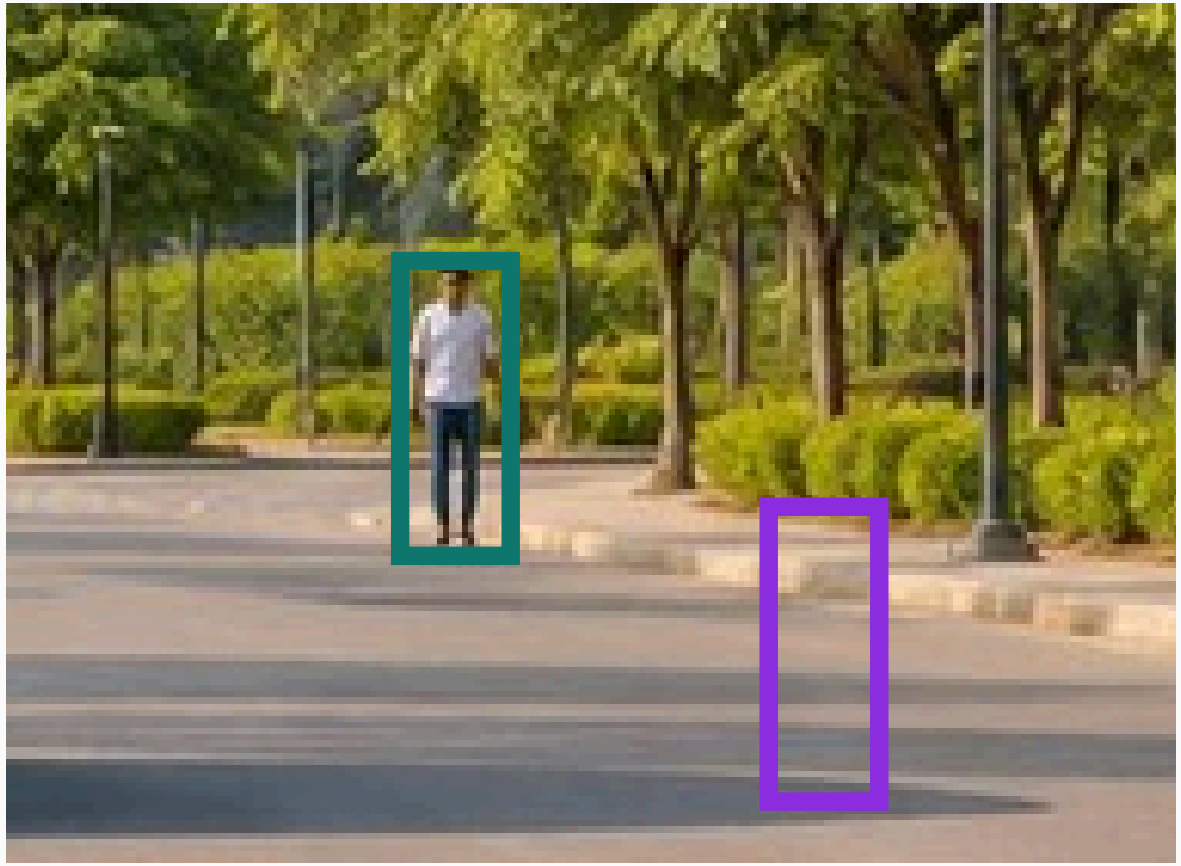
Idea 1: the distance between the centres, $\sqrt{(60^2 + 40^2)} = 72$ px.

> One idea: measure how far apart the centres are. Here 72 px. Is that a lot?

The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 3

The same slip on a small object



A person far down the street, 18 px wide.

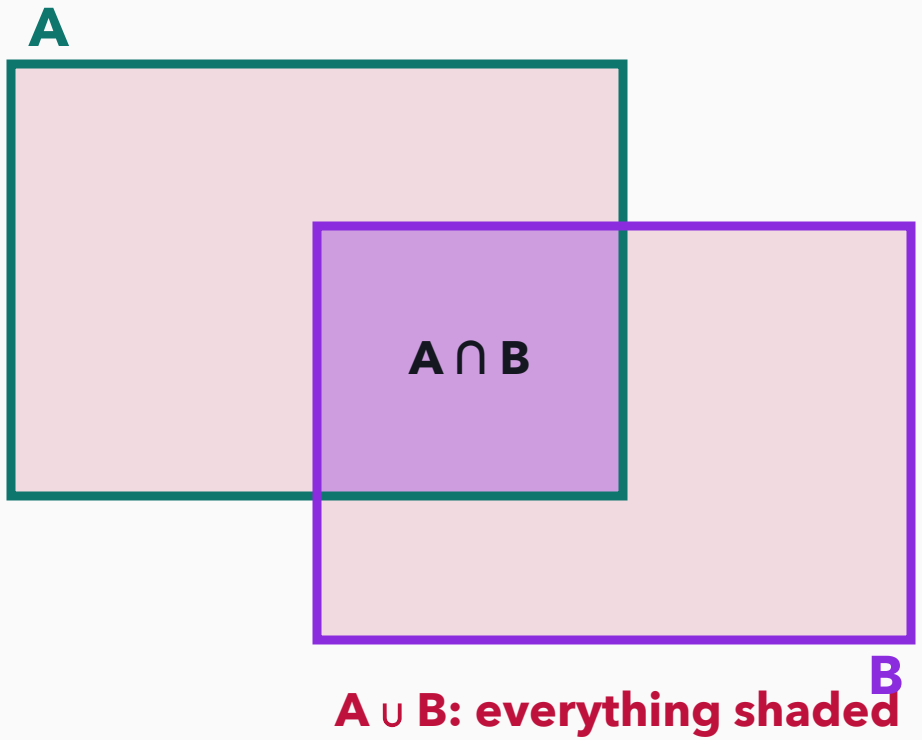
Move the box by the same 60 px right and 40 px down: it no longer touches the person.

> The same 72-pixel slip misses the person entirely. A small error for the dog is a miss here, so compare overlap instead of distance.

The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 3

Intersection over union (IoU)



Intersection: the area covered by both boxes.

Union: the area covered by at least one of them.

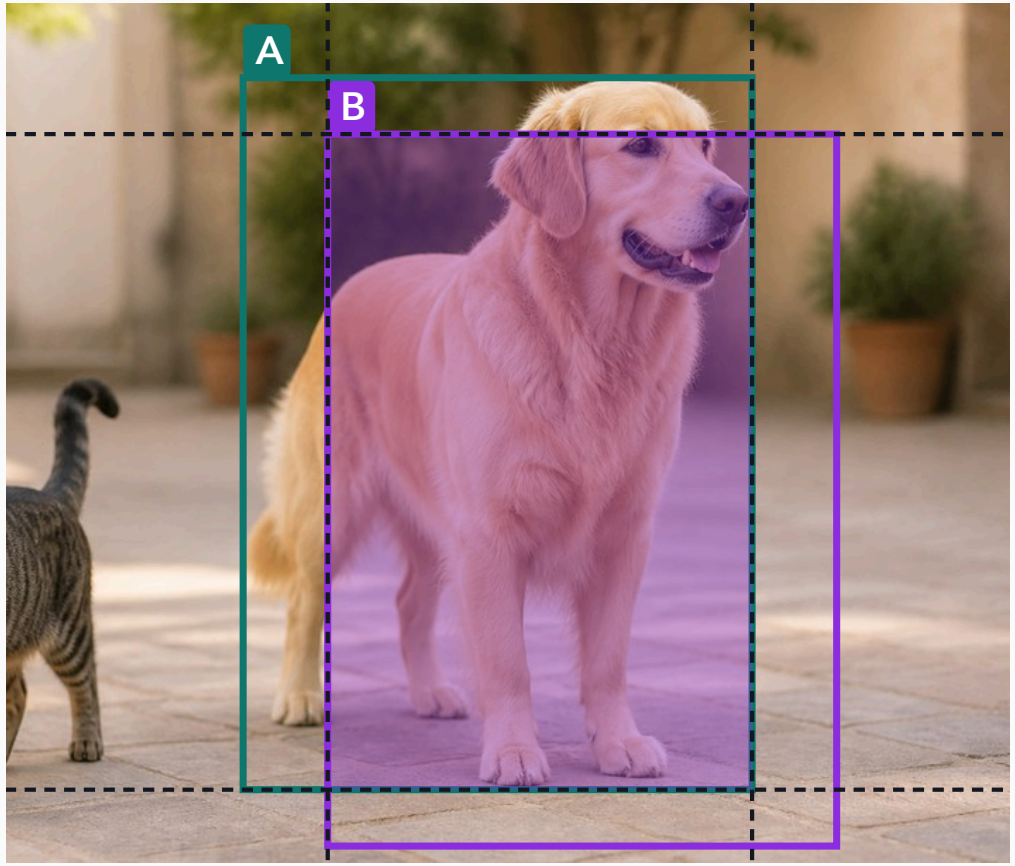
$$\mathrm{IoU}(A, B) = \frac{\mathrm{area}(A \cap B)}{\mathrm{area}(A \cup B)}$$

Identical boxes: IoU = 1. Boxes that do not touch: IoU = 0.

IoU divides the first by the second. It is a fraction of area, so it does not care how big the boxes are.

▶ Notebook, part 3

IoU, step 1: the overlap rectangle



$$\text{left} = \max(407,\ 467) = 467$$

$$\text{top} = \max(83,\ 123) = 123$$

$$\text{right} = \min(768,\ 828) = 768$$

$$\text{bottom} = \min(588,\ 628) = 588$$

$$\text{area} = 301 \times 465 = 139{,}965$$

> Its area: 301 × 465 = 139,965 square pixels. (If a width came out negative, the boxes would not touch: area 0.)

The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 3

IoU, step 2: the union, then the ratio

$$\text{area}(A) = 361 \times 505 = 182{,}305, \qquad \text{area}(B) = 182{,}305$$

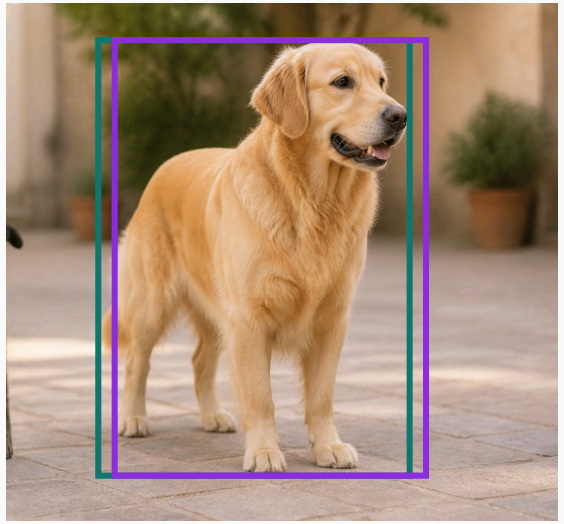
$$\text{union} = 182{,}305 + 182{,}305 - 139{,}965 = 224{,}645$$

$$\text{IoU} = \frac{139{,}965}{224{,}645} = 0.623$$

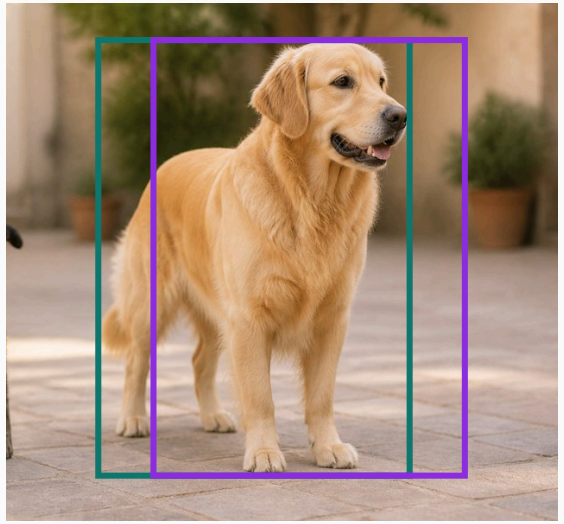
> IoU = 0.623. A move of 60 px right and 40 px down cost more than a third of the score.

▶ Notebook, part 3

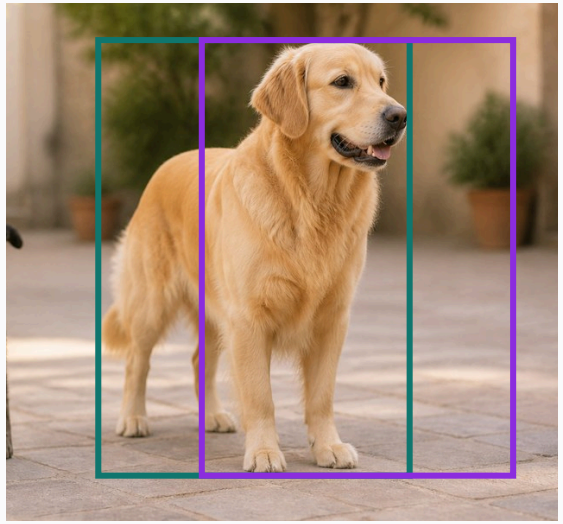
What IoU values look like



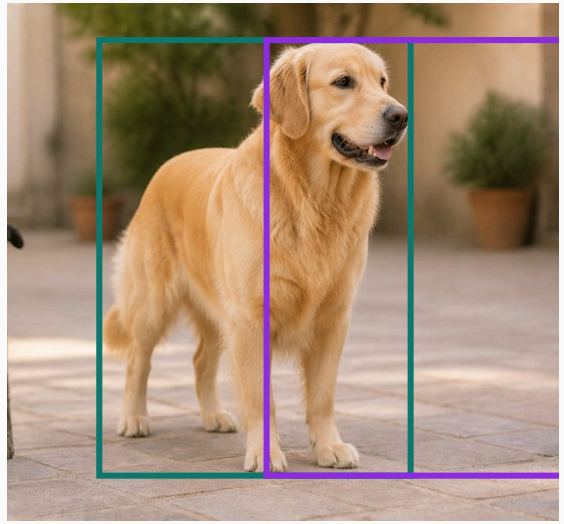
IoU 0.9 · moved 19 px • **hit**



IoU 0.7 · moved 64 px • **hit**



IoU 0.5 · moved 120 px • **hit**



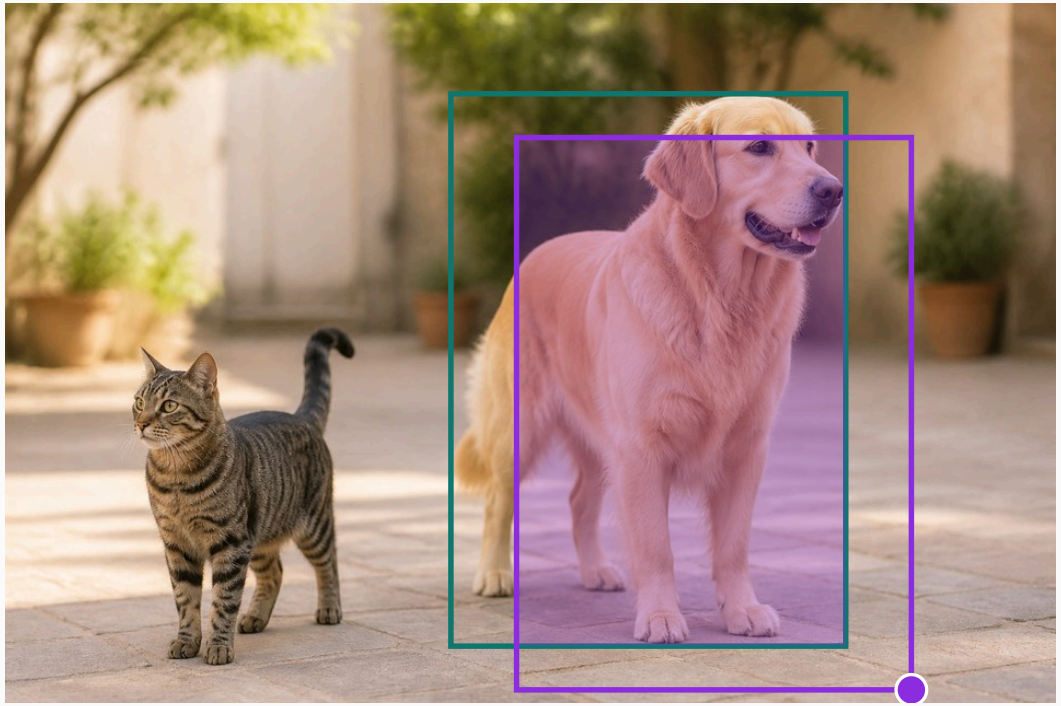
IoU 0.3 · moved 194 px • **miss**

> Most benchmarks call IoU ≥ 0.5 a hit. A trained detector does much better on this dog: 0.983.

YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json.

▶ Notebook, part 3

Try it: drag a box over the dog



overlap	139,965 px²
union	224,645 px²
IoU	**0.623**

> Drag the purple box. Can you find a box with IoU 0.5 that looks good? One with IoU 0.7 that looks bad?

▶ Notebook, part 3

IoU in code: every pair at once

```python
def box_iou(a, b):                  # a: (N, 4), b: (M, 4), corners
    lt = torch.max(a[:, None, :2], b[None, :, :2])  # overlap's top-left
    rb = torch.min(a[:, None, 2:], b[None, :, 2:])  # overlap's bottom-right
    inter = (rb - lt).clamp(min=0).prod(-1)         # (N, M); 0 if apart
    area = lambda x: (x[:, 2:] - x[:, :2]).prod(-1)
    return inter / (area(a)[:, None] + area(b)[None, :] - inter)
```

```python
box_iou(torch.tensor([[407., 83., 768., 588.]]), torch.tensor([[467., 123., 828., 628.]]))
# tensor([[0.6230]])  -- torchvision.ops.box_iou gives the same
```

> On our two boxes: 0.6230. torchvision.ops.box_iou returns the same N × M matrix.

Part 3 in one slide

1. A box is four numbers: corners (407, 83, 768, 588), or centre and size.
2. A label file holds one line per object: a class and four numbers.
3. IoU = overlap ÷ union: 1 for the same box, 0 for boxes apart. The 60-px guess: 0.623.
4. IoU ≥ 0.5 is the usual “found it”.

> **Next:** ask a network for one box, then several, and see where it breaks.

▶ Notebook, part 4

Part 4 · One box is easy; several are not

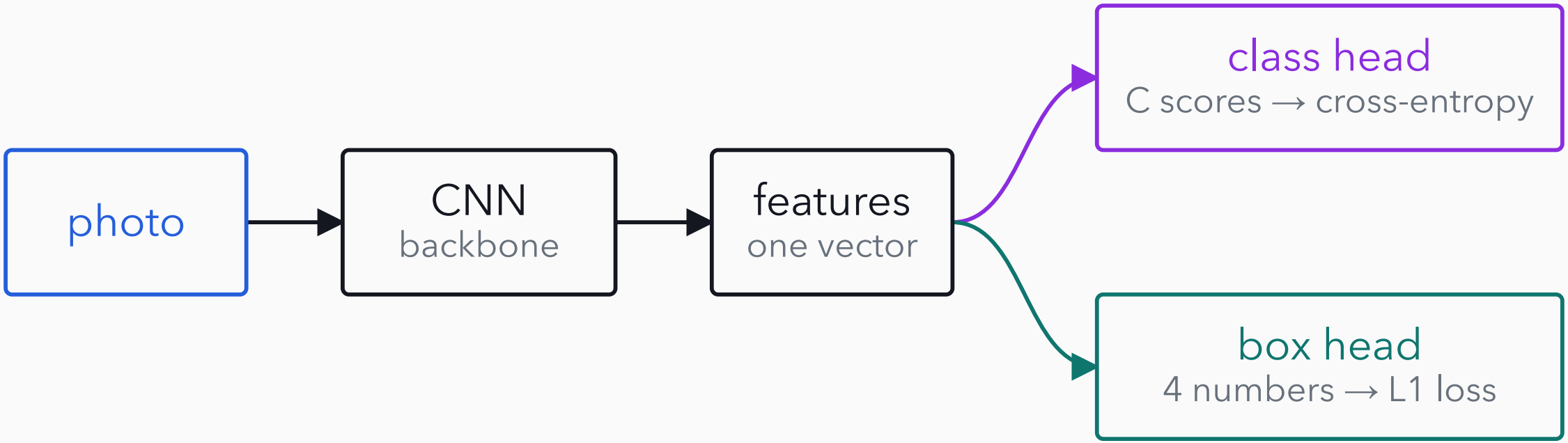
A network can output four numbers for one box. Asking it for several boxes shows the real problem.

1. Add a box head to a classifier; write the target it learns, with and without an object.
2. Ask the same head for two animals, and see it hedge.
3. Give it K slots: count and order break the loss.

> One box is plain regression. Several boxes raise a new question: which output answers for which object?

▶ Notebook, part 4

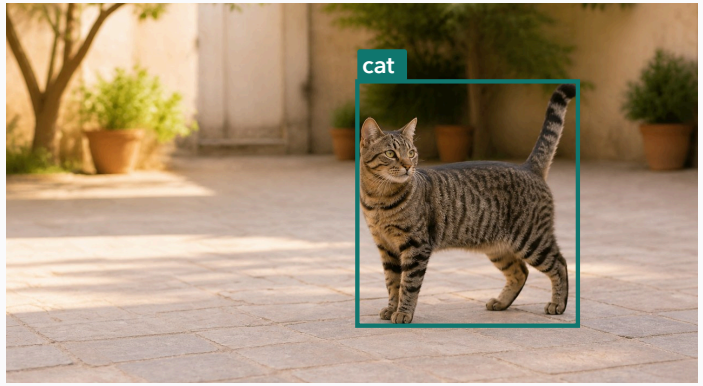
One object: a box head beside the class head



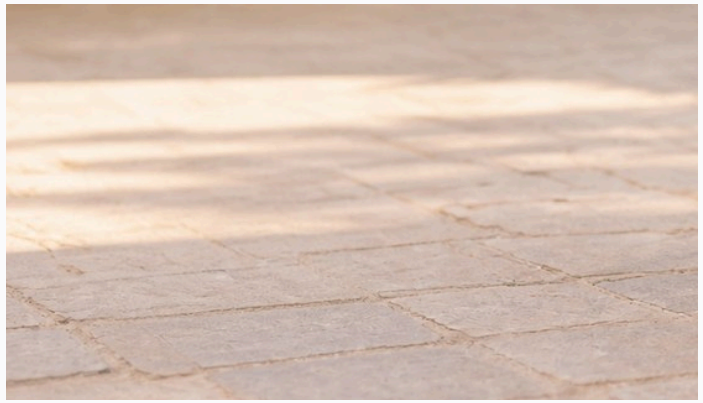
> Train both at once: cross-entropy for the class, and a loss that compares the four box numbers with the annotation's.

▶ Notebook, part 4

Write the answer as numbers: the target for one photo



object?	c_x	c_y	w	h	cat	dog
1	0.667	0.527	0.319	0.642	1	0



object?	c_x	c_y	w	h	cat	dog
0	?	?	?	?	?	?

Only “object?” is learned here: each **?** is left out of the loss.

> The loss counts only entries with a right answer: for the floor, just the first. A detector meets this “?” at most places in a photo.

Our annotation of the single-cat photo (data/annotations.json). The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 4

Training the box head: compare four numbers

$$\mathcal{L}_{\text{box}} = \sum_{k=1}^{4} \left| \hat{b}_k - b_k \right|$$

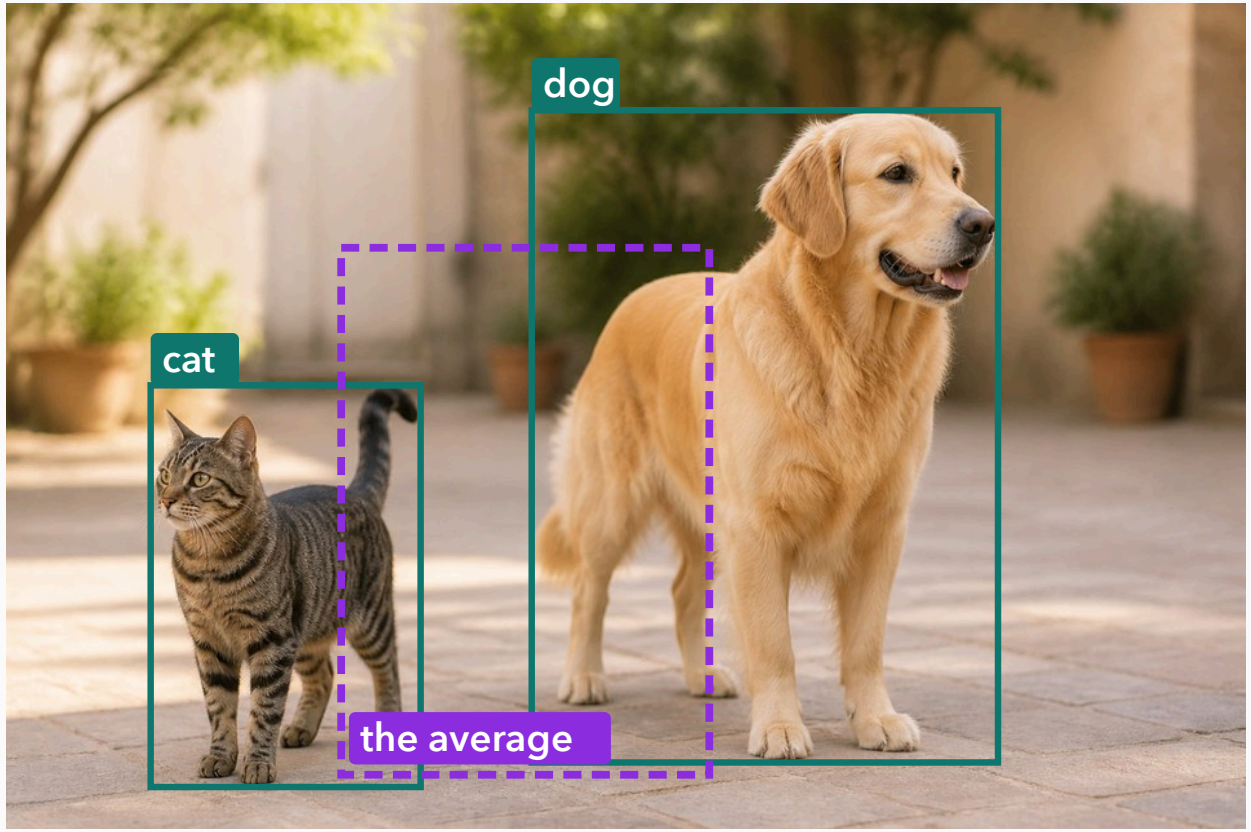
	c_x	c_y	w	h
target (the dog)	0.612	0.524	0.376	0.789
prediction (the guess)	0.674	0.587	0.376	0.789
|difference|	0.062	0.063	0.000	0.000

$$\mathcal{L}_{\text{box}} = 0.062 + 0.063 + 0.000 + 0.000 = 0.125$$

> Loss 0.125. It is zero only if all four numbers match; gradient descent moves the four outputs toward the target.

▶ Notebook, part 4

Two animals, one box: which one should it draw?



The annotation lists both animals. The head has room for one box.

Its safest single answer lies in between: the average box.

$$\mathrm{IoU}(\mathrm{avg}, \mathrm{dog}) = 0.225$$
$$\mathrm{IoU}(\mathrm{avg}, \mathrm{cat}) = 0.114$$

> A box between two objects describes neither of them.

The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 4

Give the head K slots



Problem 1: the crowd photo has 11 people, and 5 slots cannot hold them.

> So choose K larger than any photo needs: DETR (Appendix A) uses K = 100.

▶ Notebook, part 4

Problem 2: which dog belongs in slot 1?



$$\text{slot 1 vs entry 1:}\ \ \mathcal{L} = 1.212$$

$$\text{slot 1 vs entry 2:}\ \ \mathcal{L} = 0.012$$

The same correct answer costs 100 times more because of the list's order. The loss must not depend on the order.

> Nothing about the prediction changed, only the list order. A loss that pairs outputs with targets by position punishes correct answers.

DETR (facebook/detr-resnet-50, Hugging Face) run on this photo by scripts/run_models.py.

▶ Notebook, part 4

Two ways to decide which output answers for which object

A · by place

- Each output owns a place in the image
- Outputs at the dog's place answer for the dog

Parts 5 to 7: YOLO and most fast detectors

B · by matching

- Keep free slots
- Before the loss, match slots to objects one to one, at the lowest total cost

Appendix A: DETR

> Fix B keeps free slots and, before each loss, matches them to objects at the lowest total cost, so order cannot matter either.

Part 4 in one slide

1. One box: a box head, an L1 loss; with no object, the box target is "?".
2. Two animals, one box: it learns the average box, which matches neither (IoU 0.23, 0.11).
3. K slots: pairing by list order punishes a correct answer (1.21 instead of 0.012).
4. Fix A: tie each output to a place in the photo.

Next: fix A, a grid of cells, each writing a box and class scores.

▶ Notebook, part 5

Part 5 · A candidate at every place

Tie each output to a place: a grid of cells over the photo, each writing one record.

1. From a crop classifier, to sliding windows, to one CNN pass: a grid of cells.
2. What each cell writes: four box numbers and 80 class scores.
3. Which cells answer for each object; targets and loss, by hand for one cell.
4. At test time, read every cell's box back out.

> Every cell of a grid writes a record. We work out its target and its loss, then read its box back out.

▶ Notebook, part 5

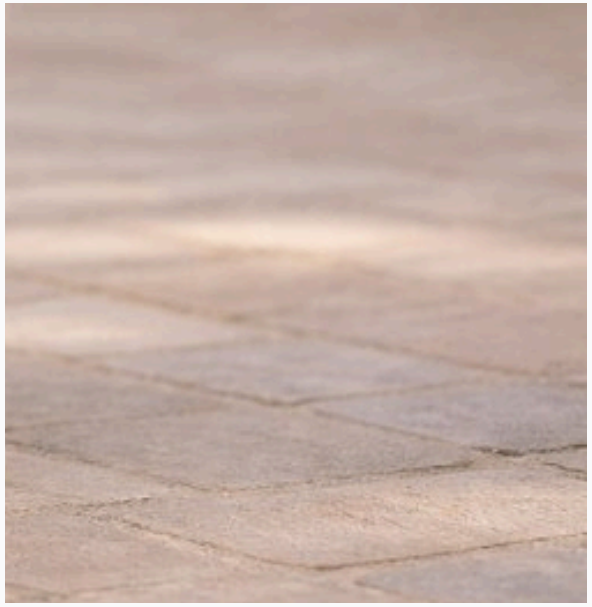
Start from a tool we have: a classifier for crops



dog



cat



nothing

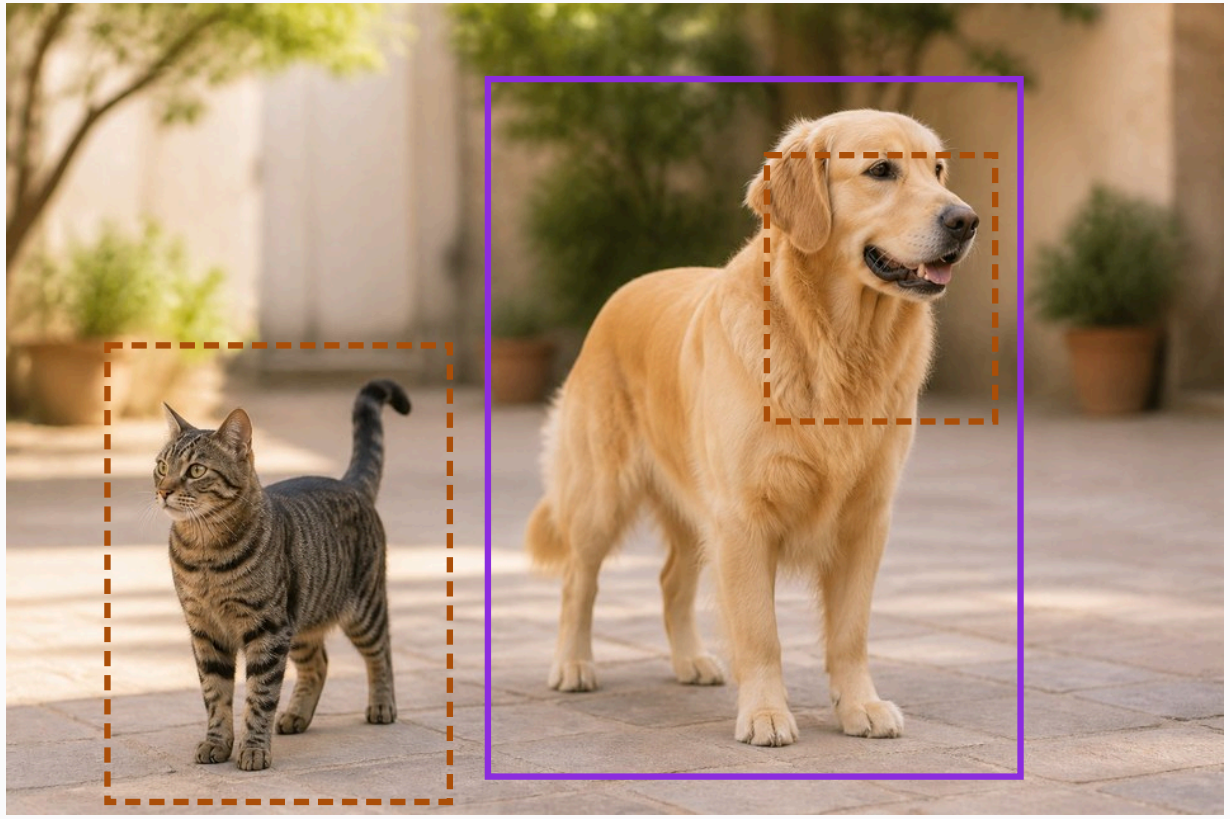
Train a classifier on many crops like these. Shown one crop, it answers: dog, cat, or nothing.

> It cannot say where anything is in a whole photo. Can we use it to find objects anyway?

The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 5

Slide the classifier over the photo



Crop a window and classify it: dog.

Move it every 32 px across and down: 30 × 20 = 600 positions.

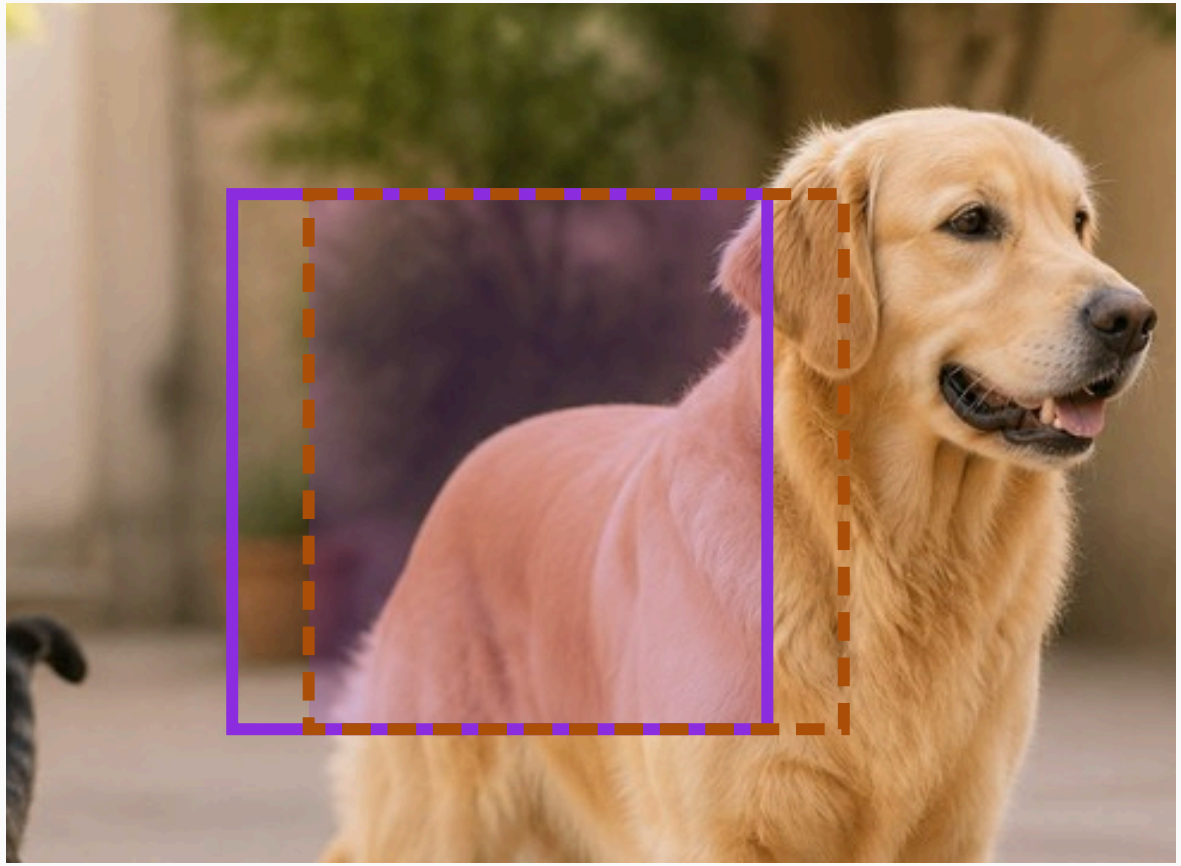
Objects come in many sizes and shapes: try 3 sizes × 3 shapes at each position. **5,400 crops**, each through a whole CNN.

> At each position try 3 sizes and 3 shapes. It works, but every crop is a full run of the classifier, so it is slow.

The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 5

Neighbouring windows see almost the same pixels



Two windows, 224 px wide, 32 px apart: they share 86% of their pixels.

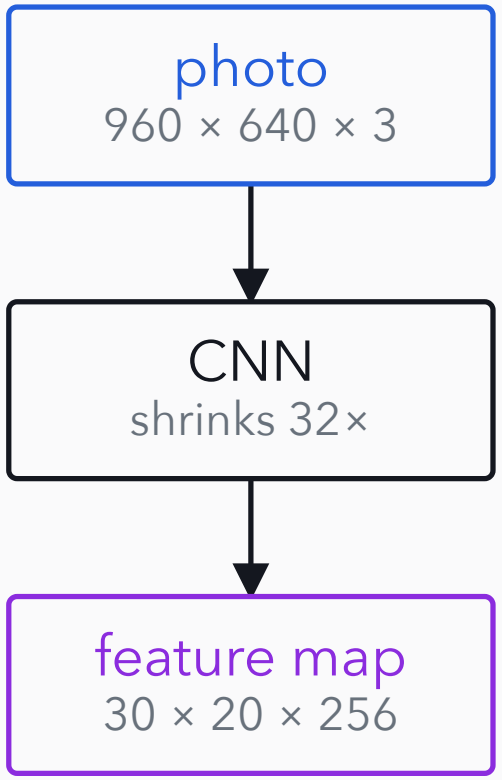
A CNN does the sliding itself: its convolutions already slide over every position, once.

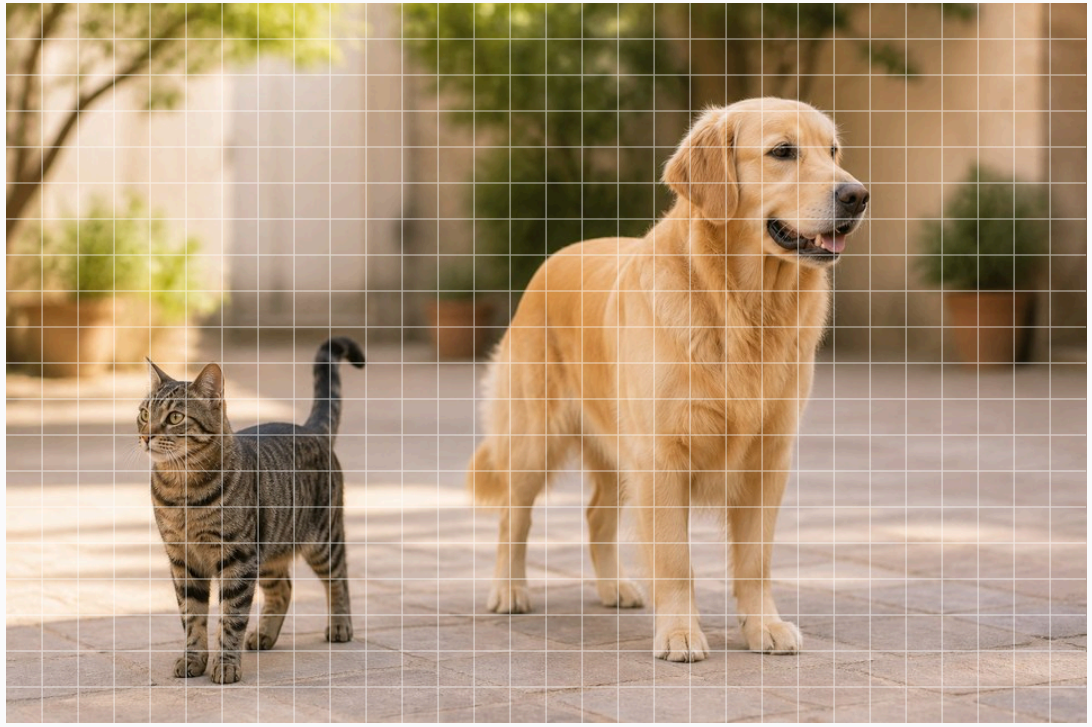
> Run the CNN once on the whole photo and read one output per position: every window at once, the shared work done once.

The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 5

One pass of a CNN: a 30 × 20 grid of feature vectors

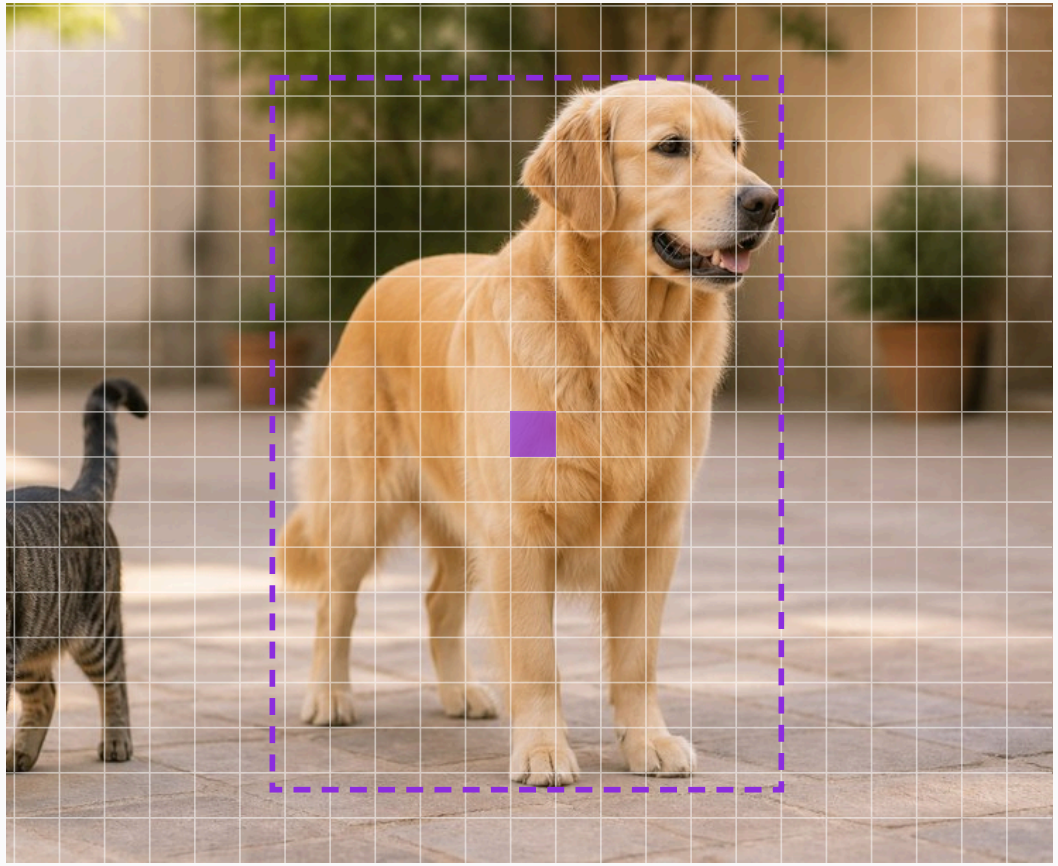




Lay that 30 × 20 grid over the photo. Each square, a cell, stands for one position of the feature map: 600 cells, 600 “windows”.

▶ Notebook, part 5

One cell: an address on the photo, with a wide view



Cell (row 10, column 18) covers a 32 × 32 block: x from 576 to 608, y from 320 to 352.

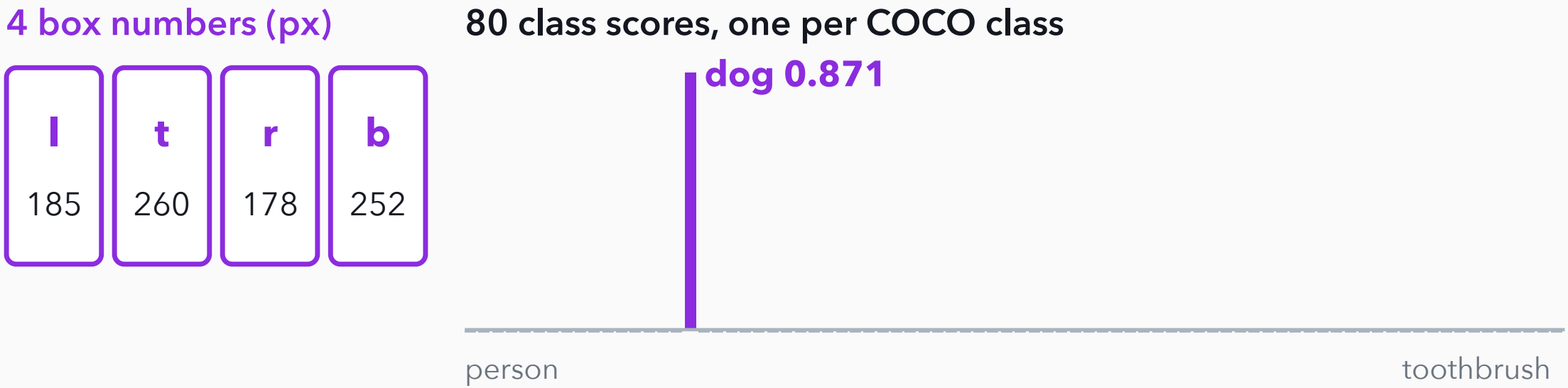
Its 256 numbers were computed from a region **much larger** than its block, wide enough to take in the whole dog.

> Each CNN layer mixes in neighbouring positions, so one cell can describe a box much larger than itself.

The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 5

Every cell writes a record: 4 box numbers, 80 class scores



There is no separate “object?” number: a cell says “nothing here” by giving all 80 classes a low score.

> Part 4’s localiser had an “object?” number. YOLOv8 does without one.

YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json.

▶ Notebook, part 5

What actually comes out: one tensor of fixed shape

```
x = load_photo("cat-dog.jpg")    # (1, 3, 640, 960): RGB values in 0..1
y, _ = net(x)                     # YOLOv8n, before any clean-up
y.shape                           # torch.Size([1, 84, 12600])
```

$$12{,}600 \text{ columns} = \underbrace{600}_{\text{this grid}} + \underbrace{12{,}000}_{\text{two finer grids (Part 7)}}$$

One column per cell: 600 from our 30 × 20 grid, the rest from two finer grids (Part 7).

YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json.

▶ Notebook, part 5

One column is one cell's record

```
y[0, :4, k]     # cell k's box: c_x, c_y, w, h in pixels
```

```
y[0, 4:, k]     # its 80 class scores, each between 0 and 1
```

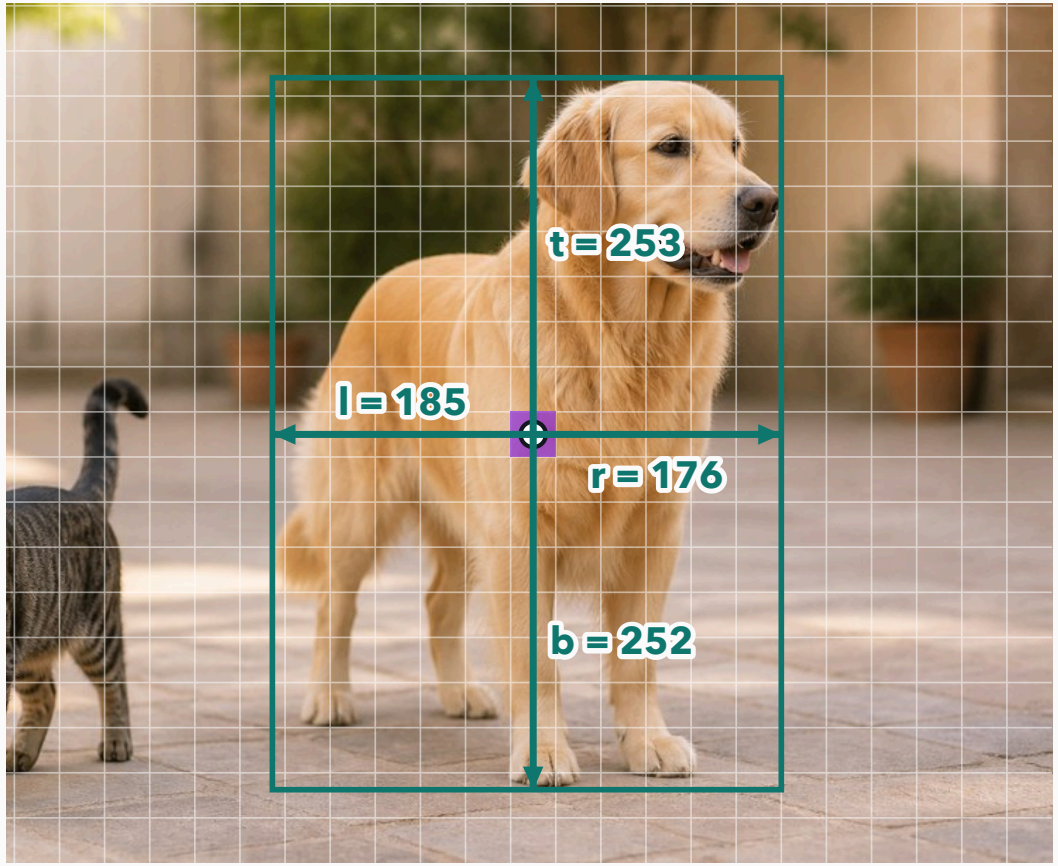
$$84 = \underbrace{4}_{\text{box}} + \underbrace{80}_{\text{classes}}$$

> The other 80 rows are its class scores: 84 = 4 + 80, whatever the photo shows.

YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json.

▶ Notebook, part 5

How a cell describes a box: its distances to the four sides



$$\text{centre} = (18.5 \times 32,\ 10.5 \times 32) = (592,\ 336)$$

$$l = 592 - 407 = 185$$

$$t = 336 - 83 = 253$$

$$r = 768 - 592 = 176$$

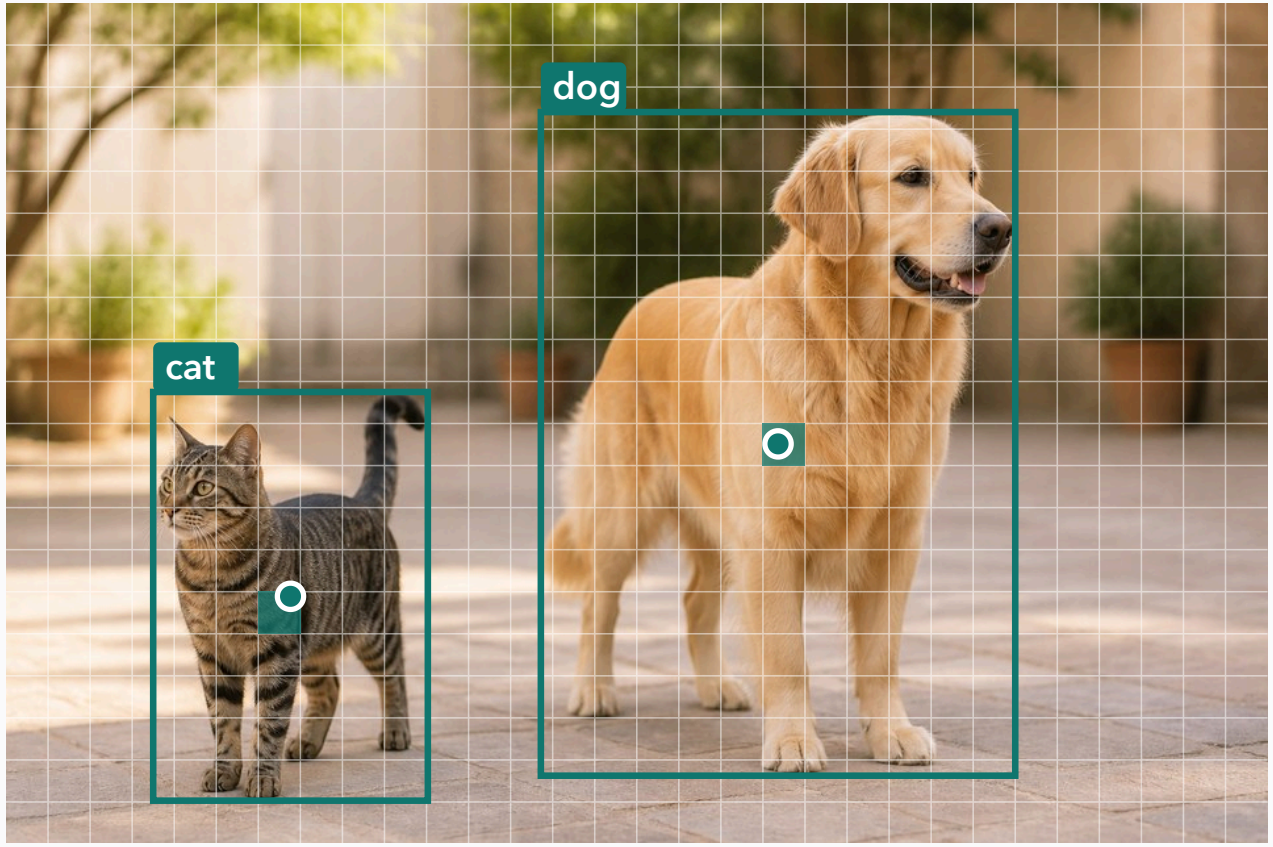
$$b = 588 - 336 = 252$$

> Bottom: 588 − 336 = 252. Like standing in a room and measuring the distance to each wall: four numbers fix the box.

The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 5

The simplest rule: the cell that holds the centre



The dog's box centre, (587.5, 335.5), falls in cell (10, 18). That cell is **responsible** for the dog.

The cat's centre, (216.5, 451.5), falls in cell (14, 6).

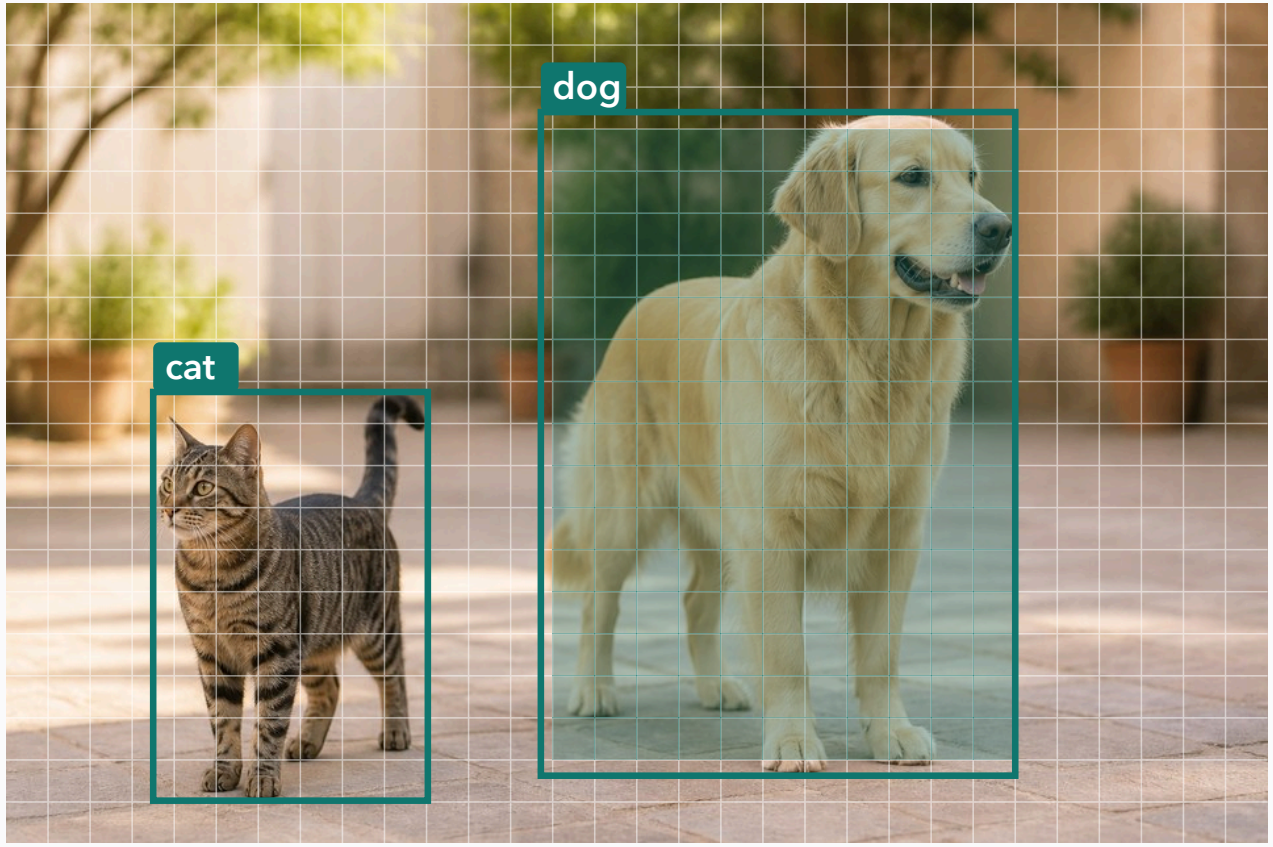
Two cells learn an object; the other 598 learn "nothing here".

> Every object now has exactly one cell, decided by where it is, so there is no slot order to argue about.

The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 5

More cells per object: every cell inside the box?



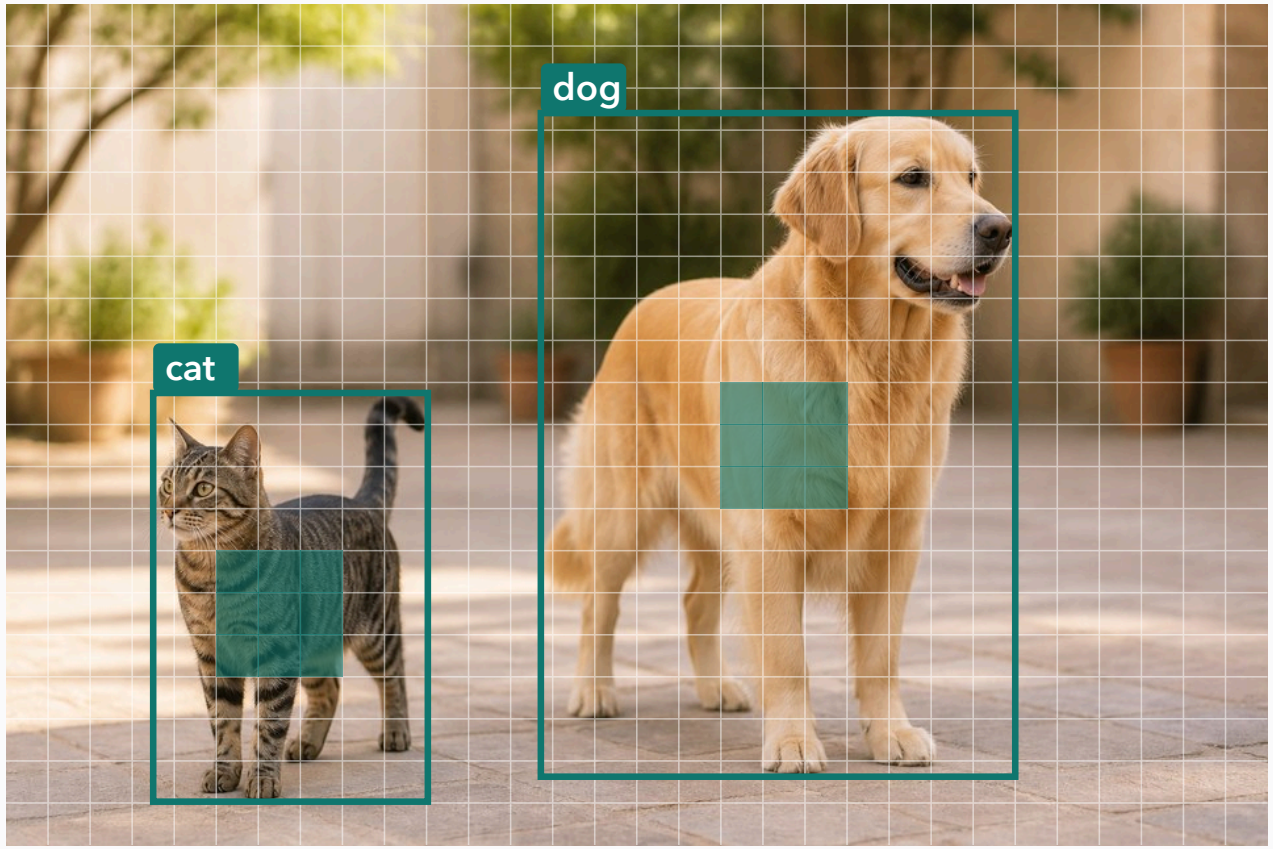
Every cell inside the dog's box: **165** cells, many of them mostly floor.

> One positive cell per object is a weak training signal. Using every cell in the box adds edge cells that see mostly floor, a noisy target.

The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 5

A middle way: a block of cells around the centre



FCOS calls this “centre sampling”. Dog: rows 9 to 11, columns 17 to 19.

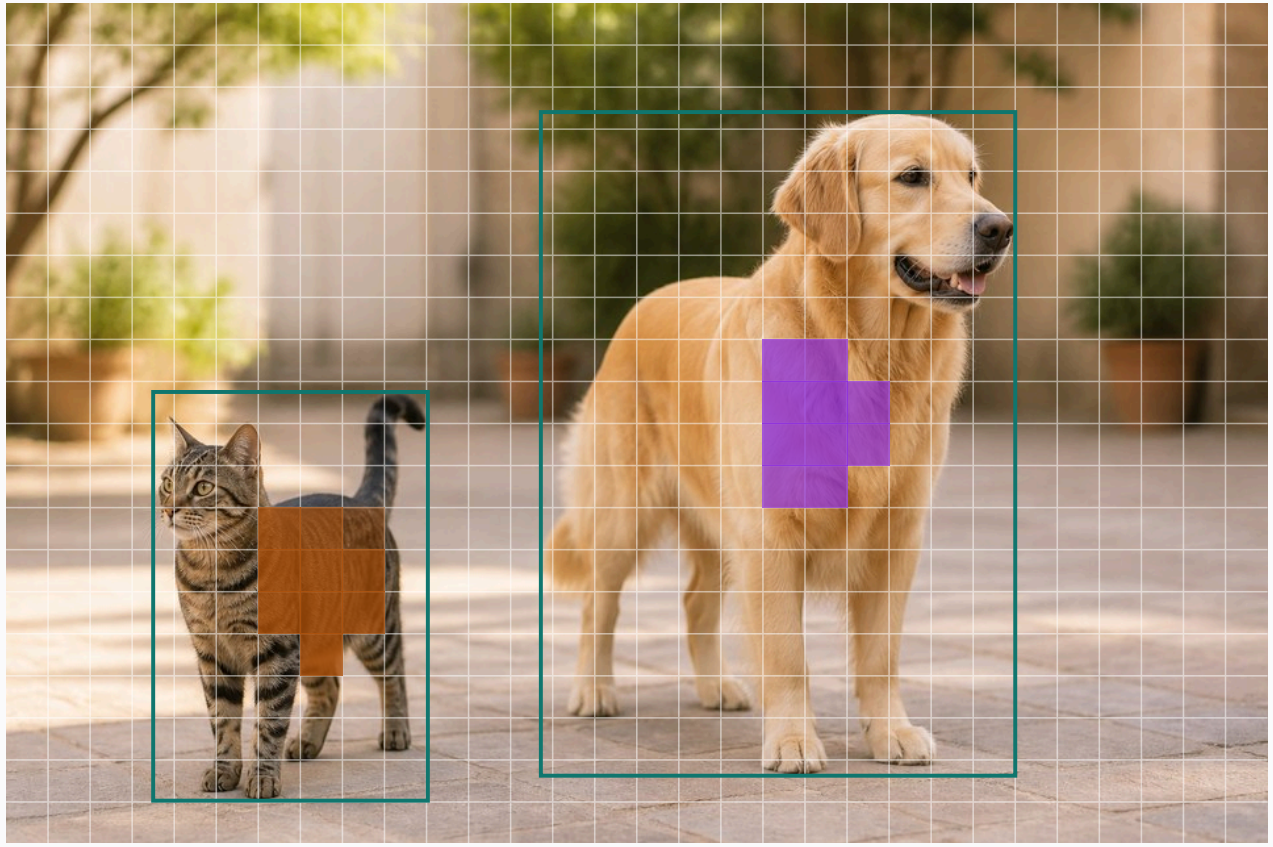
Cat: rows 13 to 15, columns 5 to 7. These 18 cells are **positives**; the other 582 are **background**.

> Do the same for the cat. Every other cell learns “no object of any class”.

The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 5

The trained YOLOv8n: about ten cells fire per object



Shade = the cell's dog score. 10 cells score above 0.25, all near the dog's centre; 6 of them are in our 3 × 3 block.

Cat: again 10 cells. YOLOv8 trains 10 positives per object.

> A second cluster for the cat. About ten cells per object, as training asked: the source of the duplicates we remove later.

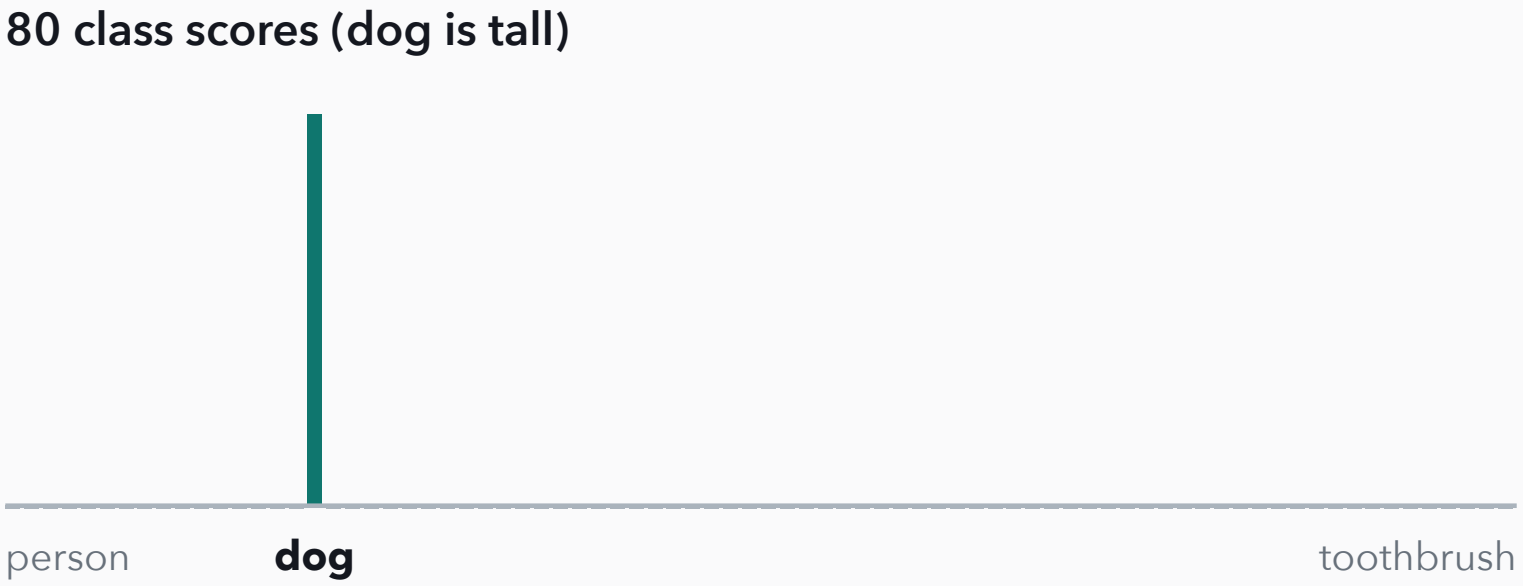
YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json.

▶ Notebook, part 5

The target at a positive cell, (10, 18)

box target (px)

l	t	r	b
185	253	176	252



Box target: the four distances, (185, 253, 176, 252) px; in cells (÷ 32), (5.78, 7.91, 5.50, 7.88).

Its box target is the four distances we just measured. A box much larger than a cell is fine.

▶ Notebook, part 5

The target at a background cell: all zeros, and “?”

box target: don’t care

l	t	r	b
?	?	?	?

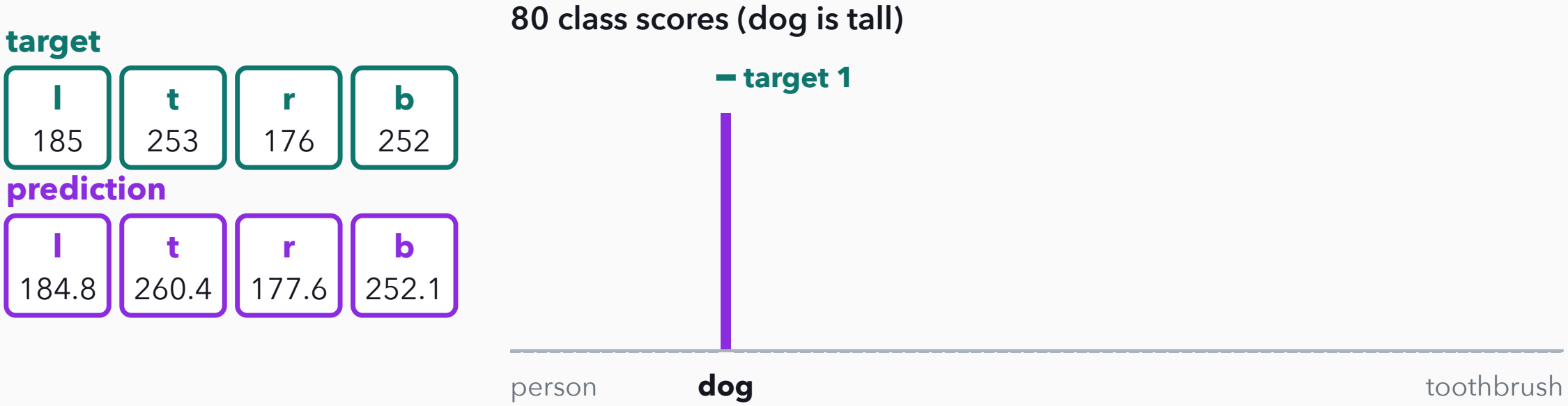
80 class targets: all 0

person **dog** toothbrush

A background cell, (3, 3) on the wall: every class target is 0; its box target is “?”, left out of the loss, as for Part 4’s empty floor.

▶ Notebook, part 5

What the network wrote at cell (10, 18)



Distances: close to the target; the top edge is furthest off.

> The loss turns all these differences, class and box, into one number to reduce.

YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json.

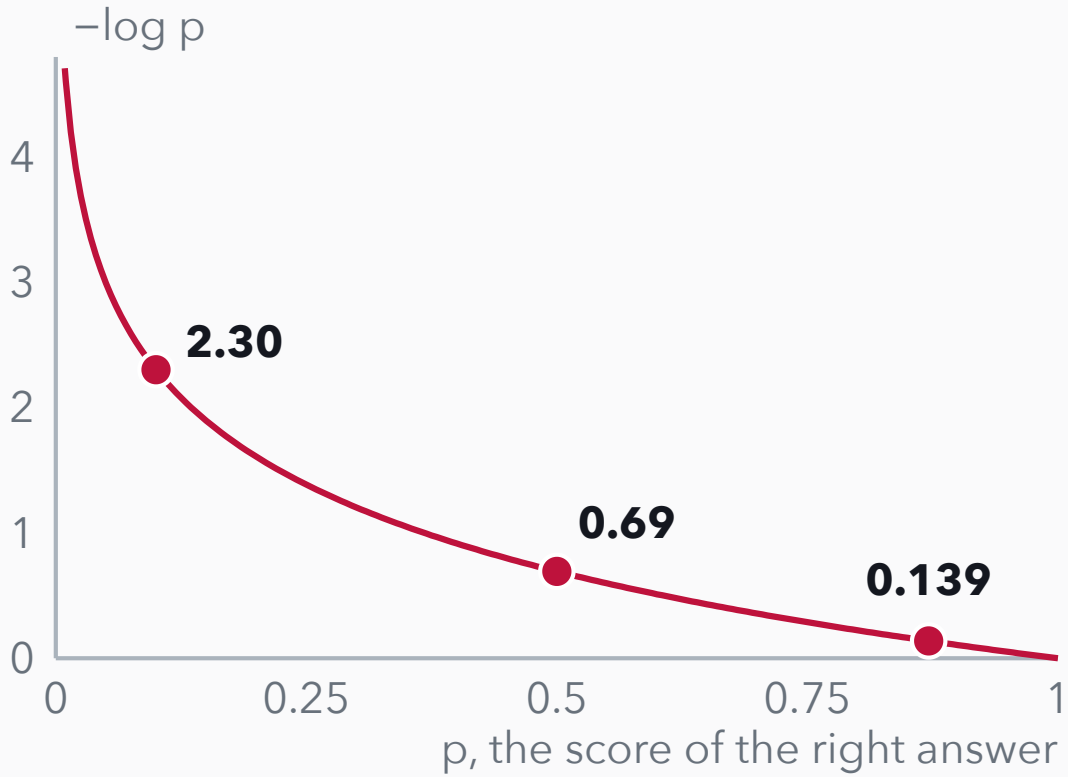
▶ Notebook, part 5

Scoring one yes/no answer: binary cross-entropy

$$\text{yes } (y = 1): \quad -\log p$$

$$\text{no } (y = 0): \quad -\log(1 - p)$$

p: the score the network gave this class; y: the target, 1 or 0. Natural logarithms.



For a yes: sure and right costs almost nothing; unsure (0.5) costs 0.69; sure and wrong (0.1) costs 2.30. Our dog cell, 0.871: 0.139.

▶ Notebook, part 5

Class loss at cell (10, 18): 80 yes/no questions

$$\text{dog: } y = 1,\ p = 0.871: \quad -\log 0.871 = 0.139$$

$$\text{the other 79: } y = 0: \quad \sum_c -\log(1 - p_c) = 0.0004$$

$$\mathcal{L}_{\text{class}}(10, 18) = 0.139 + 0.0004 = 0.139$$

The cell's class loss: 0.139, almost all from the dog question.

▶ Notebook, part 5

Box loss: how far off are the four distances?

$$\mathcal{L}_{\text{box}} = \sum_{k \in \{l,t,r,b\}} |\hat{d}_k - d_k|$$

$$= |184.8 - 185| + |260.4 - 253| + |177.6 - 176| + |252.1 - 252|$$

$$= 0.2 + 7.4 + 1.6 + 0.1 = 9.3 \text{ px}$$

> 0.2 + 7.4 + 1.6 + 0.1 = 9.3 pixels. Most of it is the top edge: the predicted box starts 7.4 pixels higher than ours.

▶ Notebook, part 5

A box loss that does not care about size: 1 − IoU

$$1 - \mathrm{IoU}(\text{predicted box}, \text{target box}) = 1 - 0.980 = 0.020$$

9.3 px is nothing for the dog, and a miss for Part 1's 13-px pedestrian. IoU, a fraction of area, scores both on one scale, as Part 3 argued.

> Another choice: one minus the IoU of the two boxes, here 0.020. Small objects then count as much as large ones.

▶ Notebook, part 5

A background cell: only a class loss, and it is small

$$\text{cell } (3,3): \quad \sum_{c=1}^{80} -\log(1-p_c) = 0.0038 \qquad (\text{largest } p_c = 0.001,\ \text{potted plant})$$

This grid has 582 background cells and only 18 positives, so detectors weight the background terms down. YOLOv1 multiplied them by 0.5; RetinaNet's focal loss shrinks the easy ones (optional slide).

> The imbalance matters: hundreds of easy background terms against a handful of positives. Without care they dominate the sum and the network learns to say "nothing" everywhere.

▶ Notebook, part 5

The loss for the whole photo

$$\mathcal{L} = \frac{1}{N_{\text{pos}}} \Bigg[\underbrace{\sum_{\text{all cells}} \sum_{c=1}^{80} \text{BCE}(p_c, y_c)}_{\text{class: every cell}} + \lambda \underbrace{\sum_{\text{positive cells}} \mathcal{L}_{\text{box}}}_{\text{box: positives only}} \Bigg]$$

Here: 600 cells contribute class terms; the 18 positives also contribute box terms; divide by N_{pos} = 18.

One number for the photo. Backpropagation sends its gradient into the shared head and the CNN; every cell's weights are the same weights, so every cell learns from every object.

> Gradient descent on this one number trains the shared layers.

▶ Notebook, part 5

The same loss in code

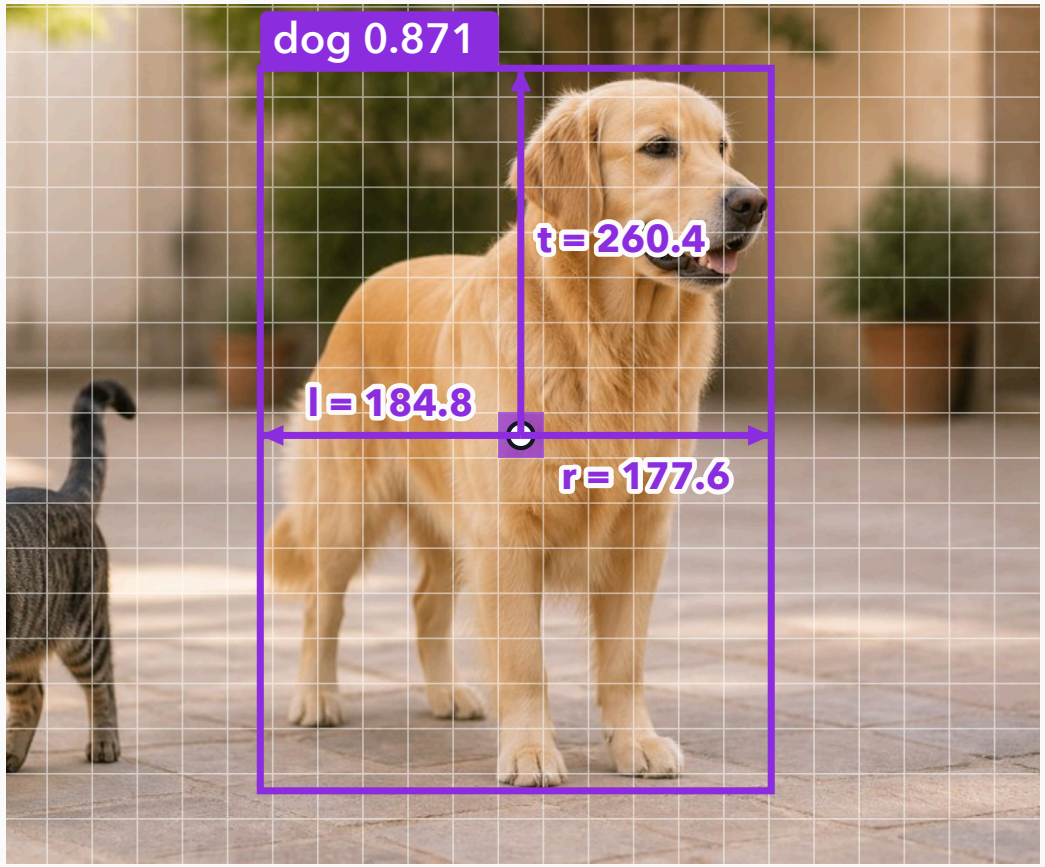
```
logits = head_cls(feats)                          # (600, 80): one row per cell
dist   = head_box(feats)                          # (600, 4): l, t, r, b in cells
y = torch.zeros(600, 80)
y[pos, labels] = 1.0                              # positives: 1 for their class
cls_loss = F.binary_cross_entropy_with_logits(logits, y, reduction='sum')
box_loss = (dist[pos] - dist_target[pos]).abs().sum()
loss = (cls_loss + box_loss) / len(pos)
```

pos: the indices of the 18 positive cells; labels: their class (16 for dog, 15 for cat); dist_target: their (l, t, r, b) targets.

> Building pos, labels and dist_target from the annotations is the “assignment” step: the 3 × 3 rule written as code. The notebook does it for this photo.

▶ Notebook, part 5

At test time: read the box back out of the cell



No annotation now: only the record.

$$x_1 = 592 - 184.8 = 407.2$$
$$y_1 = 336 - 260.4 = 75.6$$
$$x_2 = 592 + 177.6 = 769.6$$
$$y_2 = 336 + 252.1 = 588.1$$

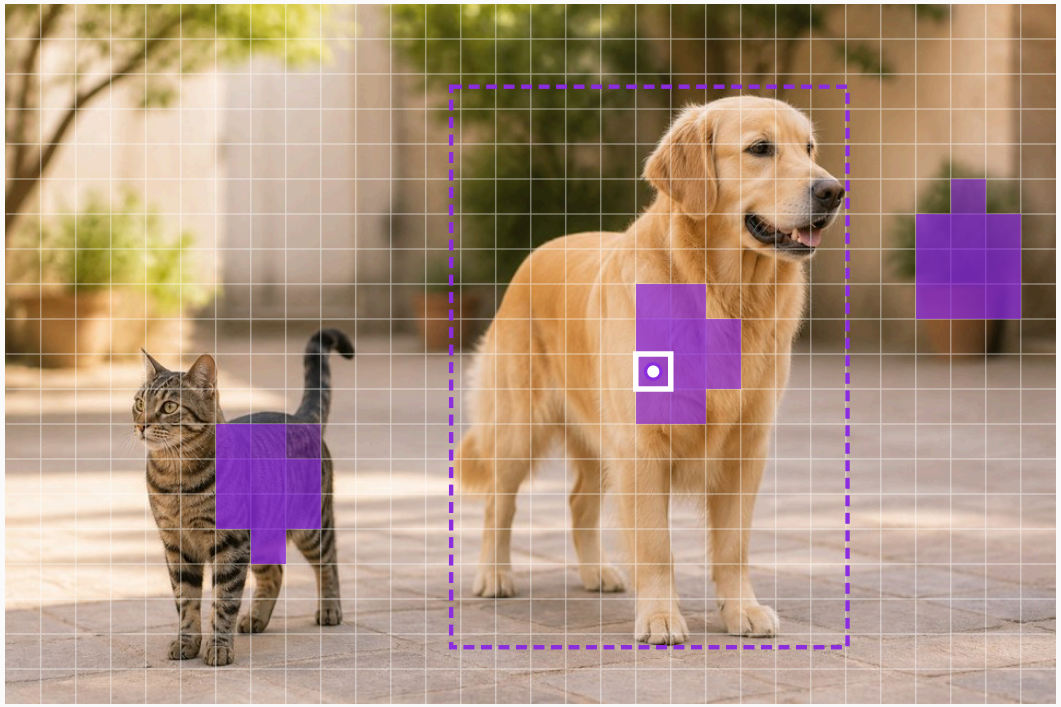
Score: the largest class score, 0.871, so the label is dog.

> So each cell gives one box, with a class and a score.

YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json.

▶ Notebook, part 5

Every cell writes a record: point at one



Cell: row 10, column 18

centre (592, 336) px

dog	0.871
cat	0.0001
skateboard	0.0001

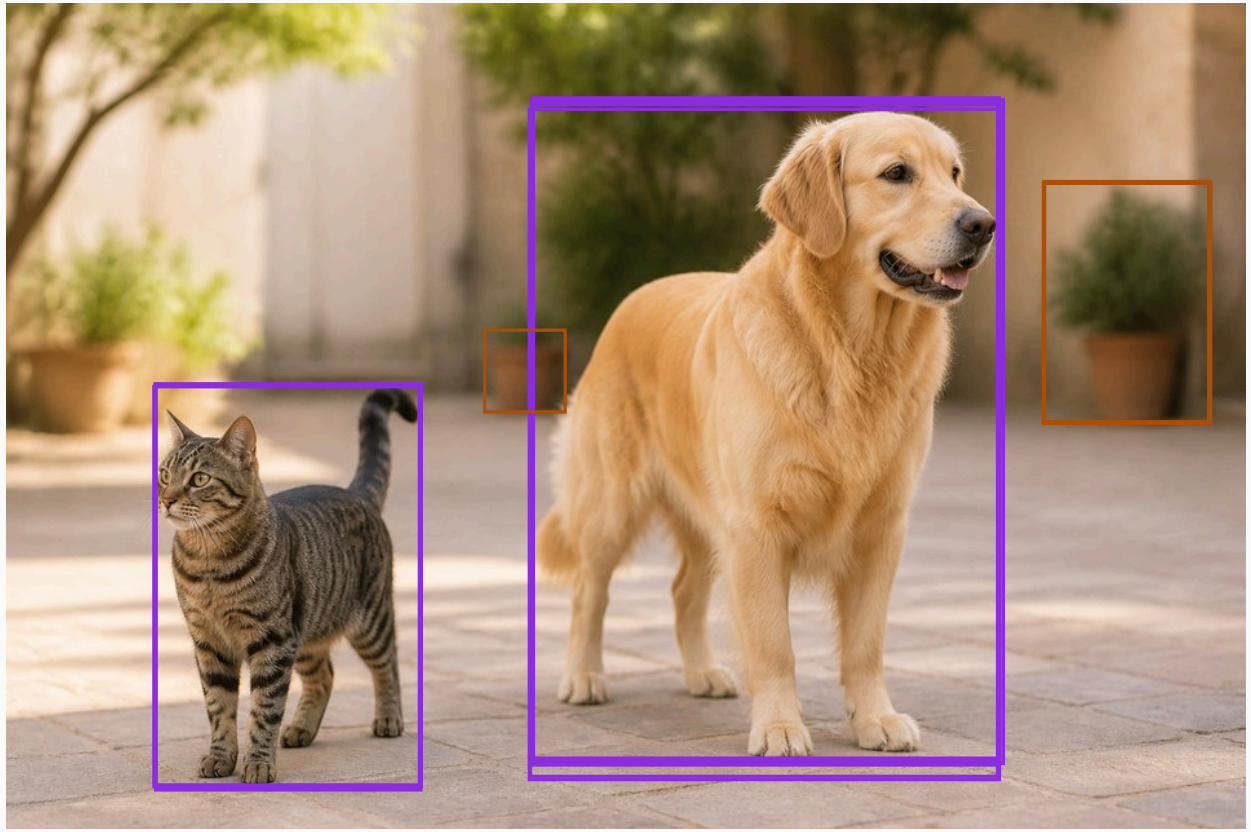
its box: (407, 76, 770, 588)

> Point at cells: near the dog they say dog and draw nearly the same box; on the wall every class score is tiny.

YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json.

▶ Notebook, part 5

Decode every cell, keep scores ≥ 0.25: 31 boxes



Always 12,600 records for a photo this size.

Scores ≥ 0.25: 31 boxes. 10 on the dog, 10 on the cat, 11 on plants.

We want one box per object.

> The ten cells that fired for the dog each drew a dog box. To get one box per object, remove the duplicates.

YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json.

Part 5 in one slide

1. One CNN pass replaces 5,400 sliding windows: a grid of cells.
2. Each cell writes four box distances and 80 class scores.
3. Cells near an object's centre are positives (18 here); the rest, background.
4. Loss: BCE for every class at every cell; a box loss at the positives.
5. Test time: decode every cell, keep scores ≥ 0.25: 31 boxes, 4 objects.

Next: each object has about ten boxes; keep one.

▶ Notebook, part 6

Part 6 · One box per object: non-maximum suppression

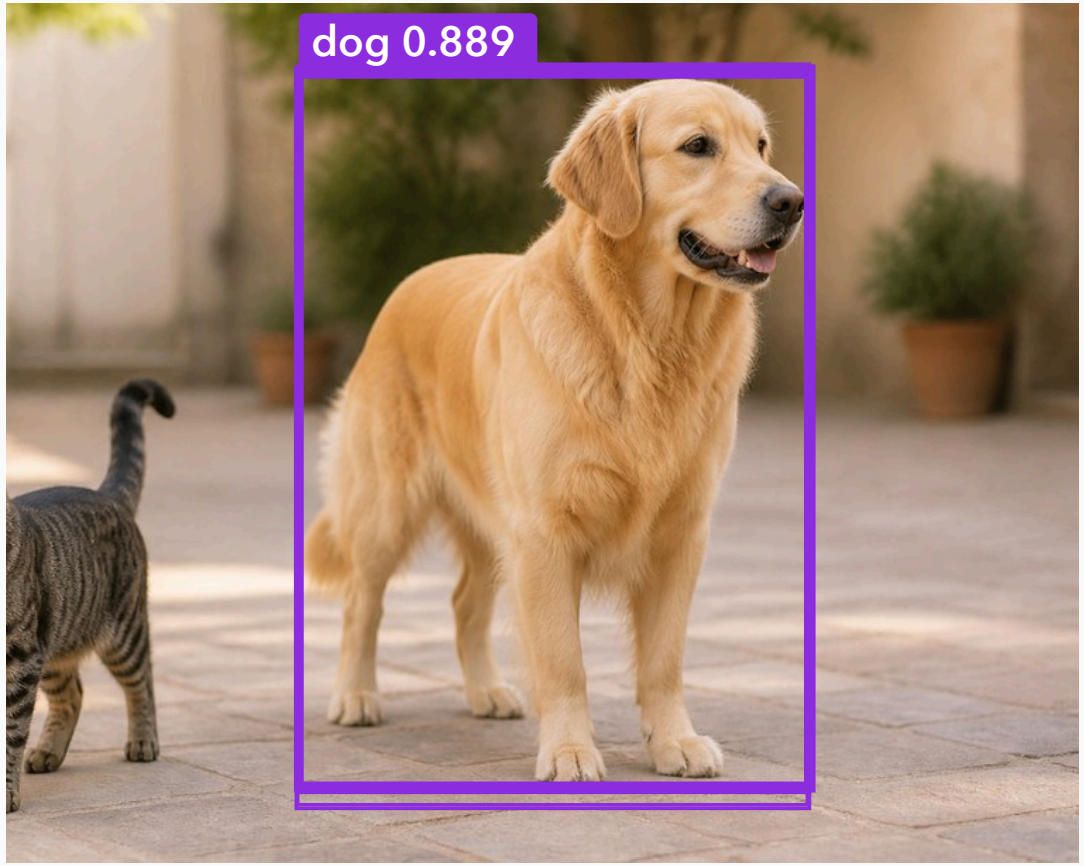
Ten boxes for one dog. Keep the best and delete the ones that overlap it heavily.

1. Run the greedy recipe on the dog's ten real boxes, then on two dogs.
2. See what the IoU threshold trades off, and why classes are kept apart.
3. Try it live; write it in eight lines; then the whole detector in code.

> NMS keeps the highest-scoring box and removes boxes that overlap it.

▶ Notebook, part 6

Ten boxes for one dog



cell (row, col)	score
(9, 19)	**0.889**
(9, 18)	0.888
(10, 18)	0.871
(10, 19)	0.862
(8, 18)	0.822
(10, 20)	0.797
(8, 19)	0.793
(11, 19)	0.792
(9, 20)	0.792
(11, 18)	0.766

> They describe one dog. Keep the most confident box, from cell (9, 19), and delete the rest.

YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json.

▶ Notebook, part 6

Non-maximum suppression: the greedy recipe

1. Sort the boxes by score, highest first.
2. Keep the top box.
3. Delete every remaining box of the same class whose IoU with it is above a threshold (0.7 here).
4. Repeat from step 2 with what is left, until nothing is left.

> Then repeat on what is left. Non-maximum suppression removes every box that is not a local maximum.

▶ Notebook, part 6

NMS on the dog's ten boxes

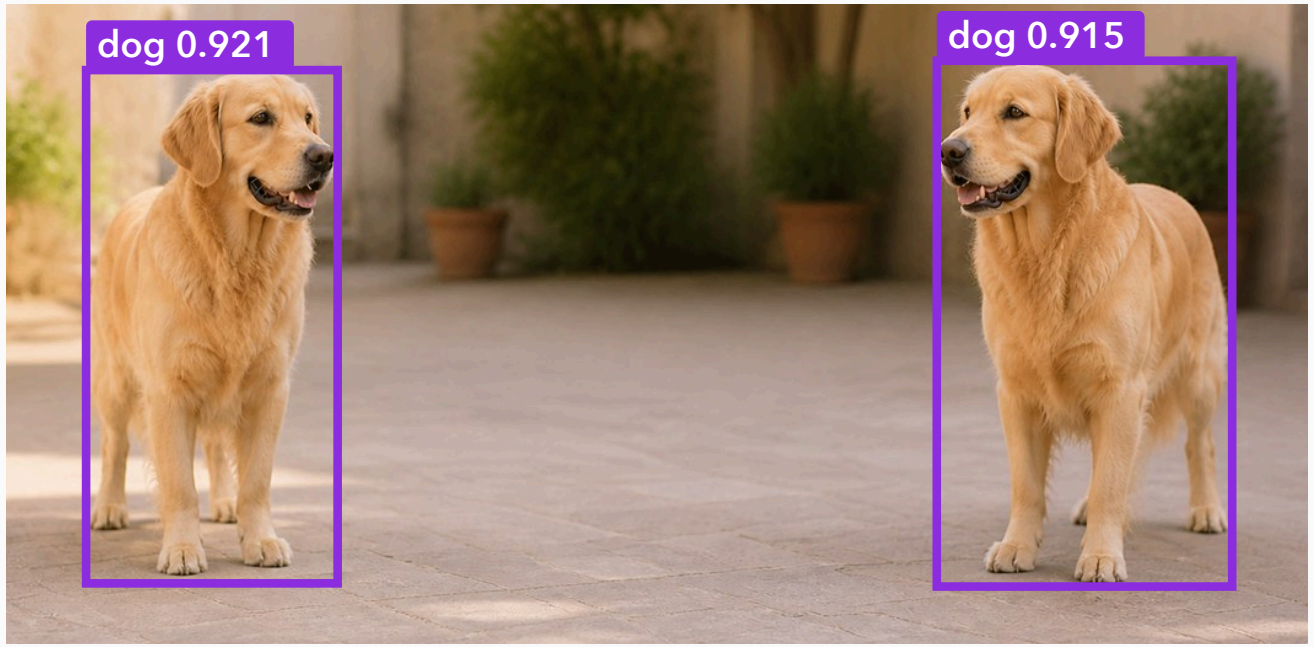
cell	score	IoU with the kept box	fate
(9, 19)	0.889	kept box	**keep**
(9, 18)	0.888	0.993	**delete (> 0.7)**
(10, 18)	0.871	0.988	**delete (> 0.7)**
(10, 19)	0.862	0.985	**delete (> 0.7)**
(8, 18)	0.822	0.961	**delete (> 0.7)**
(10, 20)	0.797	0.981	**delete (> 0.7)**
(8, 19)	0.793	0.956	**delete (> 0.7)**
(11, 19)	0.792	0.980	**delete (> 0.7)**
(9, 20)	0.792	0.996	**delete (> 0.7)**
(11, 18)	0.766	0.977	**delete (> 0.7)**

> Every IoU is above the 0.7 threshold, so all nine go and one dog box remains.

YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json.

▶ Notebook, part 6

Two dogs: why the loop keeps the second one



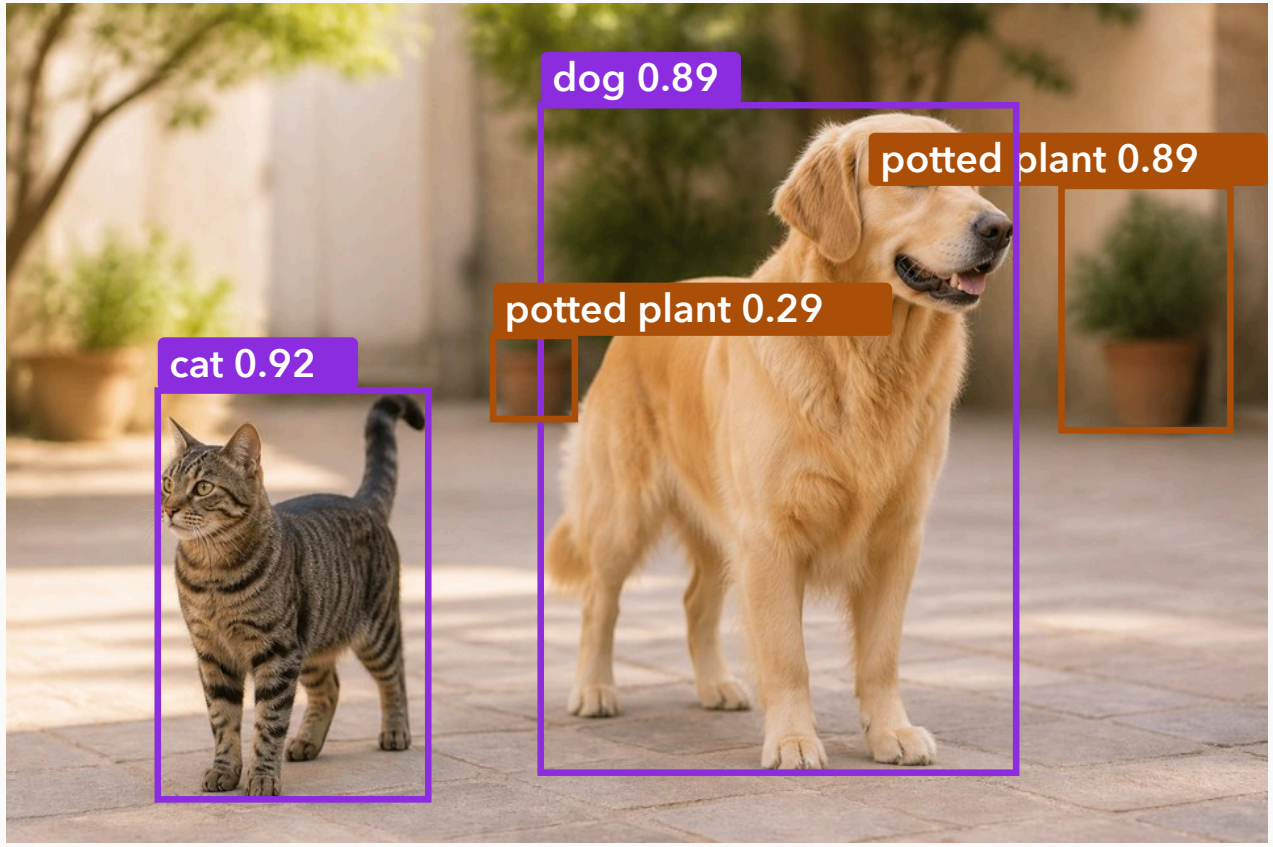
Round 2 keeps 0.915 on the right dog and deletes its 9 twins, leaving one box per dog.

> Round 2 keeps the right dog. Keeping only the single best box overall would have lost a dog; the loop keeps one per object.

YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json.

▶ Notebook, part 6

All 31 boxes → 4 detections



Before: 31 boxes.

After: 4. cat 0.921, potted plant 0.890, dog 0.889, potted plant 0.290.

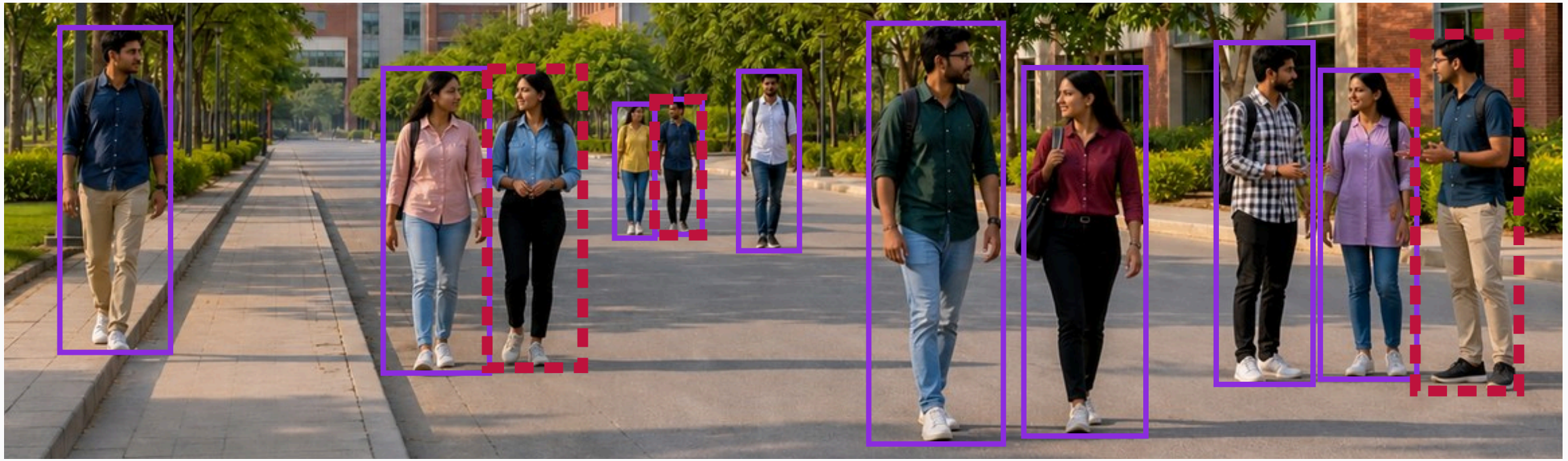
Our eight-line loop keeps exactly what Ultralytics' own post-processing keeps (checked in scripts/science.py).

> Four detections: cat, the big potted plant, dog, and a small plant on a finer grid. A fixed 12,600 records became a set of four.

YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json.

▶ Notebook, part 6

The IoU threshold is a trade-off



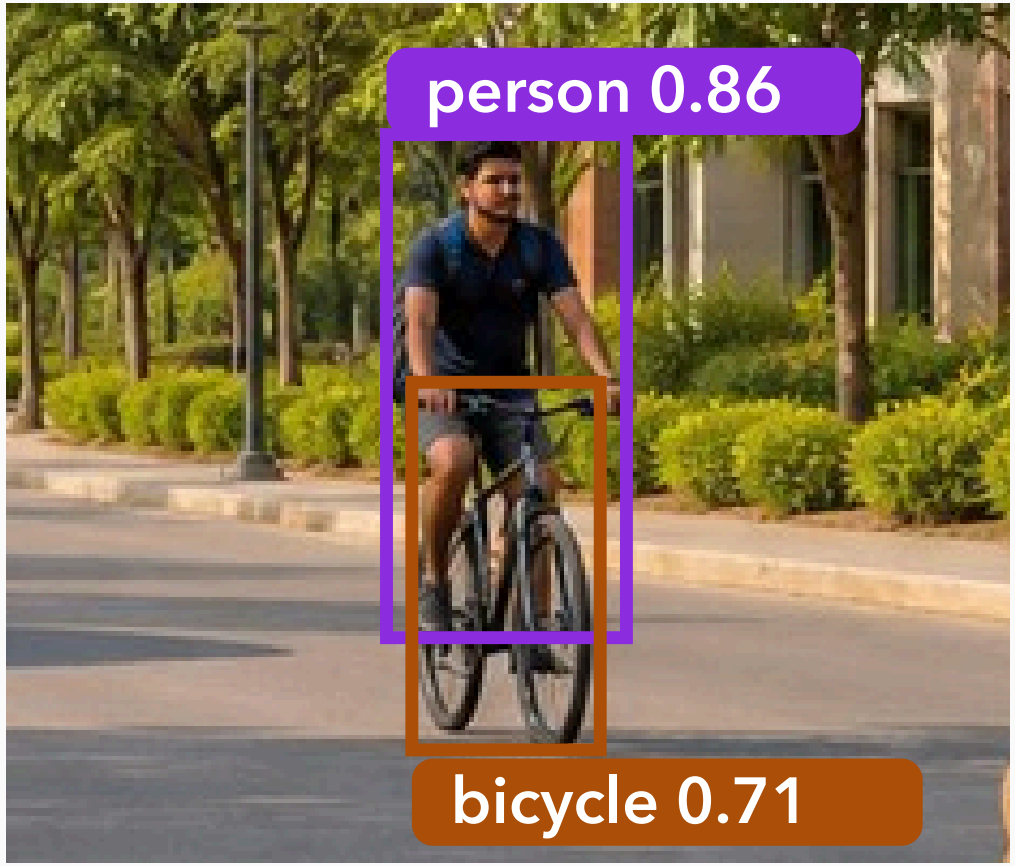
Too low, 0.0: 9 boxes. Two real people (red) are deleted.

> Lower it to 0 and any touching box is deleted, real neighbours included. No single threshold suits every scene.

YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json.

▶ Notebook, part 6

Boxes of different classes do not suppress each other



$$\mathrm{IoU}(\text{person}, \text{bicycle}) = 0.339$$

Classes ignored, threshold 0.3: the bicycle is deleted.

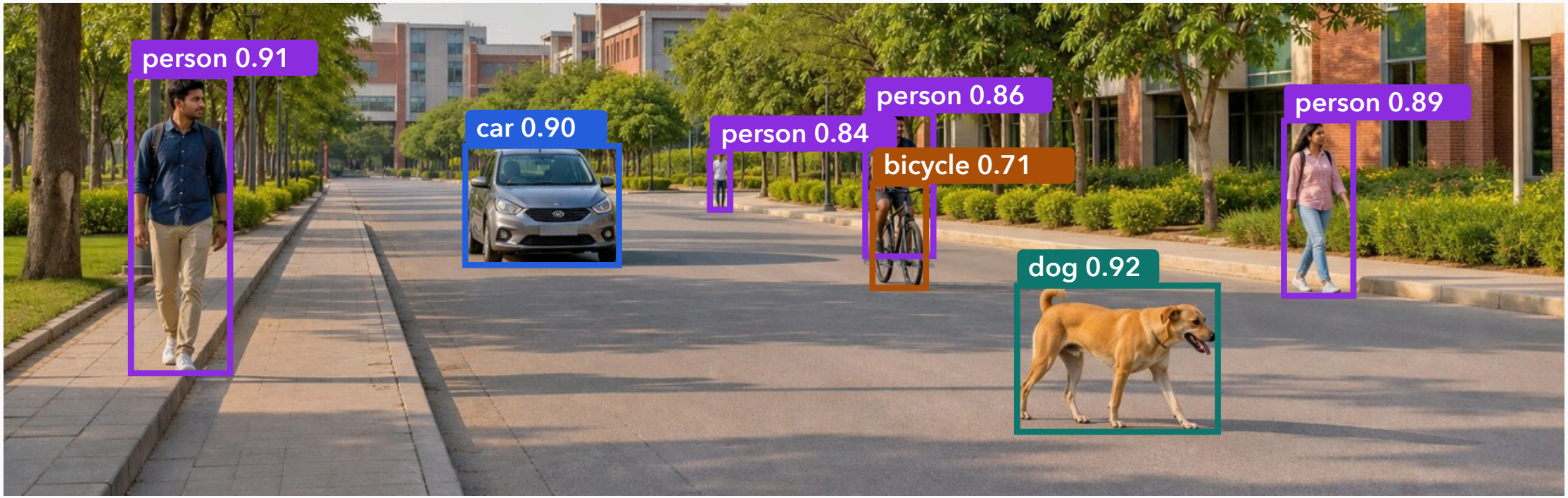
Class-aware NMS, the usual choice (torchvision's batched_nms): both survive.

> Run NMS separately for each class and both are kept. That is why step 3 of the recipe says "of the same class".

YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json.

▶ Notebook, part 6

Try it: thresholds on the street photo



> Move the score threshold, then the IoU threshold; try “all classes together” and watch the bicycle; Step runs one round of the loop.

YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json.

▶ Notebook, part 6

NMS in code

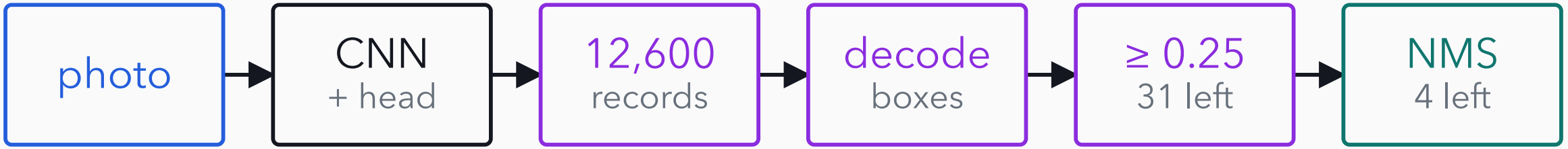
```python
def nms(boxes, scores, thr=0.7):
    order = scores.argsort(descending=True)          # step 1
    keep = []
    while order.numel() > 0:
        i = order[0]; keep.append(i)                 # step 2
        ious = box_iou(boxes[i, None], boxes[order[1:]])[0]
        order = order[1:][ious <= thr]               # step 3, then repeat
    return torch.stack(keep)
```

```python
torchvision.ops.nms(boxes, scores, 0.7)              # same indices
torchvision.ops.batched_nms(boxes, scores, labels, 0.7)  # per class
```

> torchvision has it built in; batched_nms runs it separately for each class label.

▶ Notebook, part 6

The dense answer, end to end



The network's output never changes shape. The score filter and NMS decide how many objects come out: here 4, for the crowd 11, for an empty wall 0.

> This is answer A to the lecture's question: place-based outputs, many per object in training, one per object after NMS.

▶ Notebook, part 6

The whole detector at test time, in eight lines

```python
def detect(photo, net, names, score_thr=0.25, iou_thr=0.7):
    y, _ = net(photo)                          # (1, 84, 12600): always this shape
    boxes = cxcywh_to_xyxy(y[0, :4].T)         # (12600, 4), in pixels
    scores, labels = y[0, 4:].max(0)           # each cell's best class and score
    keep = scores >= score_thr                 # 31 cells left
    boxes, scores, labels = boxes[keep], scores[keep], labels[keep]
    keep = batched_nms(boxes, scores, labels, iou_thr)   # 4 left
    return [(names[l], s, b) for b, s, l in zip(boxes[keep], scores[keep], labels[keep])]
```

```python
detect(courtyard, net, names)
# cat 0.921, potted plant 0.890, dog 0.889, potted plant 0.290, each with its box
```

> On the courtyard photo it returns the 4 detections of the lecture's first slide. Nothing in it depends on how many objects there are.

YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json.

▶ Notebook, part 7

Part 7 · Small and large objects: three grids

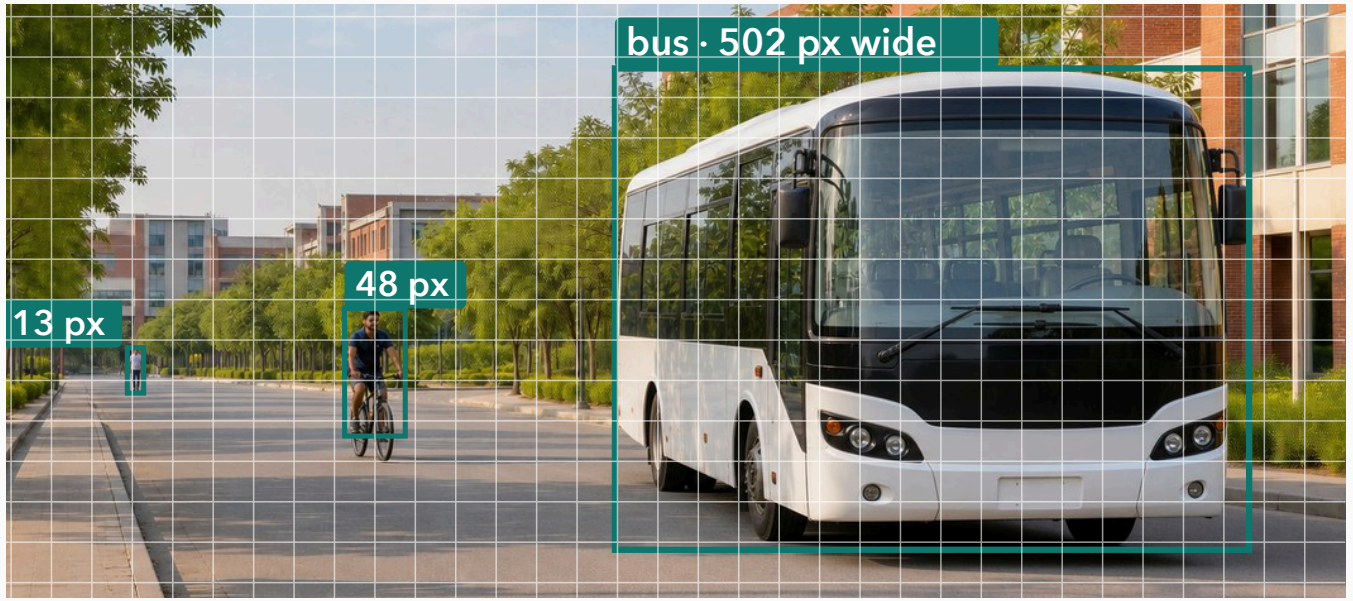
A 32-pixel cell is coarse for a person 13 pixels wide. YOLOv8 predicts from three grids at once.

1. See the range of sizes in one street photo, now with the grid.
2. Count the cells of the three grids: where 12,600 came from.
3. Check which grid found each real detection.

> YOLOv8 uses three grids with different cell sizes, and every cell writes the same record.

▶ Notebook, part 7

Part 1's street again, with the 32-pixel grid



The bus is 502 px wide; the cyclist 48 px; the far pedestrian 13 px.

On a 32-px grid the pedestrian is under half a cell.

> Draw the 32-px grid. The bus spans about 16 cells; the pedestrian sits inside one. A coarse grid suits big objects; small ones need finer cells.

The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 7

Three grids: cells of 8, 16 and 32 pixels

grid (cell size)	courtyard, 960 × 640	street, 1280 × 704	built from
8 px	120 × 80 = 9,600	160 × 88 = 14,080	early layers: sharp, local
16 px	60 × 40 = 2,400	80 × 44 = 3,520	middle layers
32 px	30 × 20 = 600	40 × 22 = 880	deep layers: wide view
all	**12,600**	**18,480**	

> Add them up: 12,600 for the courtyard, the number we decoded in Part 5. A bigger photo gets more cells: 18,480 for the street.

▶ Notebook, part 7

Which grid found each object? The real answer



object	grid
bus	32 px
person	16 px
person	8 px
bicycle	16 px

Street photo: dog and near person from 32 px; car, cyclist and bicycle from 16 px; the far person from 8 px.

> The street photo agrees: large from coarse, small from fine. Training assigns each object to cells whose grid suits its size.

YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json.

Part 7 in one slide

1. One grid cannot suit a 502-px bus and a 13-px pedestrian.
2. YOLOv8 predicts from grids of 8, 16 and 32-px cells: 12,600 records.
3. Every cell writes the same 84-number record; NMS runs on all of them.
4. Small objects come from the fine grid, large ones from the coarse.

> **Next:** measure how good the detections are.

▶ Notebook, part 8

Part 8 · Measuring a detector

1 · Why detect? | 2 · A list | 3 · Boxes, IoU | 4 · One box, many | 5 · Dense grid | 6 · NMS | 7 · Three grids | **8 · Measuring**

Compare detections with annotations: decide which are right, then summarise.

1. Decide true positive, false positive and miss, on the street photo's real detections.
2. Precision and recall at one threshold, then at all of them: walk down a ranked list.
3. Average precision and mAP; then back to Part 1's bird.

> We judge a detector against annotations, one detection at a time, then summarise.

▶ Notebook, part 8

When is a detection correct?

1. **True positive (TP)**: the right class and IoU ≥ 0.5 with an annotated box…
2. …that no higher-scoring detection has already claimed. Go through detections by score.
3. **False positive (FP)**: everything else: a box where nothing is, a wrong class, or a second box on a claimed object.
4. **False negative (FN)**: an annotated object that no detection claimed: a miss.

> Every annotated object left unclaimed is a false negative.

▶ Notebook, part 8

Real check: people in the street photo



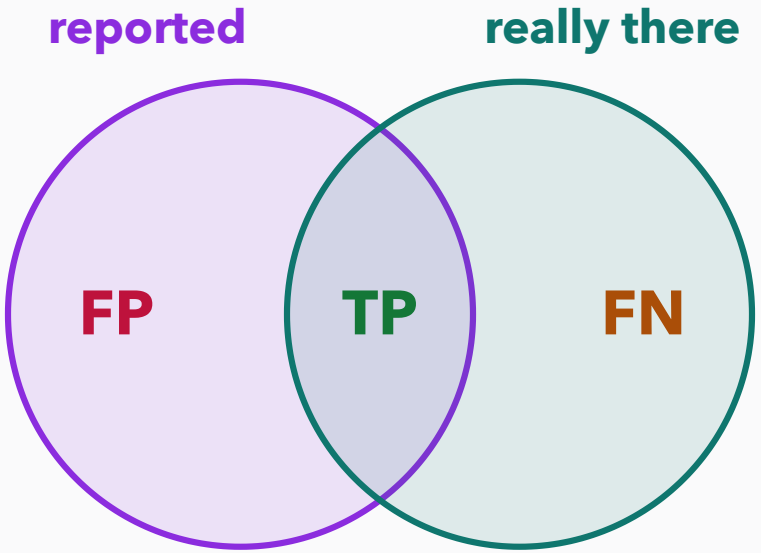
The other 8 (scores 0.042 down to 0.011) sit on the car's windscreen, IoU 0 with every person: **8 FP**. No person is missed: 0 FN.

> The rest sit on the car's windscreen, where nobody is.

YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json.

▶ Notebook, part 8

Precision and recall: two questions



$$\text{precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}$$

Of what we reported, how much was right?

$$\text{recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

Of what was really there, how much did we find?

Recall looks at the right circle, everything that was really there: the fraction of it we found.

▶ Notebook, part 8

The street's people at two score cut-offs

keep scores ≥	TP	FP	FN	precision	recall
0.25	4	0	0	4/4 = 1	4/4 = 1
0.01	4	8	0	4/12 = 0.33	4/4 = 1

> Keep everything down to 0.01 and the 8 windscreen boxes join: recall stays 1, precision drops to 0.33. The cut-off trades one for the other.

▶ Notebook, part 8

Back to the bird: a lower cut-off finds it



keep scores ≥	found	TP	FP	FN	precision	recall
0.25	cat, dog	2	0	1	1.00	2/3
0.05	bird, cat, dog	3	0	0	1.00	3/3

> At 0.05 the bird's box gets through: recall 3/3, and nothing false comes with it. On the street, going lower brought eight false people.

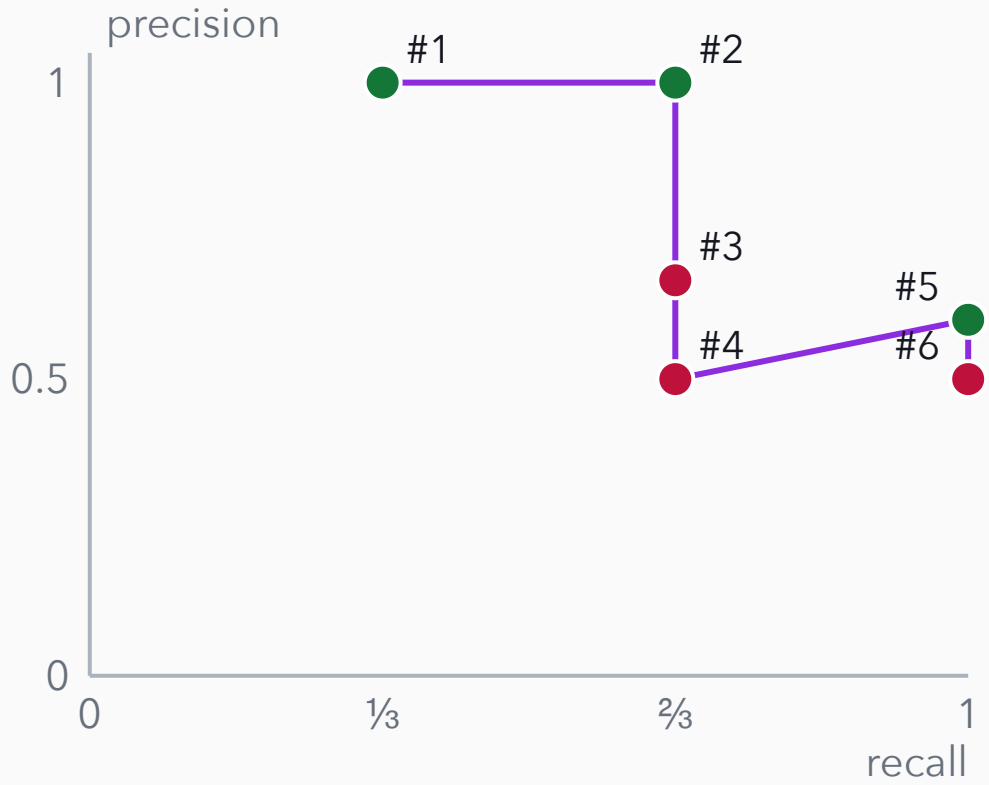
YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json.

▶ Notebook, part 8

Walk down the ranked list, one detection at a time

rank	score	outcome	why	precision	recall
#1	0.95	**TP**	match	1/1	1/3
#2	0.90	**TP**	match	2/2	2/3
#3	0.80	**FP**	duplicate	2/3	2/3
#4	0.70	**FP**	no truth here	2/4	2/3
#5	0.60	**TP**	match	3/5	3/3
#6	0.40	**FP**	no truth here	3/6	3/3

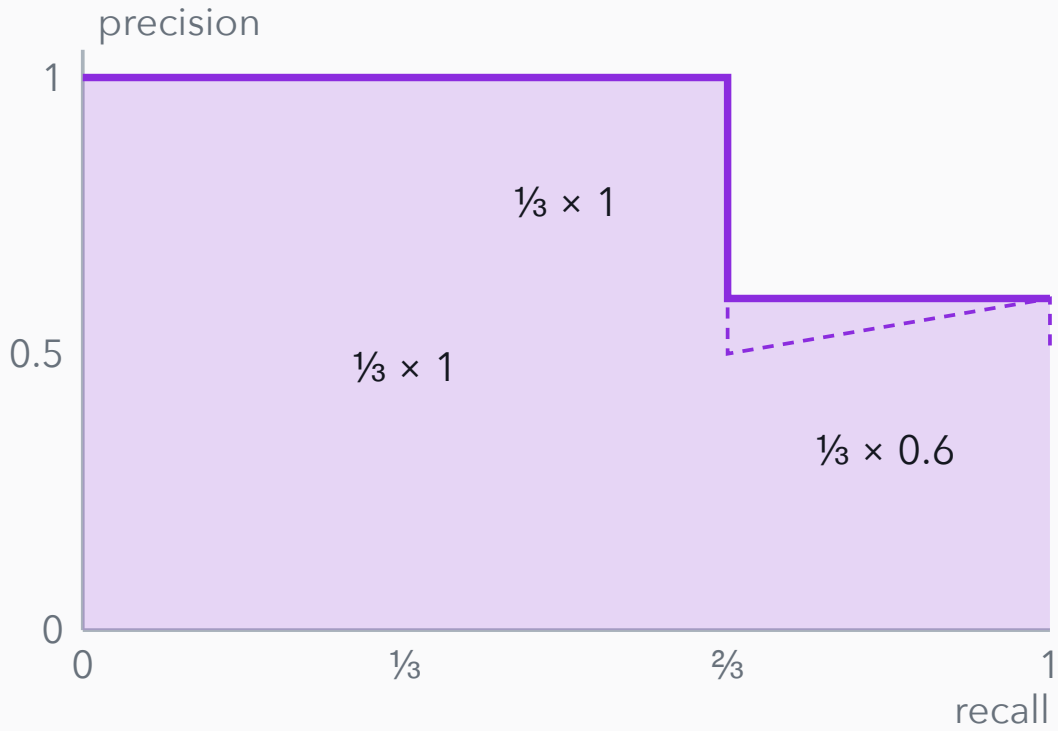
A small invented list (3 objects, 6 detections) so that hits and misses interleave.



> #6 (0.40): FP. 3/6, 3/3. Every point is one possible threshold.

▶ Notebook, part 8

Average precision: the area under the curve



Replace each precision by the best precision at that recall or beyond. The curve now only steps down.

$$\text{AP} = \tfrac{1}{3}(1) + \tfrac{1}{3}(1) + \tfrac{1}{3}(0.6) = 0.867$$

AP = 1 only if every object is found before the first false positive, as for the street's people.

> AP rewards ranking: correct detections should come before wrong ones. The street people scored AP = 1.

▶ Notebook, part 8

mAP: average over classes, and over IoU thresholds

$$\mathrm{mAP}_{50} = \frac{1}{C}\sum_{c=1}^{C} \mathrm{AP}_c \quad \text{(a hit needs IoU} \geq 0.5\text{)}$$

$$\mathrm{AP}_{[.5:.95]} = \text{mean of mAP at IoU } 0.50, 0.55, \ldots, 0.95$$

YOLOv8n	mAP@0.5	AP@[.5:.95]
our 8 photos, 7 classes, 31 objects	1.000	0.936
COCO validation, 5,000 photos, 80 classes (published)	not given	0.373

> On our eight clean photos: 1.00 at IoU 0.5, 0.94 over the strict thresholds. On COCO, 0.373: real scenes are far harder.

Our evaluator on data/annotations.json; COCO number: Ultralytics YOLOv8 documentation.

▶ Notebook, part 8

Evaluation in code

```python
from torchmetrics.detection import MeanAveragePrecision
metric = MeanAveragePrecision(iou_type="bbox")
metric.update(preds=[{"boxes": boxes, "scores": scores, "labels": labels}],
              target=[{"boxes": gt_boxes, "labels": gt_labels}])
res = metric.compute()          # res["map"] = AP@[.5:.95], res["map_50"] = AP@0.5
```

> Libraries do the matching, ranking and averaging. The notebook also has a twenty-line version that reproduces the walk-through.

One question, one answer

Question: a network's output has a fixed shape; the answer is a list of any length, in no order.

1. A candidate at every place: each cell of three grids writes a box and 80 class scores.
2. In training, the cells near each object learn it; every other cell learns "nothing here".
3. At test time, keep the confident cells, then NMS keeps one box per object.

> In the dense answer, the score filter and NMS decide the list's length, after the network.

Three uses of IoU, easy to confuse

IoU used for	pairs	when
assignment	cells ↔ annotated objects	training
NMS	predictions ↔ predictions	test time
evaluation	predictions ↔ annotated objects	measuring

> Measuring: a detection is right if its IoU with an annotation is at least 0.5. All three pairings use the same measure.

Back to the six jobs: the three everyday ones

job	what is hard	what to do
road	far pedestrians are tiny	the fine grid (Part 7); follow boxes across frames
crowd	people stand close and overlap	check the NMS threshold (Part 6)
camera trap	small animals get unsure scores, like the bird	a lower cut-off (Part 8); more photos of small animals

> What remains is tuning: the grids, the NMS threshold, the score cut-off.

Back to the six jobs: the three new ones

job	new classes	what is hard	what to do
microscope	cell types	cells touch	label cells, fine-tune; NMS must keep touching cells
warehouse	parcel	many sizes	label parcels, fine-tune the same detector
factory	crack, scratch, dent	thin, small defects	higher resolution, the fine grid; measure recall

> Label a few hundred photos with boxes, fine-tune the same detector, and measure precision and recall on photos it has not seen.

Next: beyond boxes



A box is a rough answer. Next lecture: which pixels belong to each object (masks), keypoints, depth, and boxes found from words: “the dog on the right”.

Beyond Boxes · Language-grounded vision

Practice: the notebook runs every number in this lecture. Appendices: A, DETR · B, more depth.

> Boxes are the coarsest description of where an object is. Next: masks, keypoints, and finding objects named in words.

Masks: YOLOv8n-seg run on the photo (scripts/run_models.py).

▶ Notebook, appendix A

Appendix A · DETR: a fixed set of slots, matched to objects

Back to Part 4's slots: make 100 of them, let each read the whole image, and match slots to objects before the loss, so no NMS is needed.

1. Follow a photo through DETR to 100 predictions, and see where two queries looked.
2. Watch duplicates vanish inside the decoder.
3. Build the matching cost one cell at a time, solve the matching, and compute the loss.

> The other answer to the lecture's question: DETR, 100 slots that read the image with attention, matched to objects instead of cleaned up by NMS.

▶ Notebook, appendix A

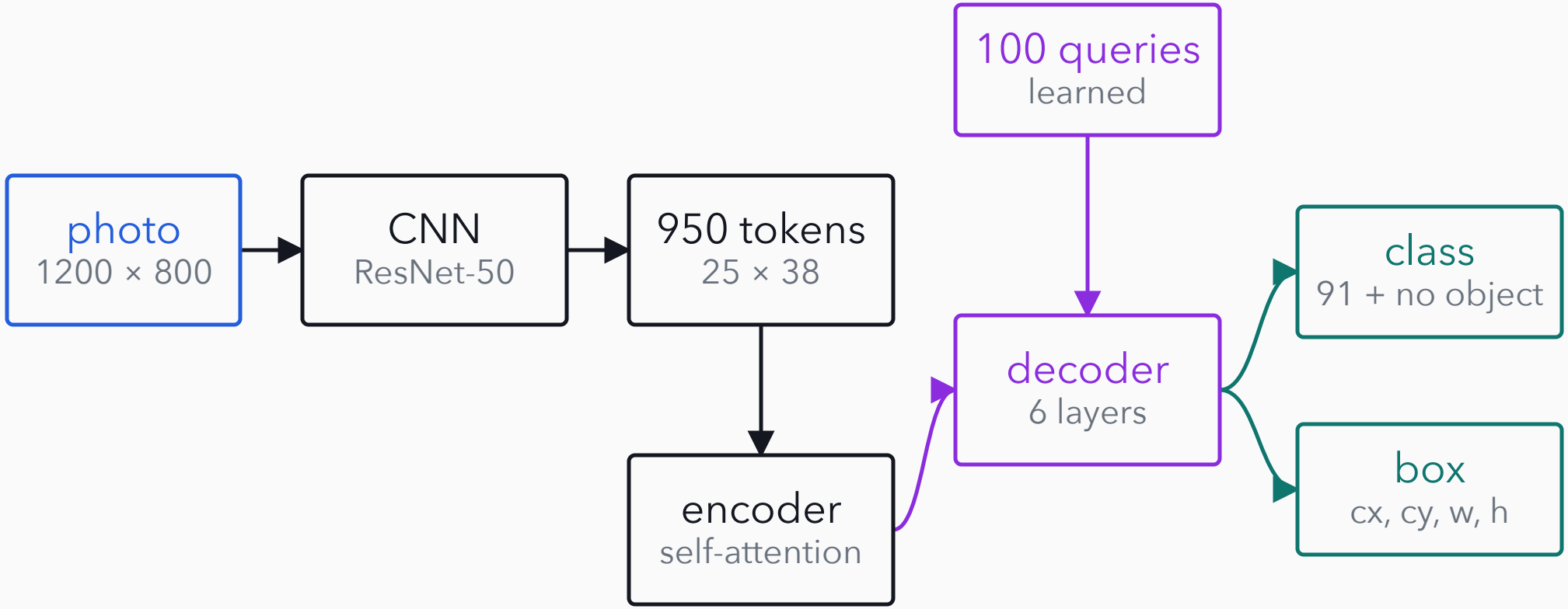
Back to the slots, with three changes

1. **100 slots**, more than almost any photo has objects; each may say “no object”.
2. **Each slot reads the whole image** with cross-attention, as the VLM lectures did.
3. **Match before the loss**: pair slots with objects so that the total cost is lowest.

> Problem 2 was order: before computing the loss, find the cheapest one-to-one pairing of slots and objects. Order in the annotation list stops mattering.

▶ Notebook, appendix A

DETR, from photo to 100 predictions



> Two small heads read each of the 100 outputs: a class among 91 COCO ids plus “no object”, and a box as normalised (c_x, c_y, w, h).

DETR (facebook/detr-resnet-50, Hugging Face) run on this photo by scripts/run_models.py.

▶ Notebook, appendix A

A query is learned once and reused for every photo

Stored: 100 query vectors, 256 numbers each. Parameters, trained like weights; the same 100 for every photo.

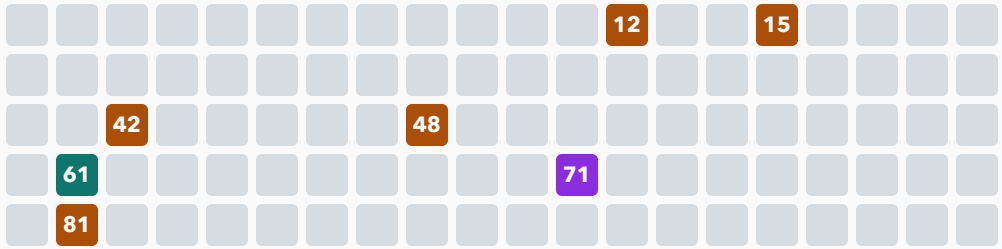
Computed: what each query writes depends on the photo, because it reads the photo through cross-attention.

As in the VLM lectures: stored query slots, computed per-image summaries.

> Their outputs are computed: the same query writes a dog box for one photo and “no object” for another, depending on what it read.

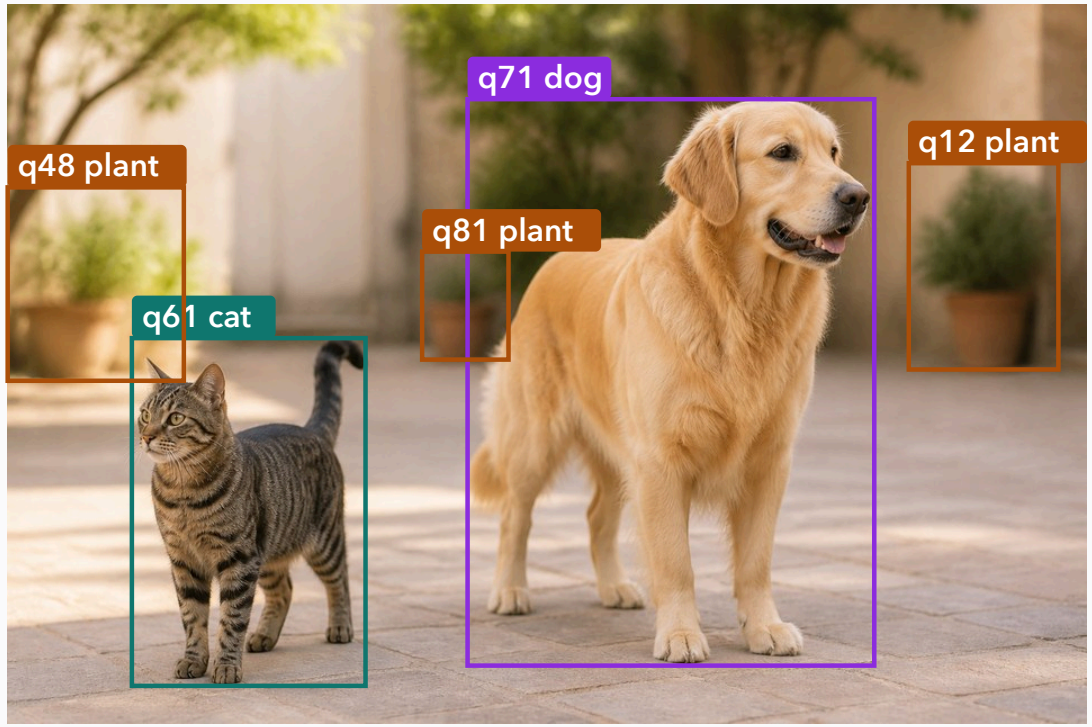
▶ Notebook, appendix A

DETR on the courtyard: 100 slots, most of them empty



Grey: “no object”. 88 of the 100 slots give it more than 0.9.

query	class	p	p(no object)
71	dog	0.999	0.001
61	cat	0.999	0.001
12	potted plant	0.998	0.002
81	potted plant	0.989	0.011
48	potted plant	0.973	0.027



> Their boxes: one per object. No duplicates to remove, and no NMS.

DETR (facebook/detr-resnet-50, Hugging Face) run on this photo by scripts/run_models.py.

▶ Notebook, appendix A

Where two queries looked



query 71 → dog



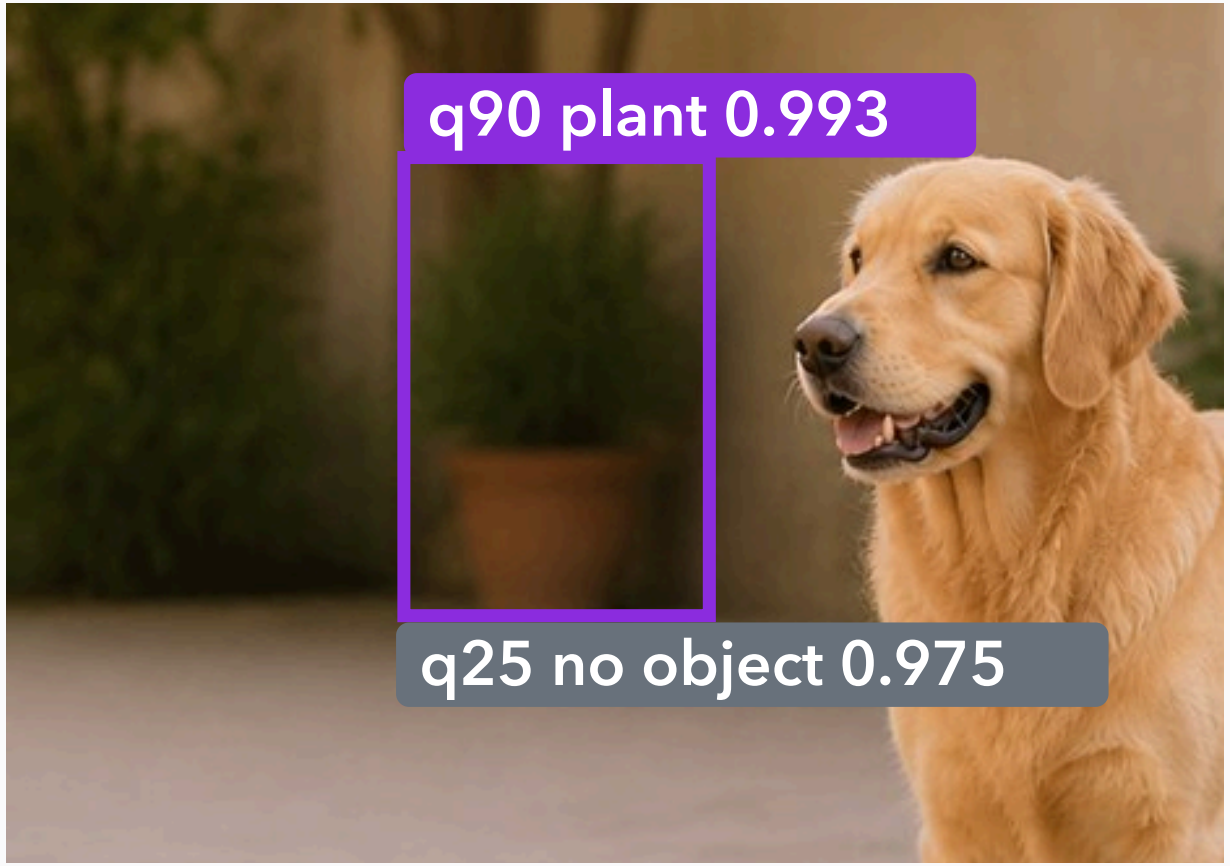
query 61 → cat

> Query 61, the cat: its attention sits on the cat. Each query attends mostly to the parts of the photo that fix its box: the extremities more than the middle.

DETR (facebook/detr-resnet-50, Hugging Face) run on this photo by scripts/run_models.py.

▶ Notebook, appendix A

Duplicates disappear inside the decoder



After layer 2: query 25 says “no object” (0.975); query 90 keeps the plant.

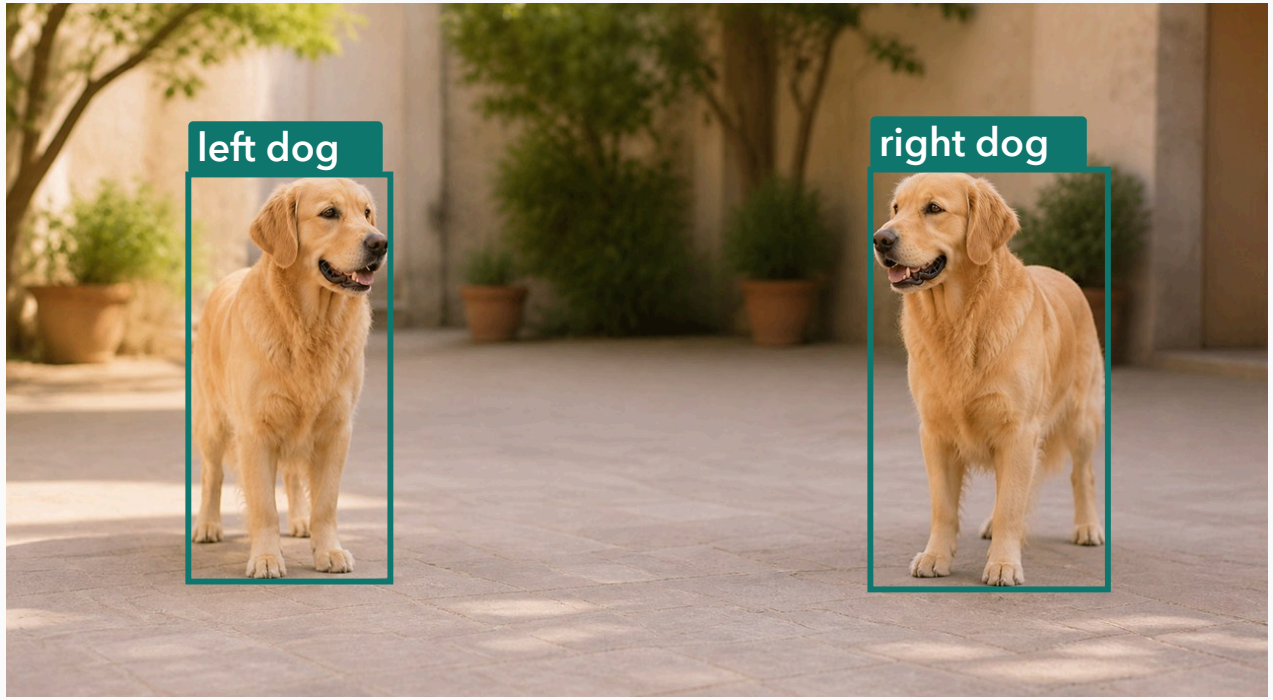
Queries attend to each other in every decoder layer, so one can see that another already has the pot. Training rewards exactly one of them per object (next slides).

> This works because training never rewards two queries for one object. That comes from the matching.

DETR (facebook/detr-resnet-50, Hugging Face) run on this photo by scripts/run_models.py.

▶ Notebook, appendix A

Training: which prediction should learn which dog?



Two targets. 100 predictions. In Part 4, comparing slot k with list entry k punished a correct answer.

Instead: give every (prediction, target) pair a **cost**, then choose the one-to-one pairing with the lowest total.

> Score every pairing with a cost: low when the prediction has the right class and a box close to the target's. Then pick, among all one-to-one pairings, the cheapest.

The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, appendix A

One more measure: generalised IoU



Any two boxes that do not touch have IoU 0: a near miss scores the same as a far miss.

$$\mathrm{GIoU} = \mathrm{IoU} - \frac{\mathrm{area}(C) - \mathrm{area}(A \cup B)}{\mathrm{area}(C)}$$

C: the smallest box enclosing both. The emptier C is, the lower the score; GIoU runs from −1 to 1.

DETR's right-dog query against the left dog: IoU 0, GIoU −0.529.

> Real example: query 98 predicts the right dog; against the left dog its IoU is 0 but its GIoU is −0.529.

▶ Notebook, appendix A

The cost of one pairing: query 61 and the left dog

$$C = -\,p(\text{class}) \;+\; 5\,\left\|\hat{b} - b\right\|_1 \;-\; 2\,\text{GIoU}(\hat{b}, b)$$

$$\text{class:}\;\; -\,p(\text{dog}) = -0.999$$

$$\text{box, L1: } |0.224 - 0.225| + \cdots = 0.004, \quad \times 5 = 0.020$$

$$\text{box, GIoU: } 0.984, \quad \times(-2) = -1.968$$

$$C = -0.999 + 0.020 - 1.968 = -2.95$$

Total −2.95. Low cost = good pairing; negative is fine, only comparisons matter.

DETR (facebook/detr-resnet-50, Hugging Face) run on this photo by scripts/run_models.py.

▶ Notebook, appendix A

Fill the table: the cost of every pairing

	q61 (dog)	q98 (dog)	q90 (plant)	q2 (no object)
left dog	**−2.95**	3.09	6.07	−1.71
right dog	3.08	**−2.90**	4.47	4.26

Cheapest pairing: left dog ↔ q61, right dog ↔ q98, total −5.85. Every other query is matched to nothing and learns “no object”.

> Choose one query per dog with the lowest total: q61 and q98. Here each dog’s cheapest query is different, so the choice is easy. It is not always.

DETR (facebook/detr-resnet-50, Hugging Face) run on this photo by scripts/run_models.py.

▶ Notebook, appendix A

Why not let each object grab its cheapest slot?

	q1	q2
left dog	1	**2**
right dog	**2**	9

Greedy: the left dog takes its cheapest, q1 (1). The right dog must take q2 (9). Total **10**.

Better: left dog → q2 (2), right dog → q1 (2). Total **4**.

Trying every pairing is hopeless with 100 queries. The **Hungarian algorithm** (Kuhn, 1955) finds the cheapest one-to-one pairing in polynomial time: `scipy.optimize.linear_sum_assignment`.

> The best pairing must be found globally. The Hungarian algorithm does it efficiently; scipy has it.

▶ Notebook, appendix A

After matching: the loss

$$\text{q61} \to \text{left dog:} \quad -\log 0.999 + 5 \times 0.004 + 2 \times (1 - 0.984)$$

$$= 0.001 + 0.020 + 0.032 = 0.053$$

$$\text{q2} \to \text{no object:} \;\; 0.1 \times (-\log 0.997) = 0.0003$$

$$\text{q90 (a plant)} \to \text{no object:} \;\; 0.1 \times (-\log 0.003) = 0.581$$

> q90 named a real plant, but plants are not in this target list, so it pays 0.58 for being right.

DETR (facebook/detr-resnet-50, Hugging Face) run on this photo by scripts/run_models.py.

▶ Notebook, appendix A

Why DETR needs no NMS

DETR · one-to-one

- Each object is matched to exactly one query
- A second query on the same object is trained to say “no object”
- So the trained network outputs one box per object

YOLO · one-to-many

- About ten cells per object are trained as positives
- All ten learn to fire
- So NMS must delete nine of them

> The dense detector trained ten cells per object on purpose (more positives, more signal) and pays for it with NMS.

▶ Notebook, appendix A

DETR in code: matching, then inference

```
from scipy.optimize import linear_sum_assignment
cost = (-prob[:, gt_labels]                                  # (100, N)
        + 5 * torch.cdist(pred_boxes, gt_boxes, p=1)
        - 2 * generalized_box_iou(to_xyxy(pred_boxes), to_xyxy(gt_boxes)))
q_idx, gt_idx = linear_sum_assignment(cost.detach())         # one query per object
```

```
model = DetrForObjectDetection.from_pretrained("facebook/detr-resnet-50")
out = model(**processor(images=img, return_tensors="pt"))
dets = processor.post_process_object_detection(out, threshold=0.9,
                                               target_sizes=[img.size[::-1]])
```

> At test time there is nothing to match: run the model, keep the queries whose best class beats a threshold. There is no NMS step.

▶ Notebook, appendix A

Two answers to one question

	dense (YOLOv8)	set (DETR)
outputs	one record per cell: 12,600	100 learned slots
who answers for an object	cells near it, by place	the matched query
training targets	one-to-many: ~10 cells	one-to-one: Hungarian matching
at test time	score filter + NMS	threshold only
on our courtyard	4 detections	5 objects at $p \geq 0.9$
our 8 photos, AP@[.5:.95]	0.936	0.882
COCO validation, AP (published)	0.373	0.420

> Both solve the lecture's question with a fixed tensor plus a way to say "nothing here", and both score well on our photos.

YOLOv1 (2016): one responsible cell per object

$$7 \times 7 \text{ cells} \times (2 \times 5 + 20) = 7 \times 7 \times 30$$

Each cell: two boxes, each (x, y, w, h, confidence), and one set of 20 class probabilities (PASCAL VOC). Only the cell containing an object's centre is responsible for it; x, y are offsets inside that cell, w, h fractions of the image.

Confidence was trained toward the IoU between the box and the truth. A cell could hold at most one object's class.

> Why did YOLOv1 struggle with flocks of birds?

Anchors: predict offsets from template boxes

$$b_x = \sigma(t_x) + c_x, \quad b_y = \sigma(t_y) + c_y, \quad b_w = p_w\, e^{t_w}, \quad b_h = p_h\, e^{t_h}$$

Faster R-CNN, SSD and YOLOv2 to v5 place several template boxes (anchors) of chosen sizes and shapes at every cell, and predict small corrections to each. Positives are anchors with high IoU to an object.

Anchor-free detectors (FCOS, YOLOv8) predict distances from the cell centre instead, as in Part 5.

> What must be chosen by hand for an anchor-based detector?

Two-stage detectors: propose, then classify

Stage 1, a region proposal network: about 2,000 boxes that might contain something.
Stage 2: crop features for each proposal (RoI pooling), classify it and refine its box.

R-CNN (2014) ran a CNN per proposal; Fast R-CNN (2015) shared one CNN pass; Faster R-CNN (2015) learned the proposals too. Mask R-CNN adds a mask per proposal: Lecture 23.

torchvision's fasterrcnn_resnet50_fpn returns post-processed detections: its NMS is already inside.

> What does the first stage output?

Focal loss: quiet the easy background

$$\mathrm{FL}(p_t) = -(1-p_t)^{\gamma}\log p_t, \qquad p_t = \begin{cases} p & y = 1 \\ 1-p & y = 0 \end{cases}$$

A background cell with p = 0.001: $p_t = 0.999$, so with $\gamma = 2$ its loss is multiplied by $(1 - p_t)^2 \approx 10^{-6}$. Easy examples nearly vanish; hard ones keep their weight.

RetinaNet (Lin et al., 2017) made one-stage detectors as accurate as two-stage ones this way.

> Which examples does focal loss down-weight?

Box formats in practice

where	format	the dog
torchvision	corners, pixels	(407, 83, 768, 588)
COCO json	(x, y, w, h), top-left, pixels	(407, 83, 361, 505)
YOLO label file	normalised (c_x, c_y, w, h)	(0.612, 0.524, 0.376, 0.789)
DETR output	normalised (c_x, c_y, w, h)	same as YOLO

> Why do format bugs often go unnoticed?

Resizing and padding move the boxes too

Detectors often resize a photo to a fixed size and pad the rest (letterboxing). The boxes must get the same treatment: scale every coordinate by the resize factor, then add the padding offset.

$$960 \times 640 \;\to\; 640 \times 427 \;(r = \tfrac{2}{3}),\ \text{pad } 106 \text{ px above}: \qquad y' = \tfrac{2}{3}\, y + 106$$

Our photos are multiples of 32 on both sides, so this lecture ran the models without padding.

> What happens to the dog's $y_1 = 83$ after this letterbox?

After DETR

- **Deformable DETR** (2021): each query attends to a few sampled points per feature level; trains about 10× faster.
- **DINO, RT-DETR** (2022 and 2023): better queries and matching; real-time DETRs.
- **YOLOv10** (2024): one-to-many and one-to-one heads; NMS-free YOLO.
- **Open vocabulary** (OWL-ViT, Grounding DINO): classes given as text, matched with CLIP-like embeddings (the language-grounded vision lecture).

> What did DETR's successors mainly improve?